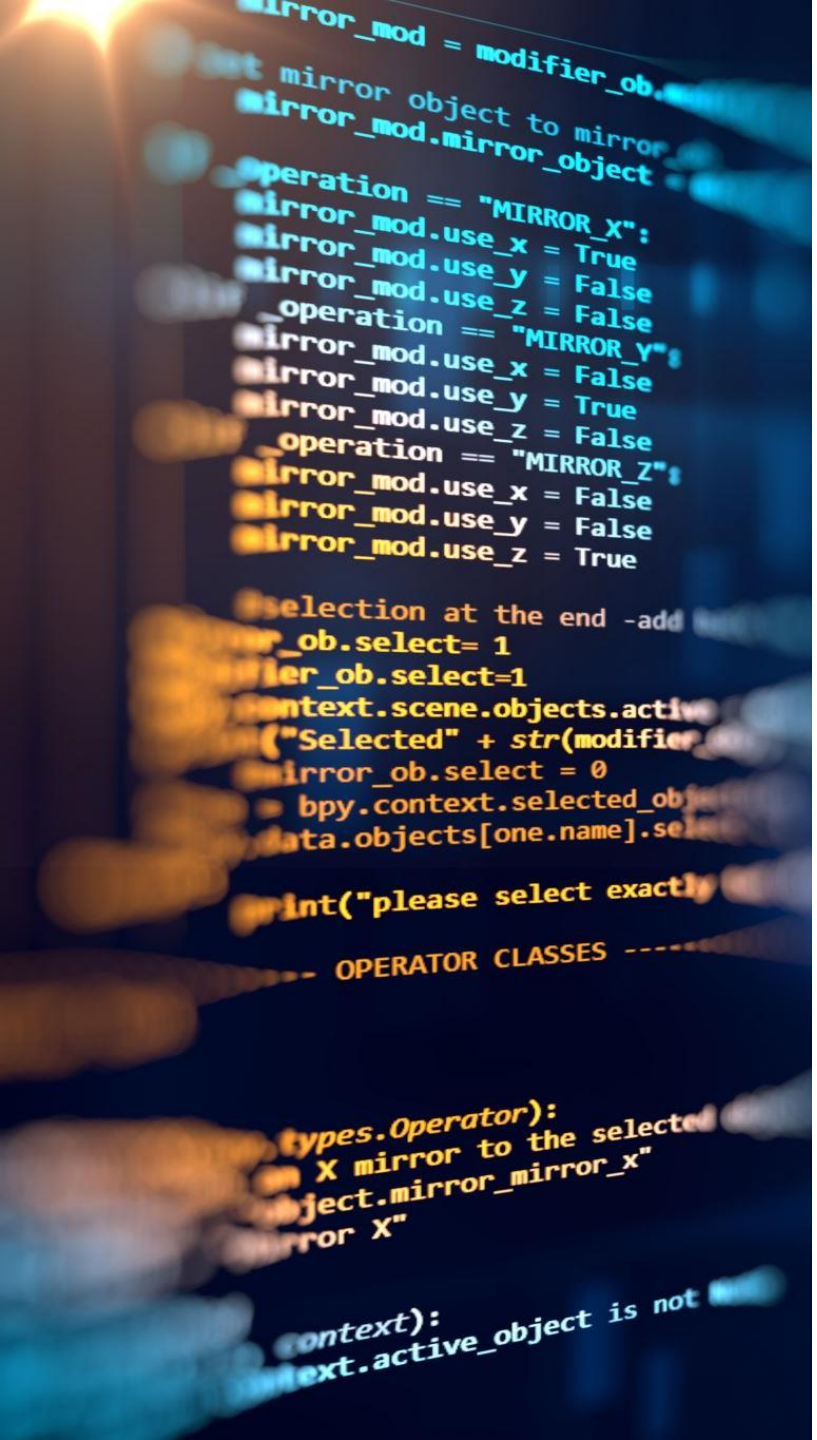


Estrutura de Dados

Fatec – Votorantim.

Profº Me. Rodrigo de Paula Diver

Rodrigo de Paula Diver

Formação Acadêmica

- **Mestrado** em Eletrônica de Potência.
  - UNESP 2023.

- **Especialização** em Empreendedorismo e Inovação Tecnológica nas Engenharias.
  - UNESP 2022.

- **Bacharel** em Engenharia de Computação.
  - FACENS 2013

"Minha carreira é movida pela curiosidade e pelo aprendizado contínuo, atuando como um profissional polivalente que integra competências de diversas áreas da engenharia para desenvolver soluções inovadoras, ao mesmo tempo em que me dedico a ensinar e inspirar."

Contatos

Email: rodrigo.diver@cps.sp.gov.br

Linkedin: www.linkedin.com/in/rodrigo-diver-a8a10264

Lattes: http://lattes.cnpq.br/8144865595329900

Objetivos

- Entender e criar algoritmos de nível não-elementar. Compreender e utilizar estruturas de dados lineares na resolução de problemas.
- Compreender e simular o funcionamento de algoritmos de ordenação.
- Entender e criar aplicações de busca sequencial e busca binária.
- Utilizar as técnicas de resolução de problemas no desenvolvimento de programas.

Ementa

- Alocação dinâmica e ponteiros; Arquivos; Introdução à notação assintótica; Tipos abstratos de dados: conceitos, operações, representações, manipulação, listas, pilhas e filas.
- Estruturas de representação de grafos (matriz de adjacência e de incidência).
- Estruturas para representação de árvores. Árvores binárias e suas aplicações.



Avaliações

- $\boldsymbol{Média = \frac{Prova\ P1 + Prova\ P2 + Laboratório}{3}}$

  - Média maior ou igual a 5,0 estará aprovado.
  - Média menor que 5,0 estará reprovado.



Bibliografia

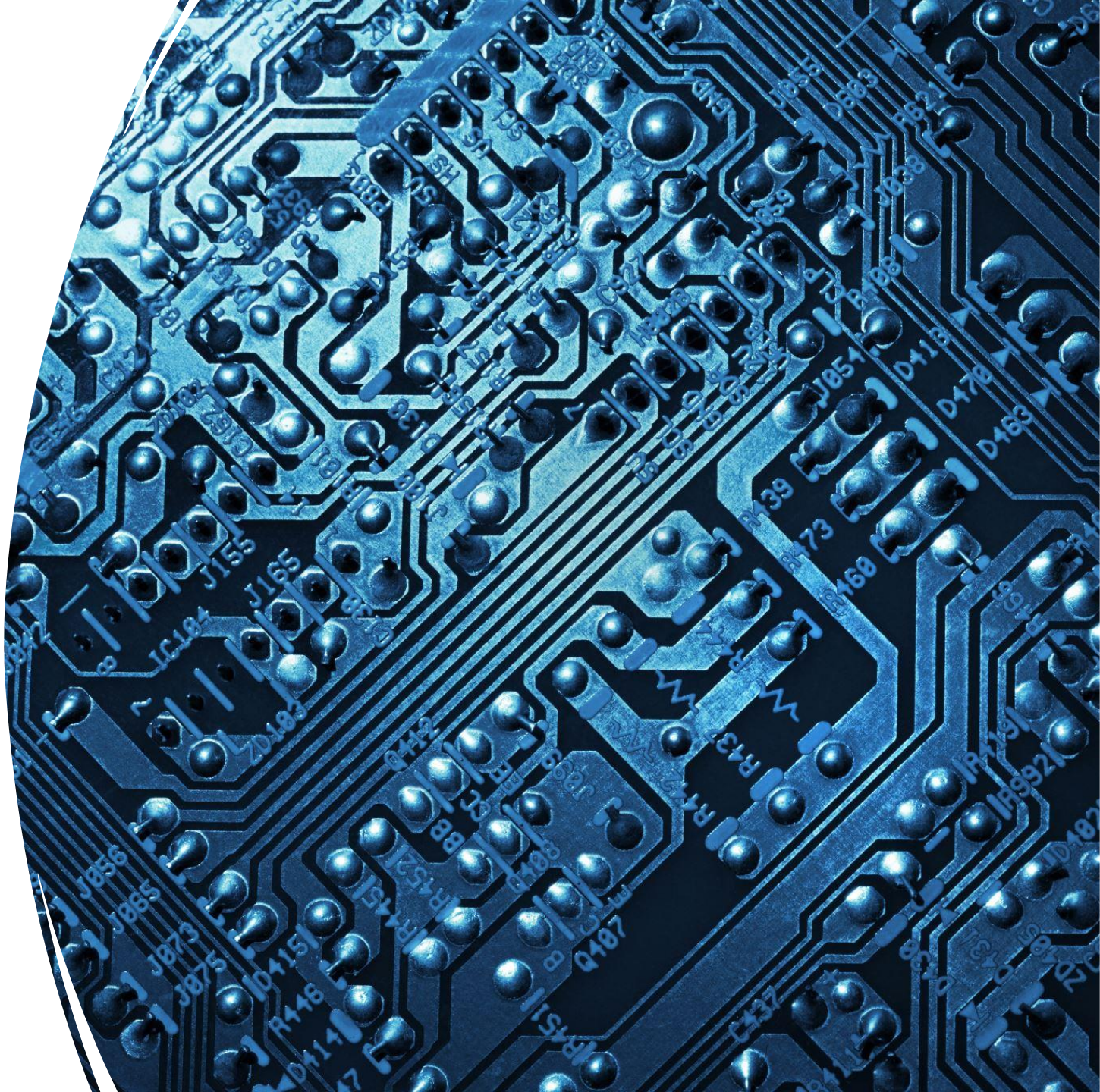
- **Estrutura de Dados e Técnicas de Programação**
  - **Piva D.J.; Nakamiti, G. S.; Bianchi, F. et (2014)**
- **Histórico da Computação e Principais Componentes Computacionais**
  - https://materialpublic.imd.ufrn.br/curso/disciplina/4/14/1
- **Estruturas de Dados em Java (Apostila)**
  - Prof. Dr. Paulo Roberto Gomes Luzzardi (2010).
- **Estruturas de Dados Abertas(Livro)**
  - Pat Morin e Joao Araujo (2021)
- **Algoritmos e Estrutura de Dados em Java (Apostila)**
  - Caelum ensino e inovação. (2021)


O que é um Computador?

O que é um computador?

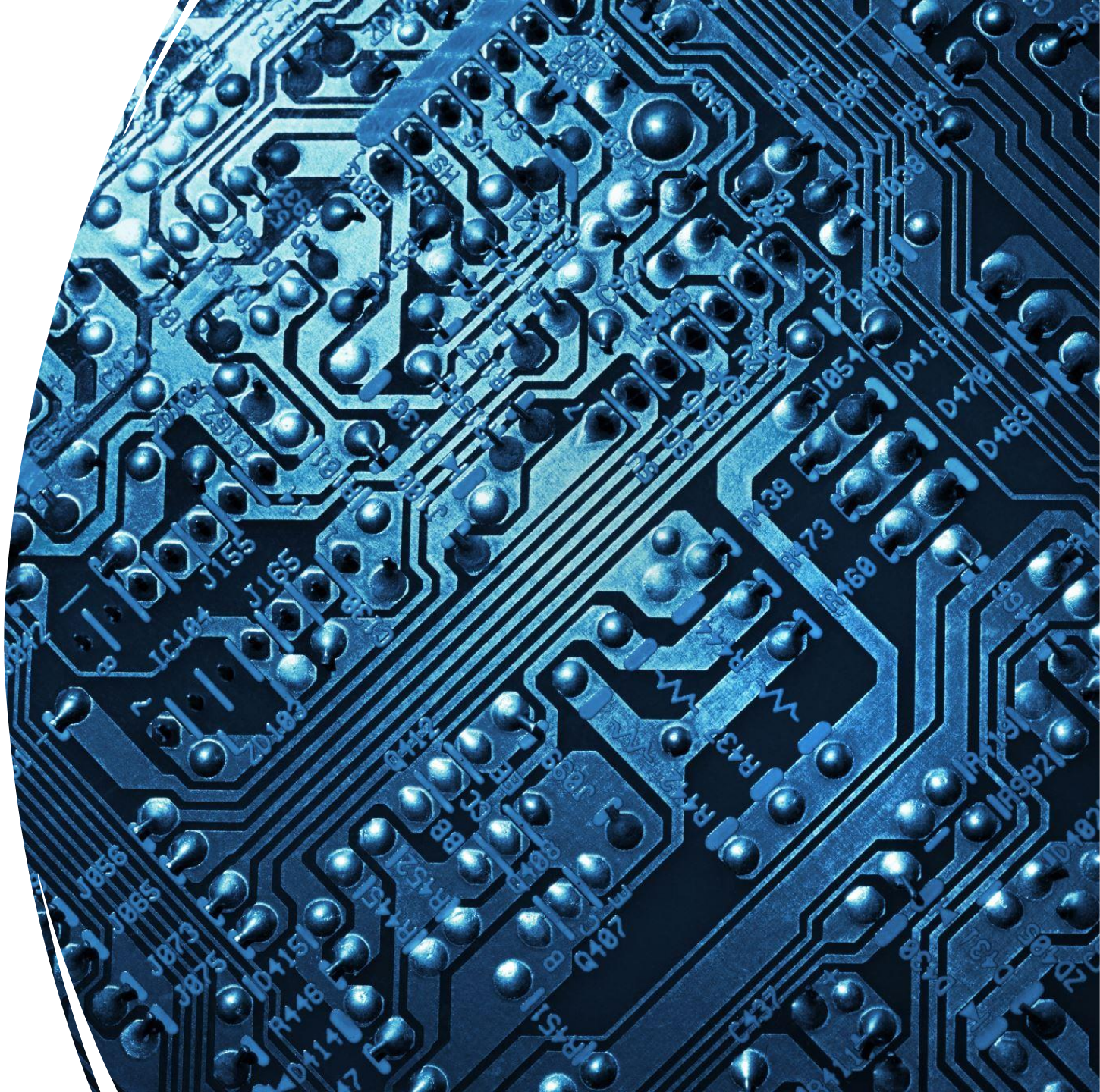
- Um computador é uma **máquina eletrônica** que processa dados.

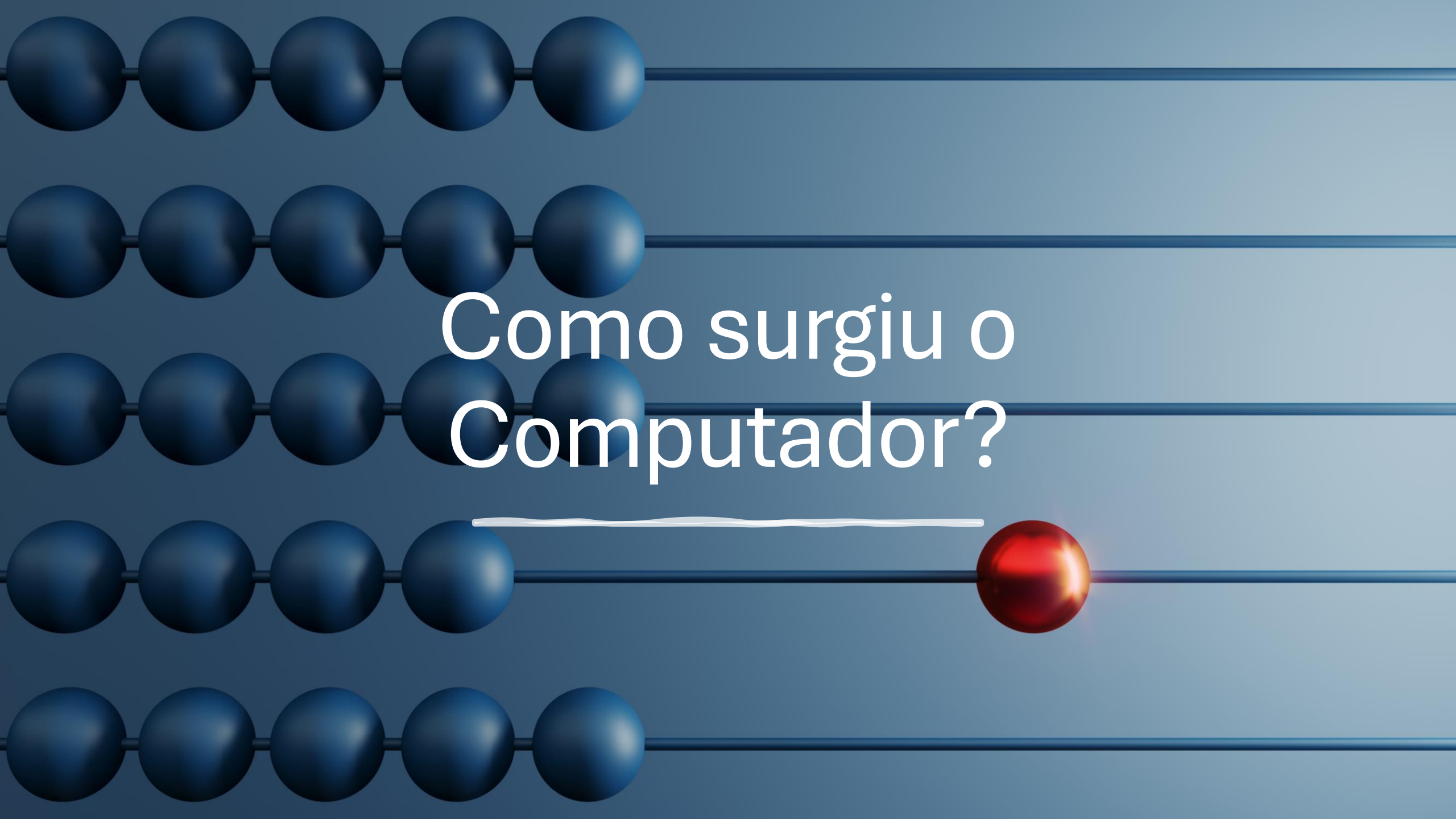
- Ele pode executar tarefas como cálculos, armazenamento de informações, desenho gráfico, entretenimento e muito mais.

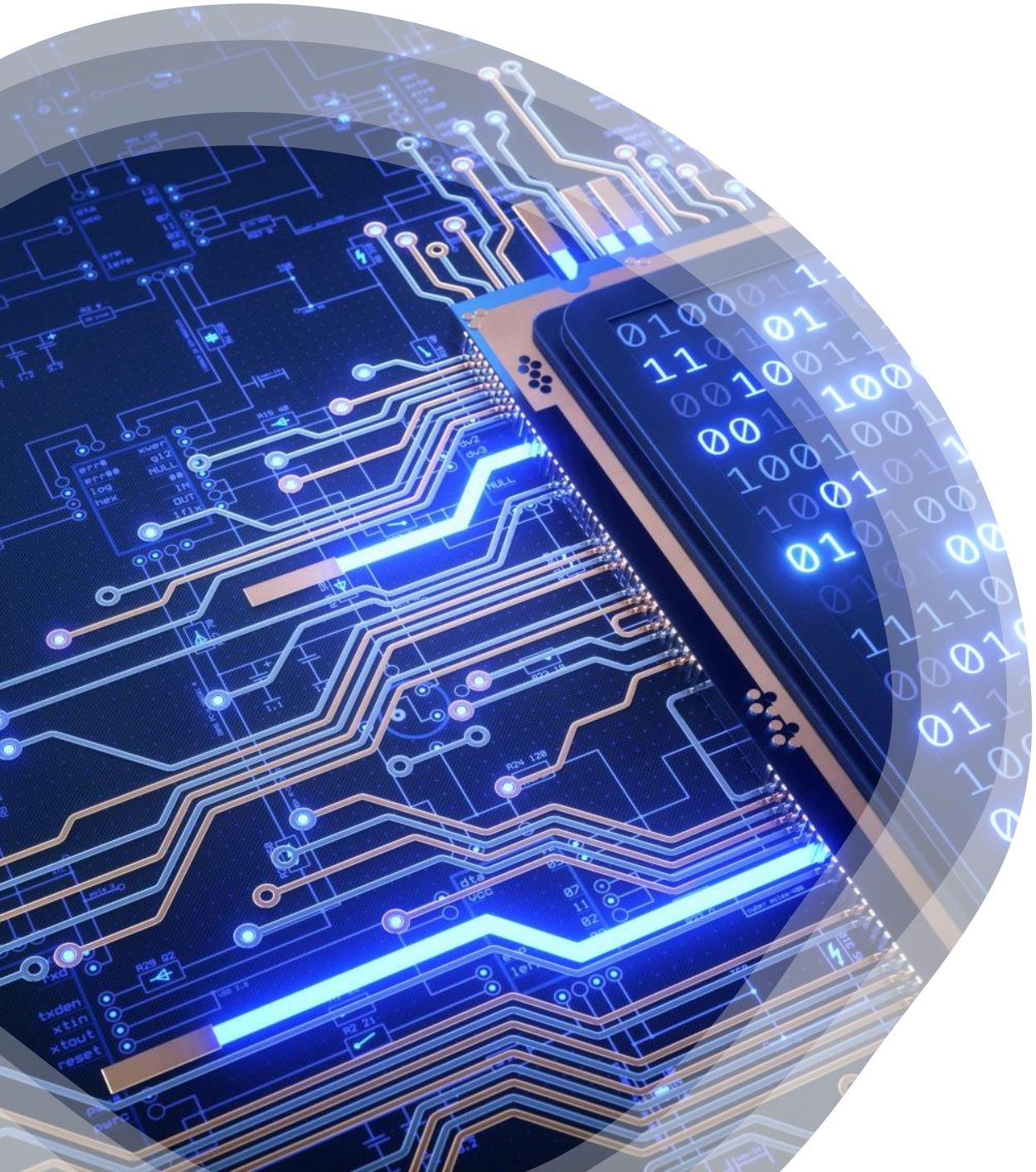


O que é um computador?

- **Dividido em duas partes distintas**
  - ***Software:*** Programas e sistemas operacionais que permitem a execução de tarefas específicas.
  - ***Hardware:*** Conjunto físico de componentes eletrônicos (como processador, memória, disco rígido, etc.).




Como surgiu o
Computador?



História da Computação

- As tecnologias digitais modernas presentes no nosso dia a dia, mesmo que imperceptíveis às nossas atividades, são fruto de muitos anos de dedicação em pesquisa em diversas áreas, como computação, circuitos eletrônicos, lógica de programação etc.

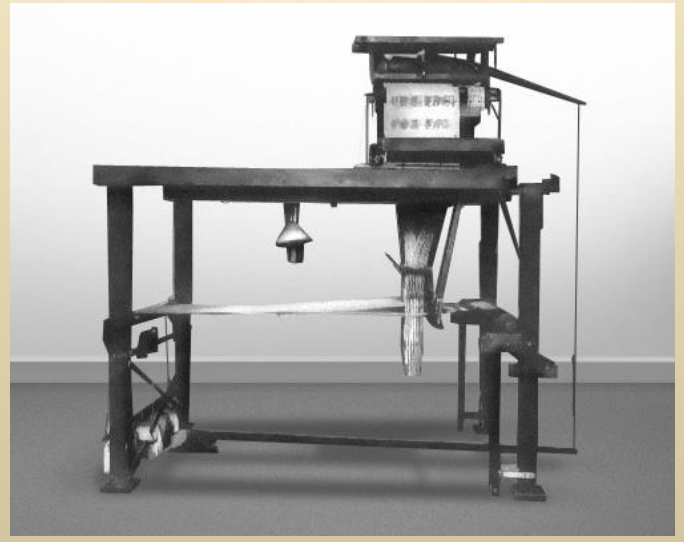
- A partir da teoria dos conjuntos, chamada álgebra booleana (em homenagem a George Boole), pode-se criar as operações lógicas e aritméticas e programar algoritmos em máquinas chamadas computadores.

História da Computação



1623

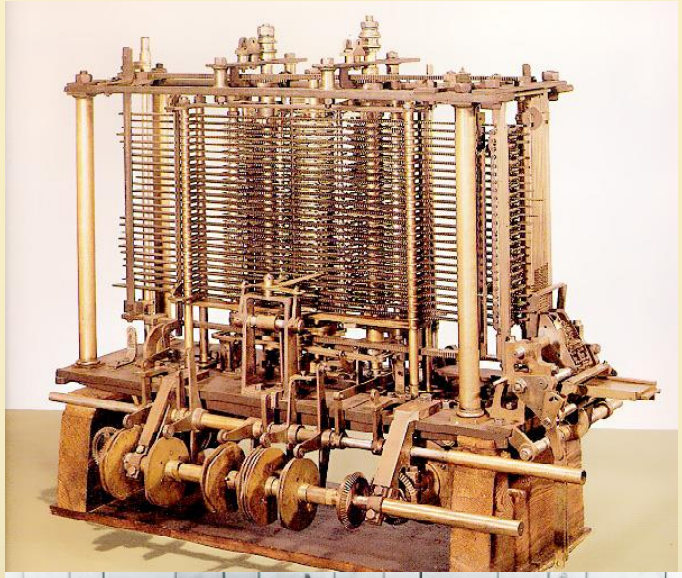
Surgiu a máquina de multiplicar, idealizada por **Wilhmen Schickard**. Com ela realizava-se operações de multiplicação através de rodas dentadas. As multiplicações eram realizadas através de somas sucessivas.



1801

Surge o conceito de programação com a criação do tear automático de **Jacquard**, o qual possuía uma entrada de dados através de cartões perfurados que controlavam a confecção dos tecidos e dos desenhos.

História da Computação



1833

Babbage cria a máquina analítica de **Babbage**. Dispunha de programa, memória, unidade de controle e periféricos de entrada e saída. Essa máquina era capaz de calcular automaticamente tabelas de logaritmos e funções trigonométricas. Devido a esse projeto, Babbage é considerado o pai da informática.



1842

É criada a máquina de recenseamento de **Hollerith**. Ela possui cartões perfurados onde eram armazenadas as informações coletadas no censo e uma máquina para ler e tabular essas informações.

História da Computação



1937

É iniciado a construção do primeiro computador eletromecânico baseado em relés e engrenagens. O **MARK I** possuía unidades de entrada, memória principal e unidade aritmética e utilizava cartões e fitas perfuradas como entrada. **Media 15 x 2,4m e pesava 5 Ton**.



1938

Claude Shannon aplica a Teoria da Álgebra de ***Boole*** na representação de circuitos lógicos, criando os princípios dos circuitos digitais utilizados nos computadores atuais. Em 1940, é criado o computador **ENIAC**, considerado o primeiro computador eletrônico. Pesava **30 Ton** e consumia **100kW** de potência com a capacidade computacional de **400 FLOOPS**.

História da Computação



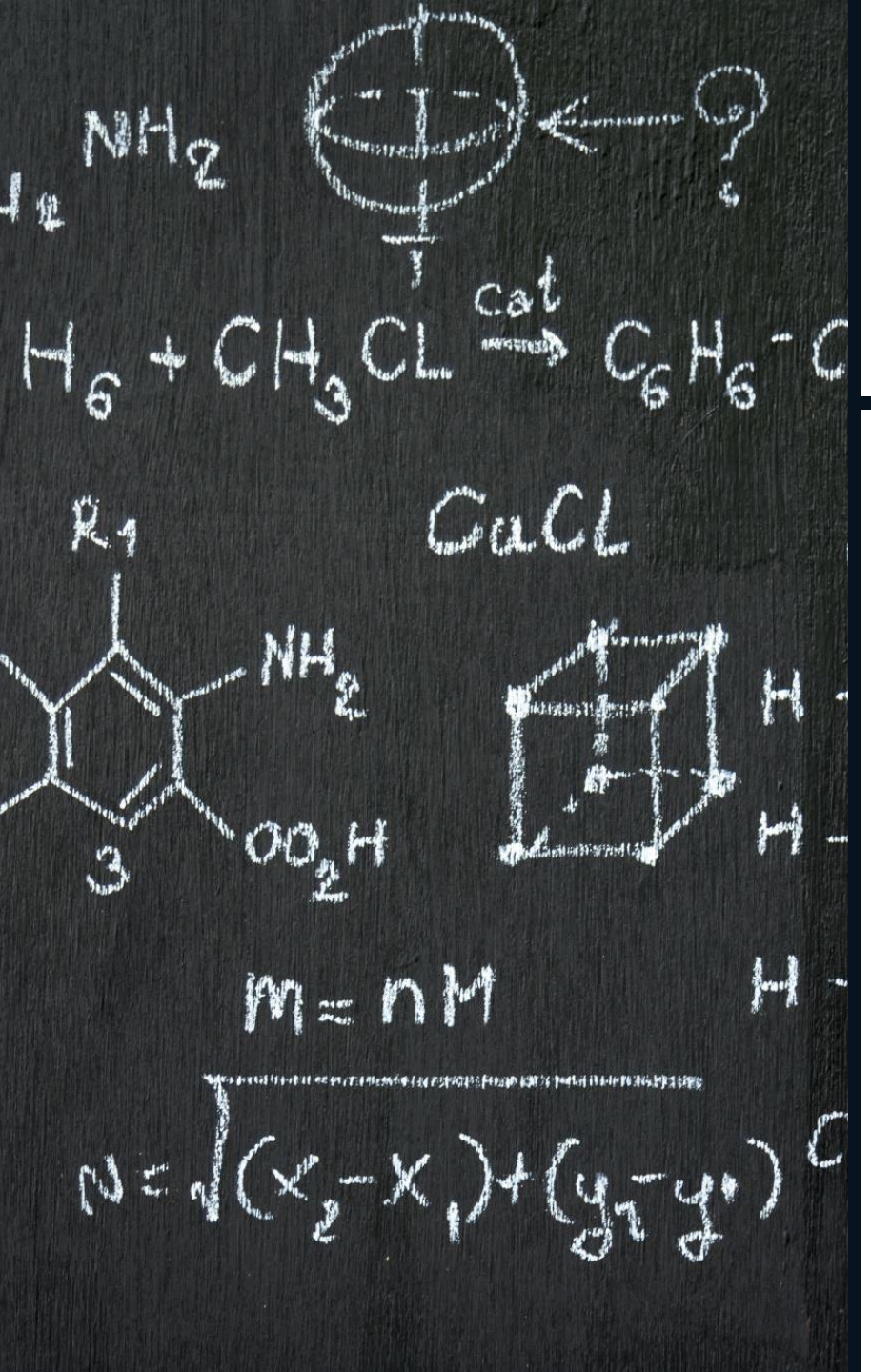
1942

John **Atanasoff** e **Clifford Berry**, do projeto **ENIAC**, construíram uma máquina eletrônica que operava em **código binário**, seguindo as ideias de **Babbage**. Foi o primeiro computador digital.



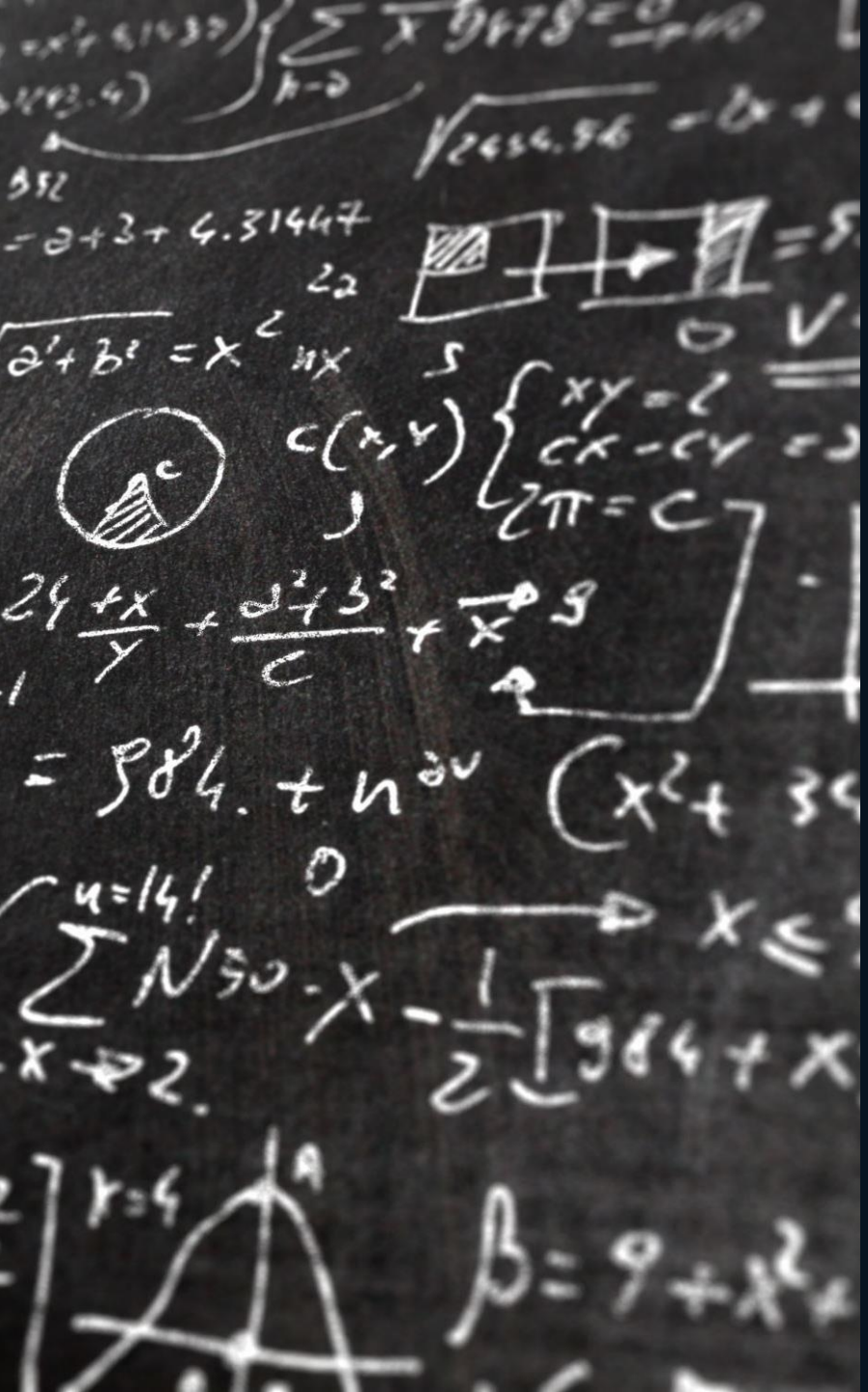
1952

É construído o computador chamado de **EDVAC** (*Electronic Discrete Variable Automatic Computer*), sendo o primeiro computador a utilizar o mesmo espaço de memória para armazenar as instruções e os dados em um unidade de memória central. Ficando conhecida como "**Arquitetura von Neumann**".



Sistemas Numéricos

- **Decimal**
  - Conhecido como sistema Indo-Arábico, utiliza 10 símbolos para representar todos os valores possíveis.
- **Binário**
  - Utilizado por sistemas computacionais, utiliza 2 símbolos 0 e 1 para representar todos os valores possíveis.
- **Hexadecimal**
  - Utilizado para representar números binários de forma mais compacta em sistemas computacionais, sendo composto por 16 símbolos.

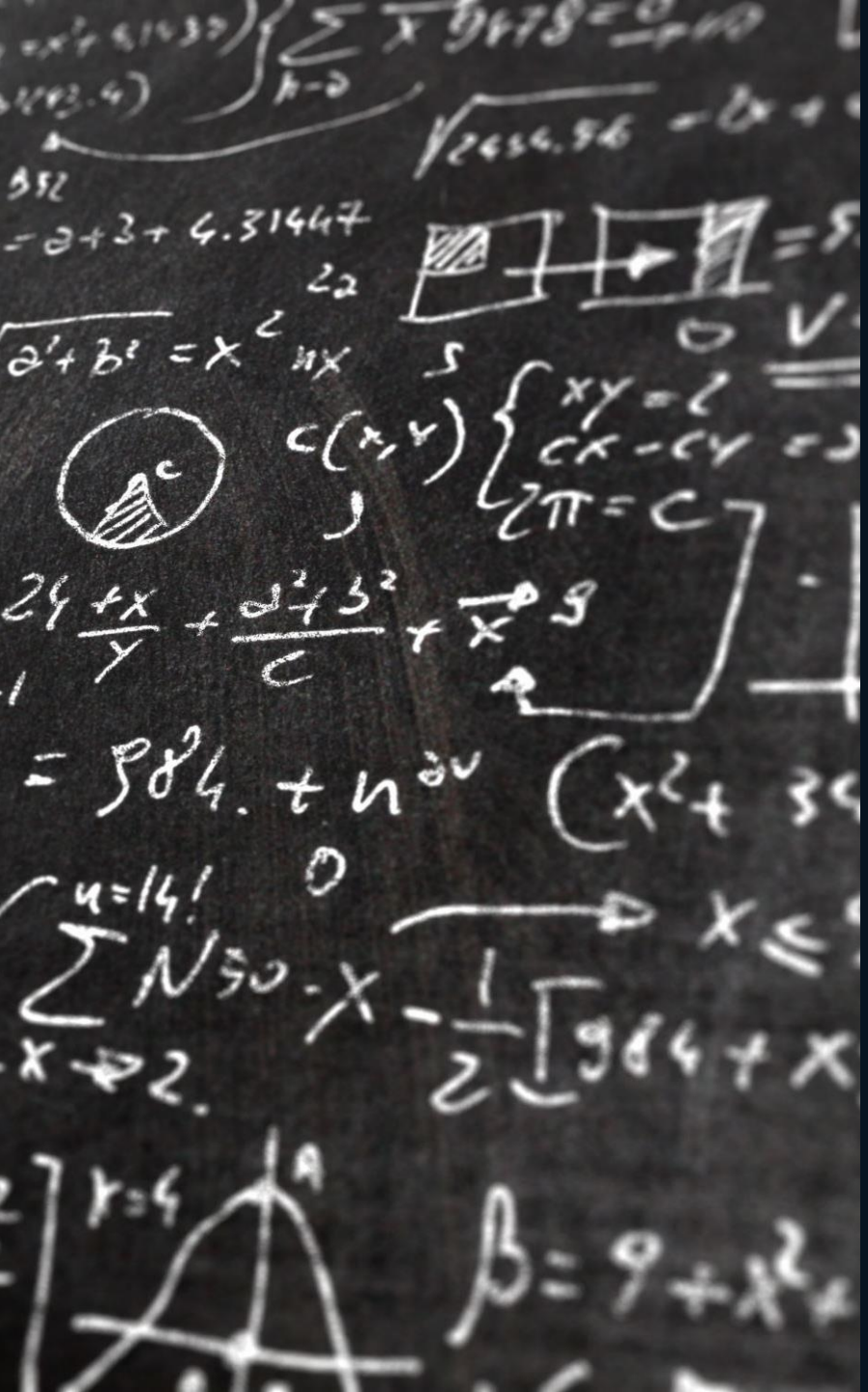


Lógica Booleana

- Toda atividade intelectual humana utiliza em algum nível a lógica, seja para compor sentenças em uma frase ou propor uma nova teoria matemática.

- E os computadores não são diferentes, todo comportamento ou ação que um computador executa é baseado em um conjunto lógico de instruções.

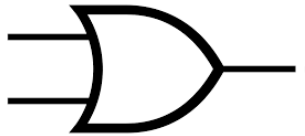
- Em **1854**, **George Boole** introduziu o formalismo que até hoje se usa para o tratamento sistemático da lógica, que permite avaliar se uma sentença é verdadeira ou falsa, chamada de Álgebra Booleana.



Operações Básicas da Álgebra Booleana

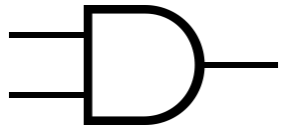
- Na álgebra Booleana, existem três operações ou funções básicas.

- **Operação OU** (Adição Lógica).



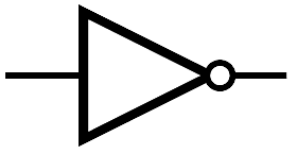
Entrada		S
0	0	0
0	1	1
1	0	1
1	1	1

- **Operação E** (Multiplicação Lógica).



Entrada		S
0	0	0
0	1	0
1	0	0
1	1	1

- **Inversão** ou negação.

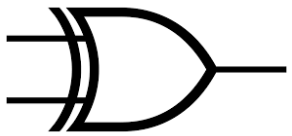


	S
0	1
1	0

Combinação de Portas Lógicas

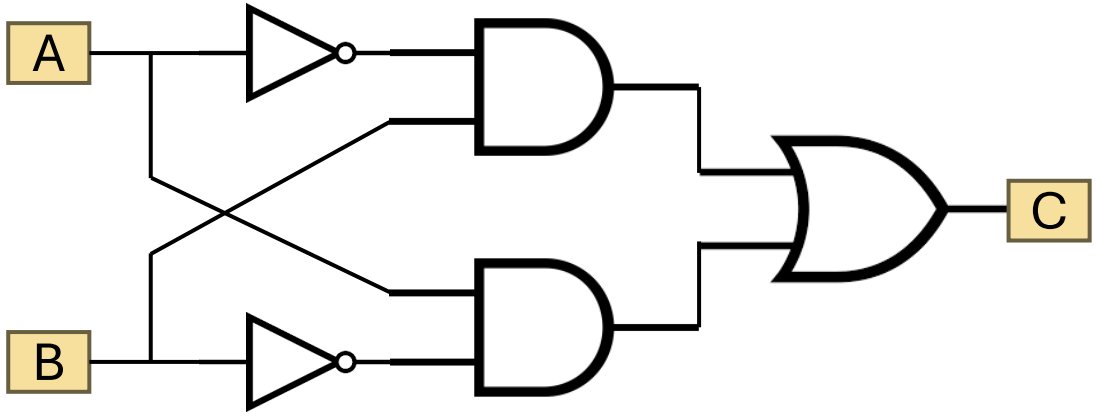
- Combinando as portas OU, E , INVERSORA é possível criar a porta lógicas OU exclusivo.

- Porta Lógica XOR (OU exclusivo)



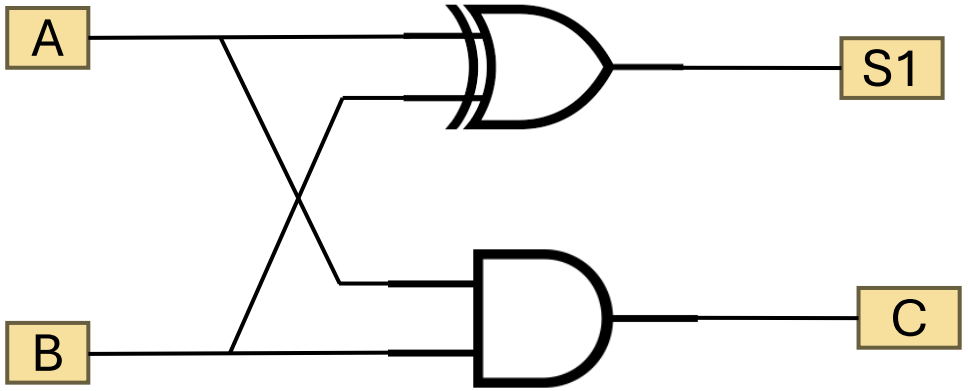
Entrada		S
0	0	0
0	1	1
1	0	1
1	1	0

Circuito Equivalente XOR



Bloco Meio Somador

- Combinando uma **porta E** com uma **XOR** é possível construir um bloco capaz de somar 2 números com 1 bit.
- Conhecido como **Meio Somador.**



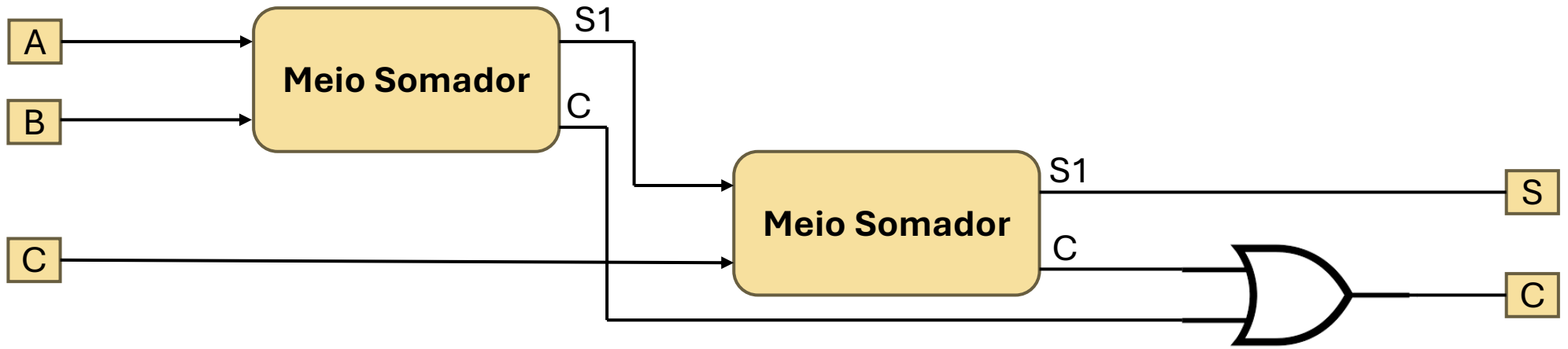
Circuito Equivalente Meio Somador

Entrada		**Saída**	
A	B	C	S1
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

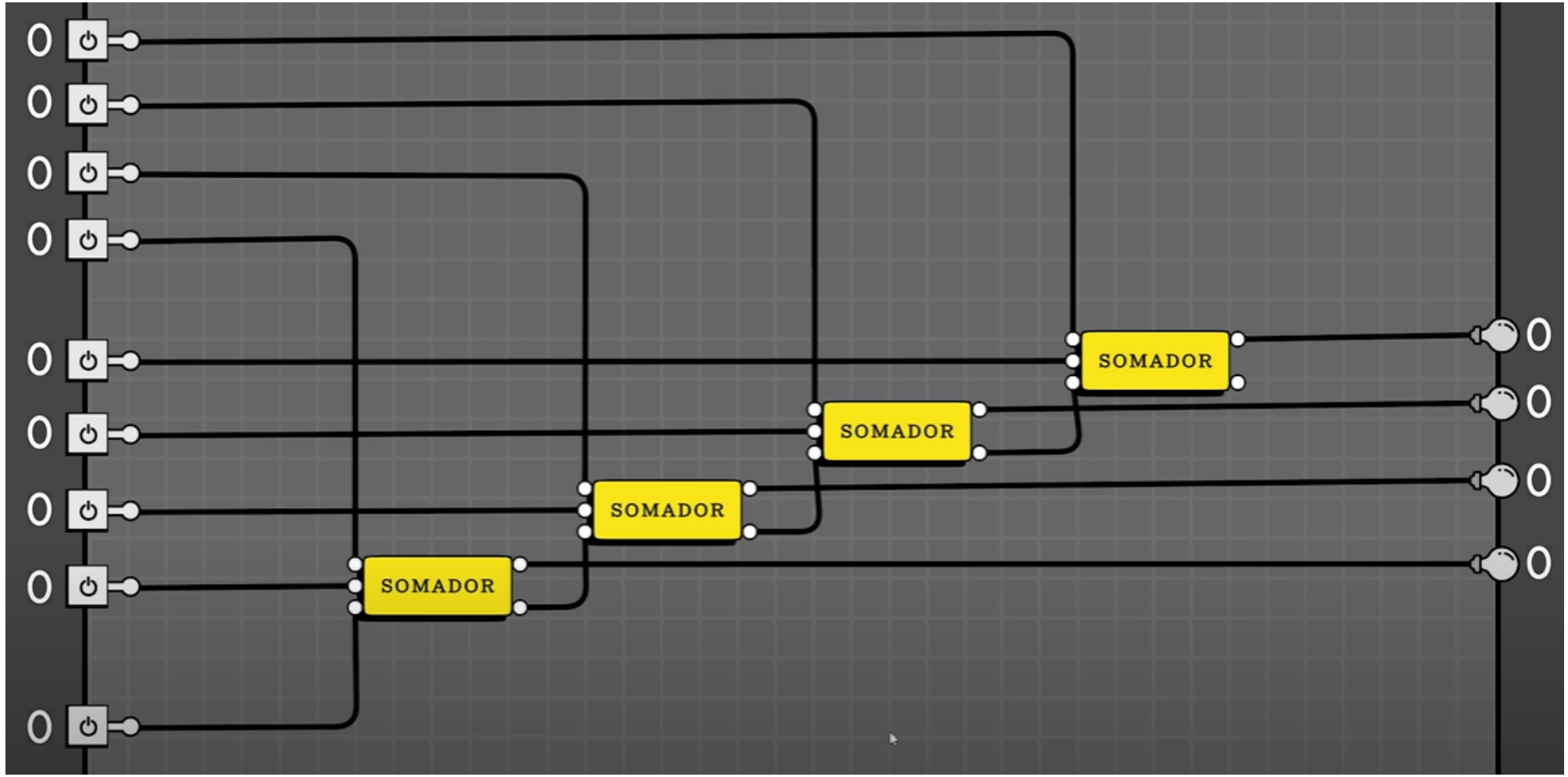
Bloco Somador Completo

- Combinando dois blocos Meio Somador e uma porto OU, obtemos um somar completo
- Que é capaz de somar com 3 bits de entrada.

Circuito Equivalente Somador Completo



Bloco Somador 4 bits



AlanTuring



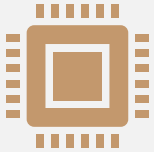
Máquina de Turing



Alan Turing, um matemático britânico, concebeu a **Máquina de Turing** em **1936**.



Naquela época, ele imaginou um dispositivo teórico capaz de realizar qualquer cálculo matemático que um ser humano pudesse fazer.



A Máquina de Turing é um modelo abstrato de um computador, focando apenas nos aspectos lógicos do funcionamento, como memória, estados e transições, sem se preocupar com a implementação física.

Máquina de Turing



Turing aplicou conceitos teóricos desenvolvidos para sua máquina universal na construção de máquinas físicas para quebrar códigos secretos das comunicações alemãs durante a Segunda Guerra Mundial.



Ela se tornou a base para o desenvolvimento dos primeiros computadores digitais e é aplicável até os dias atuais.



Em resumo, a Máquina de Turing é um marco fundamental na história da computação e influenciou profundamente o desenvolvimento dos computadores modernos.

Máquina de Turing

1. Inicialização: A máquina começa no estado inicial, o cabeçote está em uma determinada posição na fita que contém a entrada.

2. Leitura: O cabeçote lê o símbolo na célula da fita onde está posicionado.

3. Transição: Com base no estado atual e no símbolo lido, a máquina consulta a tabela de ação (ou função de transição) para determinar o próximo estado, o novo símbolo a ser escrito na célula atual e a direção para mover o cabeçote (esquerda ou direita).



Máquina de Turing

4. Escrita e Movimento: A máquina escreve o novo símbolo na célula atual, substituindo o símbolo antigo. Em seguida, move o cabeçote para a esquerda ou para a direita conforme determinado pela tabela de ação.

5. Repetição: A máquina repete os passos 2-4 até que não haja uma ação definida na tabela de ação para a combinação atual de estado e símbolo lido.

6. Parada: Quando não há ação definida na tabela de ação para a combinação atual de estado e símbolo lido, a máquina para.



John Von Neumann

- **John Von Neumann** foi um matemático húngaro de origem judaica, naturalizado americano nos anos 30 do século XX. Nasceu em **28 de dezembro de 1903** e fez contribuições importantes em **Mecânica Quântica, Teoria dos Conjuntos, Ciência da Computação, Economia e Teoria dos Jogos**.

Arquitetura de Von Neumann

- Von Neumann propôs um modelo que separava o **hardware** (componentes físicos) do **software** (instruções e programas).
- Com as **instruções armazenadas na memória do computador**, permitindo que o processador as buscasse e executasse sequencialmente.
- Esse modelo influenciou a arquitetura de **quase todos os computadores modernos**.

Contribuições para a Ciência da Computação

- **Máquina de Estados Finitos**: Von Neumann trabalhou em teoria de autômatos e máquinas de estados finitos, que são fundamentais para a compreensão da computação.
- **Lógica e Algoritmos**: Ele contribuiu para o desenvolvimento de algoritmos e técnicas de programação.
- **Teorema da Parada**: Von Neumann também estudou problemas de decisão e formalizou o conceito de algoritmo.

ENIAC e EDVAC

- Von Neumann foi um dos **construtores do ENIAC (Electronic Numerical Integrator and Computer)**, um dos primeiros computadores eletrônicos.
- Ele também trabalhou no projeto do **EDVAC (Electronic Discrete Variable Automatic Computer)**, que incorporou o modelo de Von Neumann.

Contribuições Além da Computação

- Von Neumann aplicou conceitos matemáticos à **economia**, **teoria dos jogos** e **ciência da decisão**.
- Seu trabalho em **mecânica quântica** e **teoria dos conjuntos** também teve impacto.
- Em resumo, John Von Neumann desempenhou um papel fundamental na definição dos fundamentos da computação moderna e sua abordagem continua a moldar nossa compreensão e prática da ciência da computação.

Arquiteturas de Computadores.

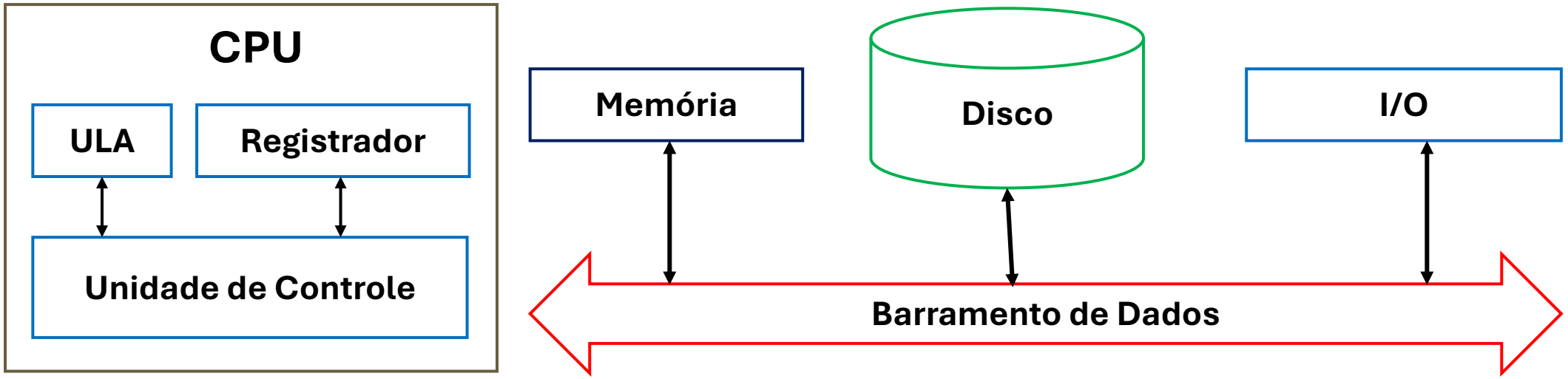
Von Neumann

- *Memória de instruções e dados em uma única estrutura, que compartilha o mesmo barramento de comunicação.*
- *Conjunto de instruções maior e mais complexo, conhecido como CISC.*
- *Utilizado nas família de processadores X86, comuns em computadores em geral.*

Harvard

- *Memória de instruções e dados em estruturas separadas, com dois barramento de comunicação.*
- *Conjunto de instruções menor e menos complexo, conhecido como RISC.*
- *Utilizado em microcontroladores de uso especifico como DSPs e em processadores para uso em dispositivos móveis como a família ARM.*

Estrutura Básica de um Computador



- O **Intel 4004** foi o primeiro microprocessador comercial programável lançado em **1971**, com **registradores de 4 bits** e conjunto de **46 instruções**.
- A **Intel** prosseguiu com o desenvolvimento de microprocessadores com capacidade cada vez maiores, resultando na **família X86**, derivada do microprocessador **8086**.
- Com o aumento do tamanho dos registradores, aumenta a capacidade de manipular valores numéricos maiores aumentando o desempenho do processador.

Gerações de computadores.

Primeira geração (1940-1952)

- Válvulas a vácuo, aplicações científicas e militares.

Segunda geração (1952-1964)

- Substituição da válvula pelo transistor.
- Surgimento da linguagem montadora e algumas de alto nível: COBOL, ALGOL e FORTRAN.

Terceira geração (1964-1971)

- Surgimento do circuito integrado em 1964.
- Tecnologias de integração.
- Evolução do *software* e dos sistemas operacionais.
- Multiprogramação, tempo real e modo interativo.
- Utilização de discos magnéticos e memórias em silício.

Quarta geração (1971- ...)

- Surgimento do microprocessador em 1971.
- Microcomputadores e computadores pessoais.

Gerações de computadores.

Anos 1990

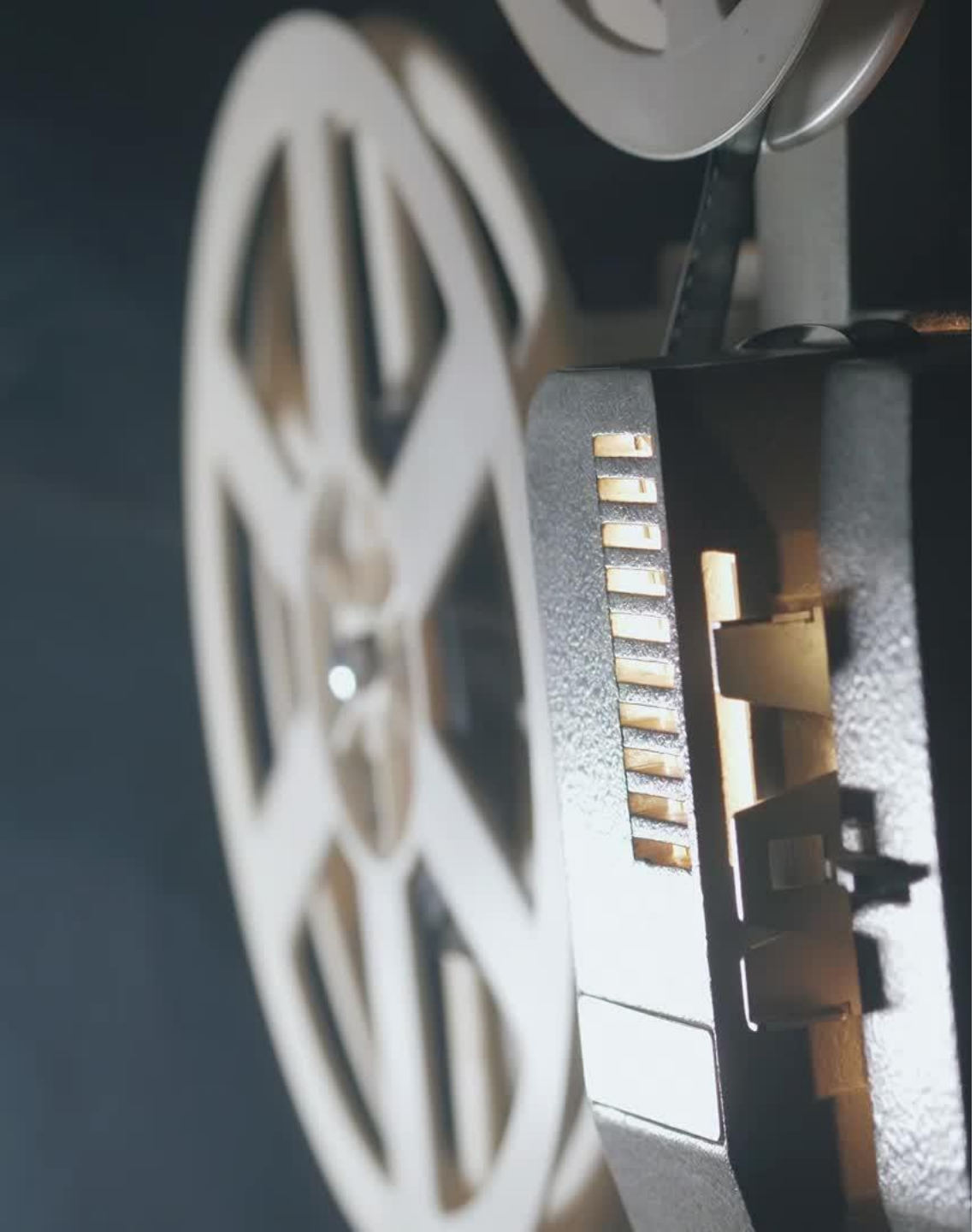
- Desenvolvimento da engenharia de *software*.
- Orientação a objetos, *frameworks*...
- Interfaces gráficas.
- Multimídia.

Anos 2000

- Grande capacidade de armazenamento.
- Aplicações mais realistas: computação gráfica avançada.
- Avanço da internet e de aplicações baseadas em rede (redes sociais, comércio eletrônico, comunicação instantânea, como chats e messengers, blogs etc).

Anos 2020

- Internet das coisas (IoT).
- Expansão no uso de IA, algoritmos de aprendizado profundo, redes neurais convolucionais (CNNs) e redes neurais recorrentes (RNNs).
- Big Data e computação em nuvem.



Para assistir.

- **O Computador de Turing e Von Neumann | Por que calculadoras não são computadores?**
- https://www.youtube.com/watch?v=G4MvFT8TGII&t=66s

- **Como Reinventar o Computador do Zero**
- https://www.youtube.com/watch?v=BbnDmeNojFA&t=3598s

- **O Jogo da Imitação**
- https://www.youtube.com/watch?v=YIkKbMcJL_4

Estrutura de Dados

Aula 2

- **Introdução ao Java**
- **Vetores e Matrizes**
- **Alocação de Memória**

FATEC – VOTORANTIM

Prof° Rodrigo Diver

Linguagem JAVA

- O Java como linguagem de programação nasceu em 1991 pelas mãos de um pesquisador da Sun Microsystems, James Gosling.

- A ideia original do projeto era desenvolver uma linguagem de programação que fosse independente de plataforma e que pudesse ser executada em diferentes dispositivos eletrônicos, como geladeiras, televisões ou fogões.

- Inicialmente, o projeto não foi prioridade para a Sun, já que alguns executivos acharam absurda a ideia de ter uma linguagem que pudesse ser processada numa televisão. Com a chegada da TV digital interativa, podemos ver que não estavam tão errados assim...

- Sendo uma linguagem de programação orientada a objetos, robusta, elegante, podendo ser utilizada nos mais diversos ambientes.

Máquina Virtual Java (JVM)

- A forma de execução da linguagem Java é baseada na interpretação por meio de uma máquina virtual, a **Java Virtual Machine (JVM).** Ela proporciona um ambiente de aplicação completo para execução de aplicativos Java.

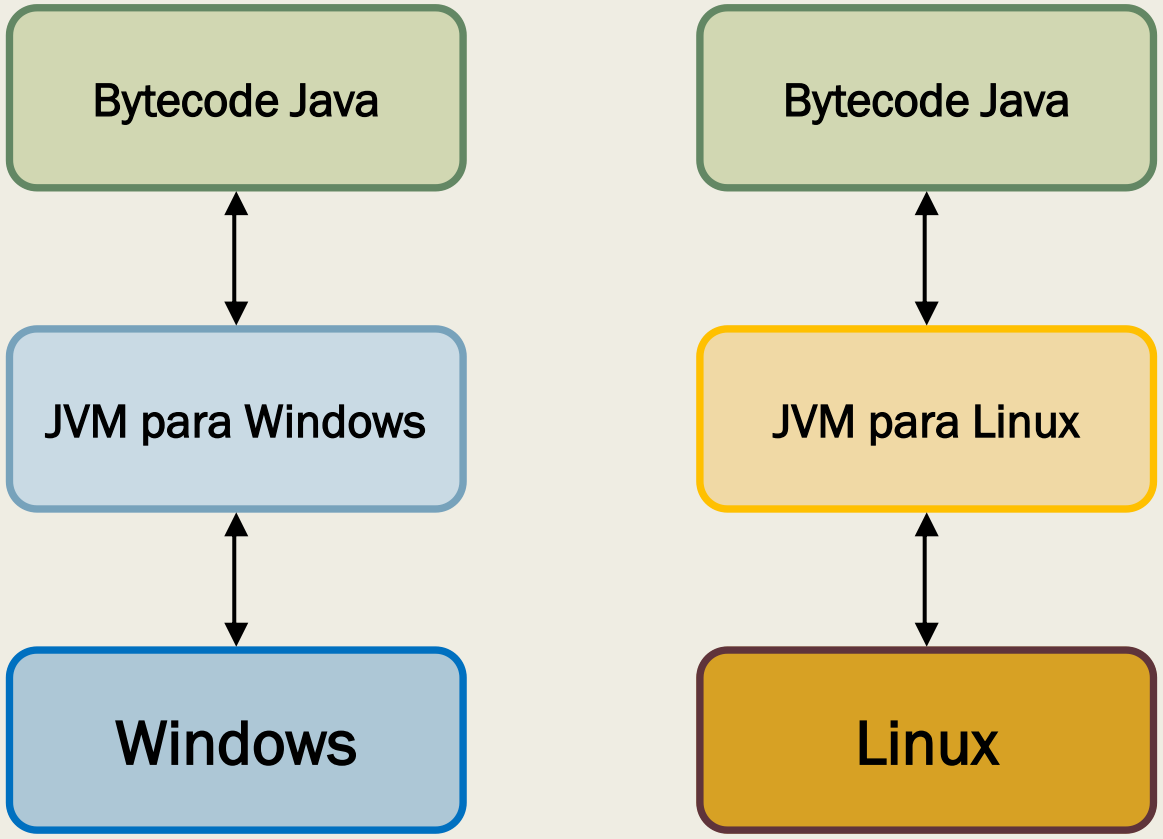
- **Java Virtual Machine (JVM)**
  - *Imagine uma máquina criada por meio de um software emulador dentro de uma máquina real, como se fosse um minicomputador dentro do seu próprio computador, e que possui gerenciamento de memória, cache, threads e outros processos de um computador com seu sistema operacional.*

- A compilação de um código fonte Java é o ***bytecode***, que é uma linguagem de máquina inteligível para a **JVM**.

- O **bytecode** é independente de hardware; assim, basta o computador ou o dispositivo eletrônico ter o interpretador adequado (a **JVM**) que poderá executar um programa Java.

Máquina Virtual Java (JVM)

- *O conceito de máquina virtual garante a independência do código em relação ao sistema operacional.*
- *E aumenta a segurança da aplicação, principalmente em servidores.*
- *O Bytecode é gerado por um compilador Java, como o **javac**.*
- *A **JVM** compila dinamicamente durante a execução do código, podendo aplicar estratégias de otimização.*



Estrutura de Dados

- **Estrutura de Dados**, é um modo particular de armazenamento e organização de dados em um computador, de modo que possam ser usados eficientemente, facilitando sua busca e modificação.

- Utilizados nas mais diversas áreas do conhecimento e com os mais diferentes propósitos de aplicação.

- As técnicas e estruturas que permitem armazenar os dados de forma ordenada e organizada, aumentam o desempenho dos algoritmos, além de minimizar o uso de memória e processamento.

Unidade básica de armazenamento

Bits	Bytes	Valores Decimais
8 bits	1 byte	-128 a 127
16 bits	2 bytes	- 32.768 a 32.767
32 bits	4 bytes	-2.147.483.648 a 2.147.483.647
64 bits	8 bytes	18.446.744.073.709.551.616

Tipos Primitivos em Java

		Valores possíveis				
Tipos	***Primitivo***	***Menor***	***Maior***	***Valor Padrão***	***Tamanho***	***Exemplo***
Inteiro	byte	-128	127	0	8 bits	byte ex1 = (byte)1;
	short	-32768	32767	0	16 bits	short ex2 = (short)1;
	int	-2.147.483.648	2.147.483.647	0	32 bits	int ex3 = 1;
	long	-9.223.372.036.854.770.000	9.223.372.036.854.770.000	0	64 bits	long ex4 = 1l;
Ponto Flutuante	float	-1,4024E-37	3.40282347E + 38	0	32 bits	float ex5 = 5.50f;

	double	-4,94E-307	1.79769313486231570E + 308	0	64 bits	double ex6 = 10.20d;
						ou
						double ex6 = 10.20;

Caractere	char	0	65535	\0	16 bits	char ex7 = 194; ou char ex8 = 'a';
Booleano	boolean	false	true	false	1 bit	boolean ex9 = true;

Vetores

- **Considerem o seguinte problema:** *ler uma certa quantidade de valores inteiros e os imprimir na ordem inversa da leitura.*
    - *Entrada (2, 5, 3, 4, 9)* ➔ *Saída (9, 4, 3, 5, 2)*

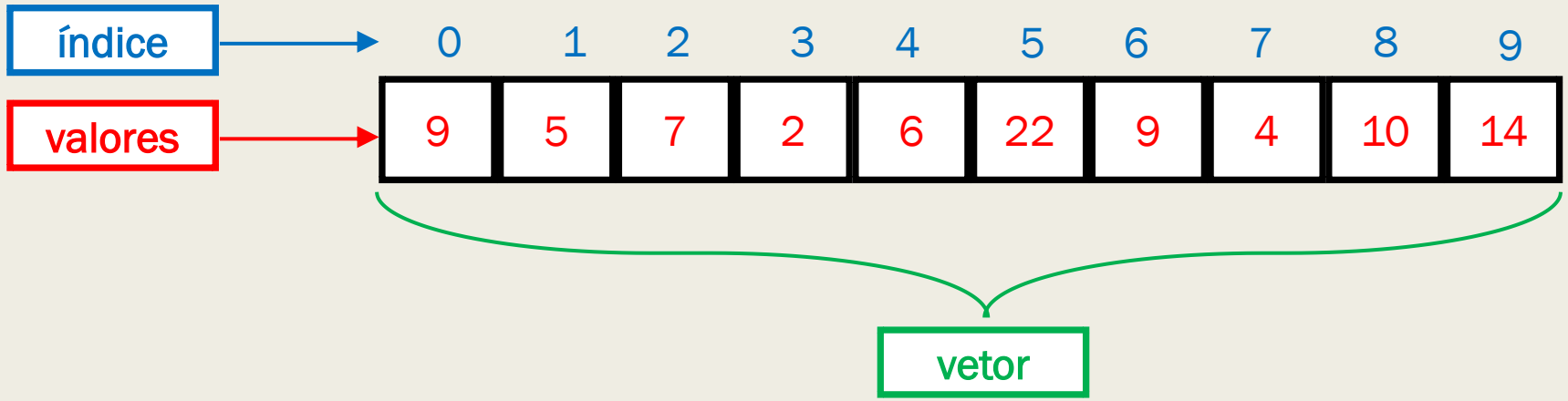
- Este tipo de problema é impossível de ser resolvido com o uso de uma única variável pois, quando se lê o segundo número, já se perdeu o primeiro da memória. Exigindo o uso de tantas variáveis quantos fossem os dados de entrada, mas notem que isto deve ser conhecido em *tempo de compilação*!

- Quando se aloca, um número inteiro em uma variável, o que ocorre é que o computador reserva uma posição de memória em algum endereço da memória RAM. Com um número inteiro exigindo 4 bytes no caso do Java.

Vetores

- Os **vetores** são estruturas de dados que permitem o acesso a uma grande quantidade de dados em memória utilizando somente um nome de variável.

- Para acessar o conteúdo de uma posição especifica do vetor é necessário a informação do índice do vetor.



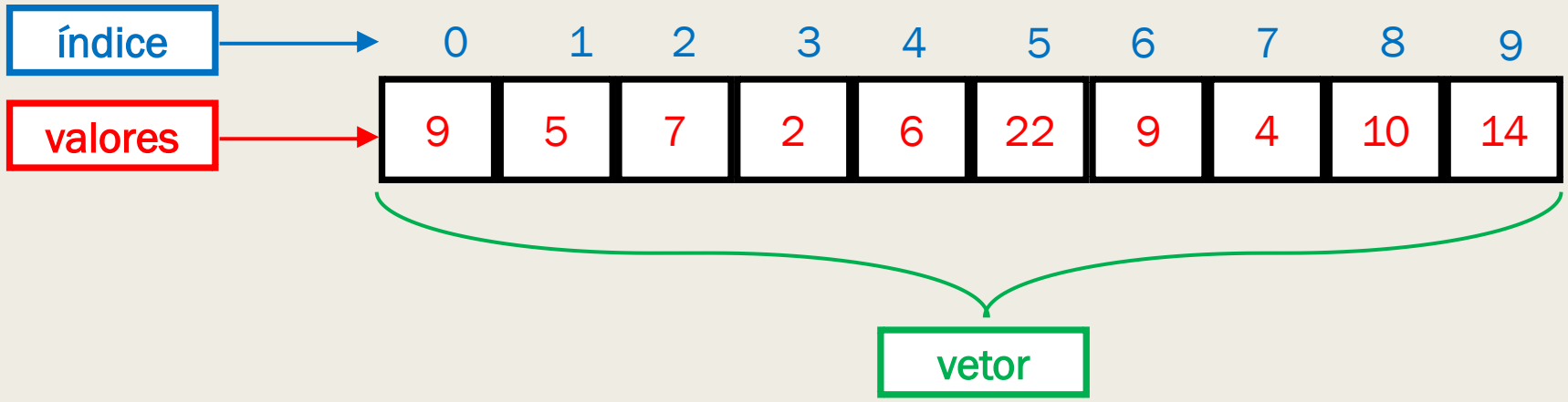
Vetores

Declaração e atribuição em Java

```
int[] vetor = new[]{9,5,2.....,14};
int[] vetor = new[10];
vetor[0] = 9;
vetor[1] =5; .....
```

Acessando os dados:

```
= vetor[0];
```

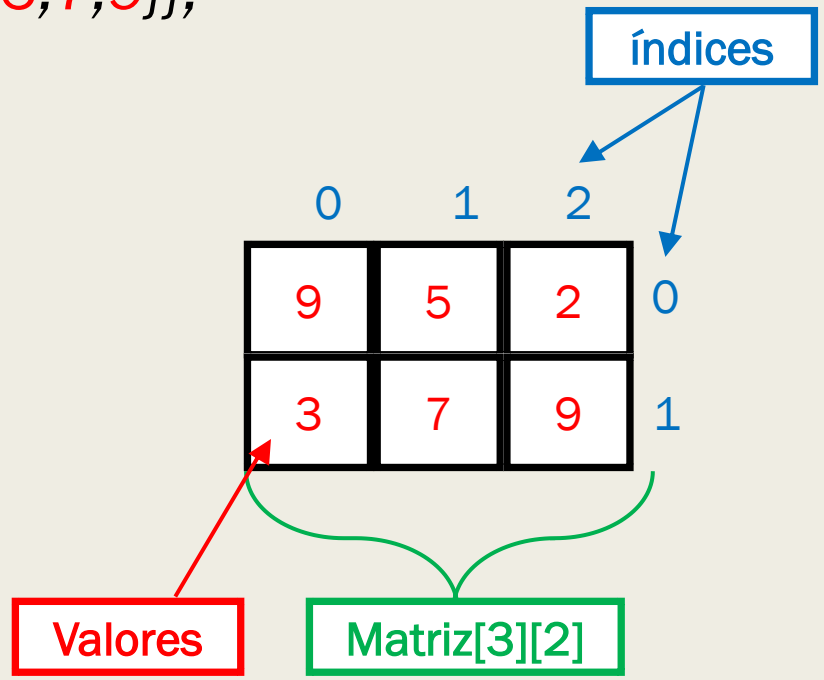


Matrizes

Declaração e atribuição em Java

```
int[][] matriz = new[][] {{9,5,2},{3,7,9}};
int[][] matriz = new[3][2];
matriz[0][0] = 9;
matriz[0][1] = 5;
matriz[1][0] =3;
matriz[1][1] =7;
```

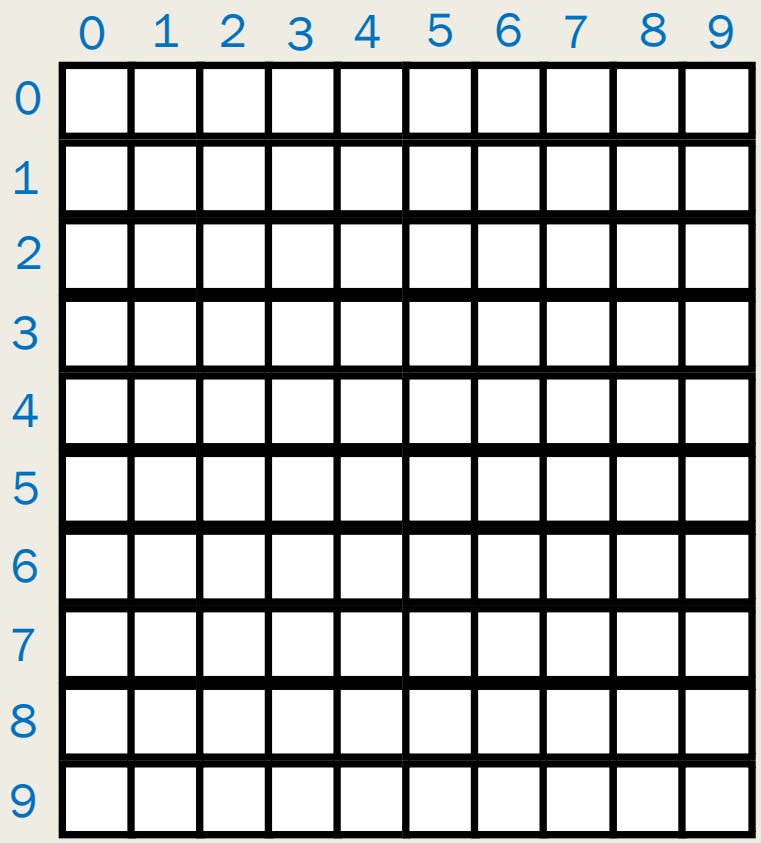
Acessando os dados: `= matriz[0][0];`

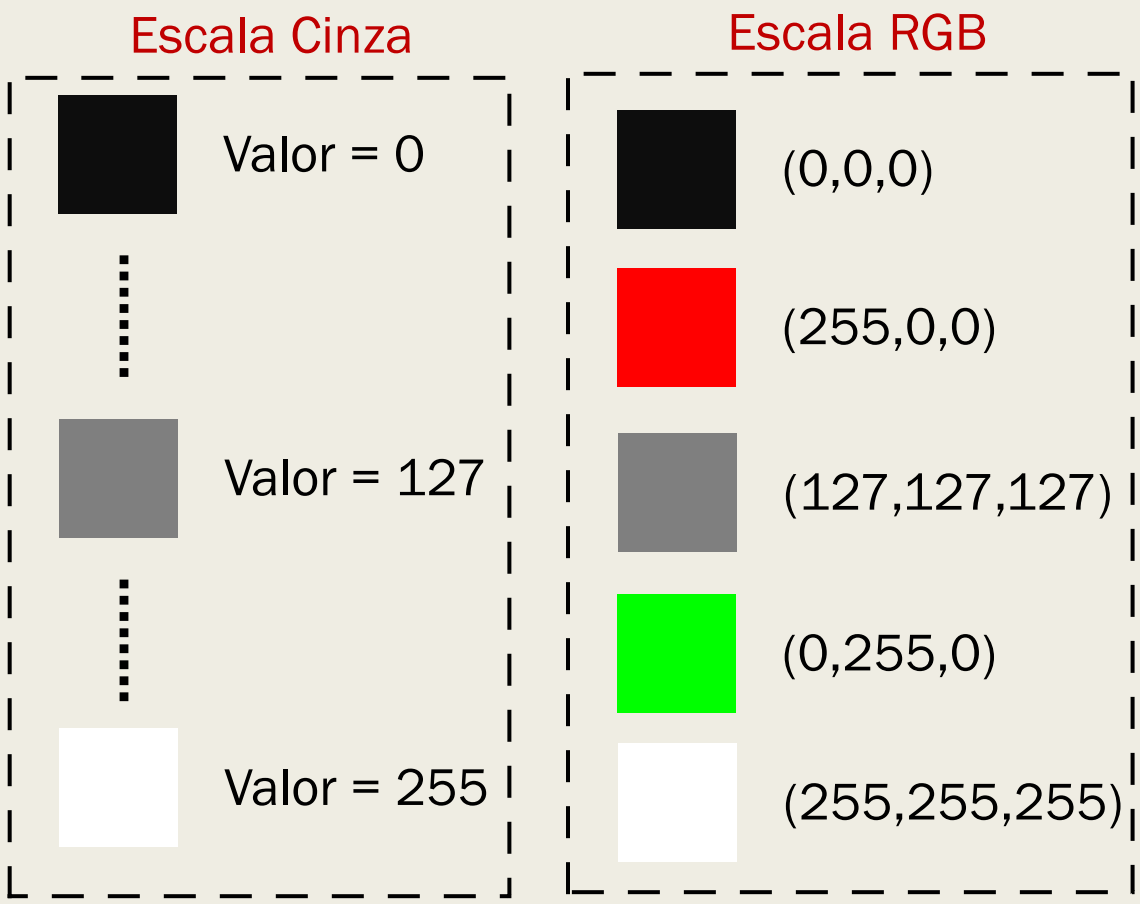


Na linguagem Java o tamanho dos vetores e matrizes é fixo, não podendo ser alterado em *tempo de execução*. Linguagens como C e VB, permitem o redimensionamento durante a execução do código.

Vetores e Matrizes

Onde posso utilizar uma matriz de valores ???





Sistema Decimal e Binário

Decimal

10^3	10^2	10^1	10^0
1	5	2	1

$$1 * 10^3 + 5 * 10^2 + 2 * 10^1 + 1 * 10^0 =$$

$$1000 + 500 + 20 + 1 = \mathbf{1521}$$

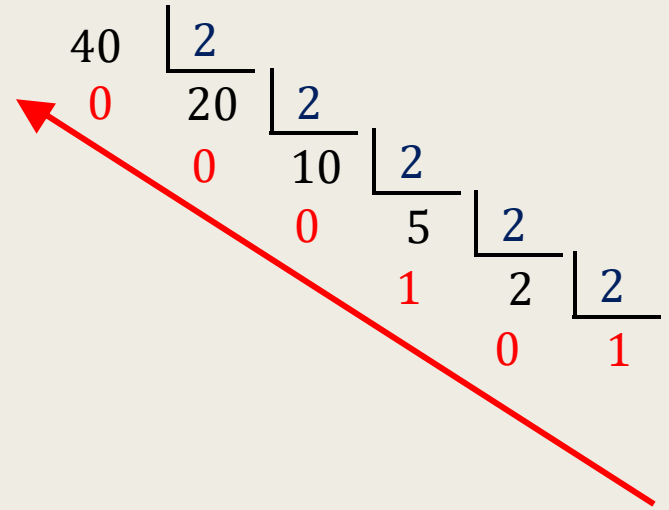
Binário

2^3	2^2	2^1	2^0
1	1	0	1

$$1 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0 =$$

$$8 + 4 + 0 + 1 = \mathbf{13}$$

Conversão Decimal para Binário.



1 0 1 0 0 0 (*binário*)

Conversão Binário para Decimal.

2^5 2^4 2^3 2^2 2^1 2^0

32+ 0 + 8 + 0 + 0 + 0 = 40

Vetores e Matrizes

Exercício

1. Crie um método que receba 1 vetor com 8 posições, inverta a ordem dos itens e retorne o vetor invertido.

2. Crie um método que receba 3 vetores com 8 posições cada e retorne uma matriz de 8x3 posições com os respectivos valores armazenados nos vetores.

3. Crie um método que receba como parâmetro um número binário na forma de um vetor com 8 posições e retorne esse número convertido para decimal.

4. Crie um método que receba como parâmetro um número decimal, converta esse número para binário e retorne na forma de um vetor com 8 posições.

* Para obter o resto de uma divisão utilize o operador %. *Exemplo: int resto = 9 % 2*

Vetores e Matrizes

Exercício

5. Escreva um método que receba dois vetores de 5 números cada, gere um terceiro vetor com os elementos dos outros dois vetores intercalados.
   Exemplo: vetor 1º ={A,B,C,D,E} | 2º={1,2,3,4,5}| 3º={A,1,B,2,C,3,D,4,E,5}

5. Escreva um método para imprimir no console uma matriz de 2 dimensões. Utilize o seguinte padrão.

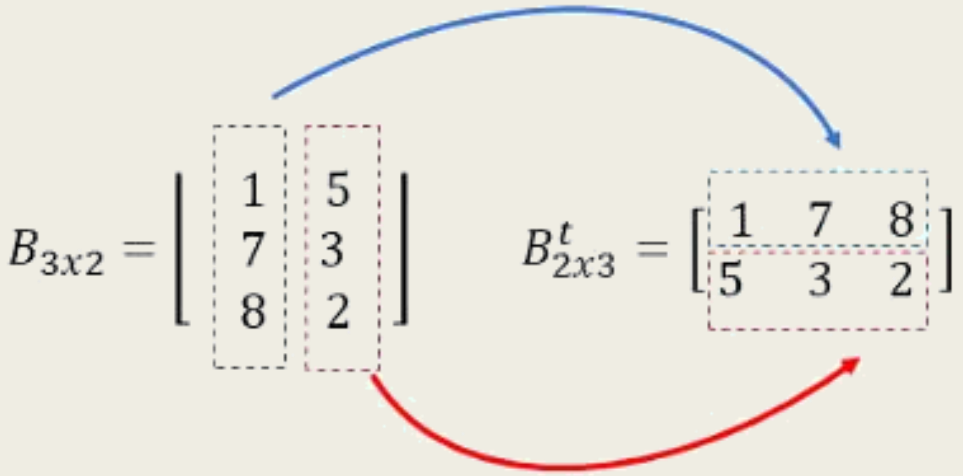
|	X	|	X	|	X	|
|	Y	|	Y	|	Y	|
|	Z	|	Z	|	Z	|

Utilize System.out.print(“|” + ”\t” + variável + “\t” + “|”) para imprimir no console os marcadores com o espaço TAB entre eles.

Vetores e Matrizes

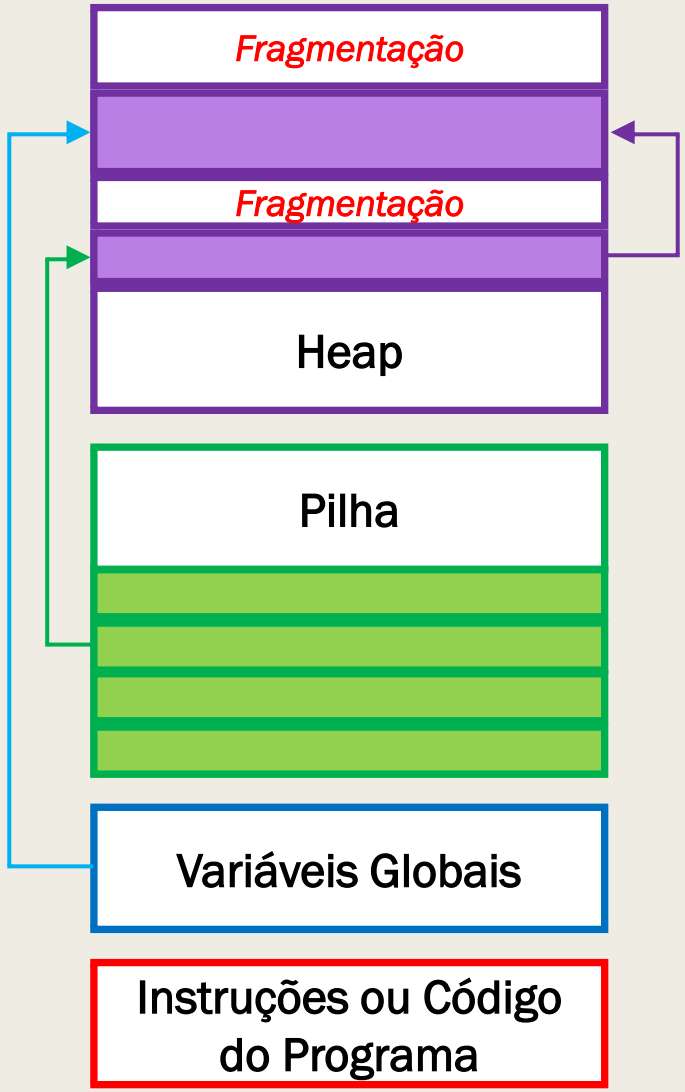
Exercício

7. Crie um método que dado uma matriz [3x2] gere sua equivalente [2x3] transposta. Conforme exemplo, utilize o algoritmo do exercício 6 para imprimir a matriz original e a transposta.



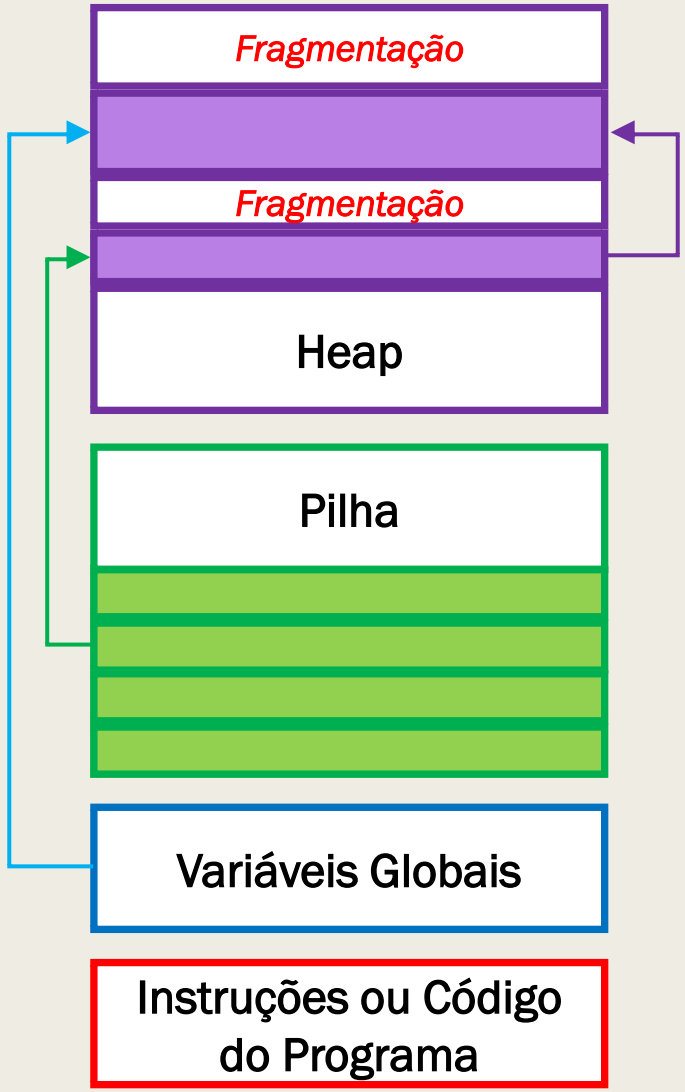
Alocação de Memória

- A maioria das linguagens de programação utilizam um esquema de segmentação de memória em quatro regiões logicamente distintas.
  - **Instruções:** utilizado para armazenar o código do programa sendo executado.
  - **Variáveis Globais:** utilizado para armazenar variáveis declaradas no escopo inicial do programa, como por exemplo constantes.
  - **Pilha:** utilizado para armazenar as variáveis locais e ponteiros de endereçamento para variáveis dinâmicas
  - **Heap:** utilizado para armazenar variáveis alocadas dinamicamente.

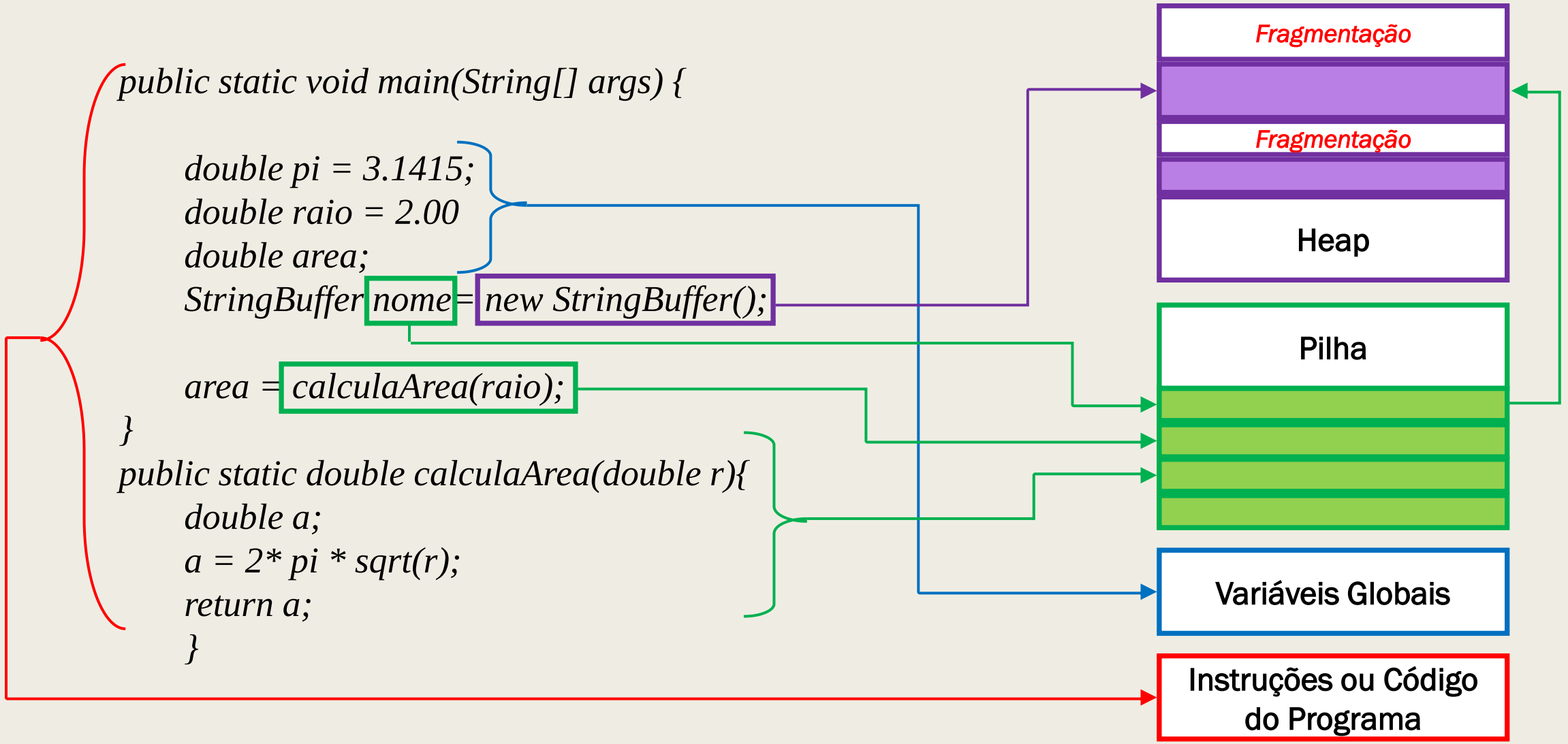


Alocação de Memória

- **Escopo de duração das variáveis:**
  - **Variáveis Globais:** espaço de memória continua alocada enquanto o programa estiver executando.
  - **Variáveis locais:** espaço de memória continua alocada enquanto a função que a declarou estiver executando.
- **Espaço para alocação:**
  - **Variáveis Estáticas:** o tamanho do espaço em memória para a alocação da variável é conhecida.
  - **Variáveis Dinâmicas:** o tamanho do espaço em memória para a alocação é desconhecido, compilador cria um espaço no **Heap** e registra o endereço em um ponteiro na **Pilha.**



Alocação de Memória



Alocação de Memória

Processo de limpeza:

- Em linguagens de programação com C/C++ o processo de limpeza da memória alocada é responsabilidade do programador.
- Na linguagem C existe 4 funções especificas para isso:
  - **Malloc**: permite a alocação de uma nova área de memória para uma variável, retornando um ponteiro para o endereço de memória.
  - **Calloc**: similar ao Malloc, mas a variável é inicializada com zero.
  - **Realoc**: permite realocar o tamanho da área ocupada pela variável.
  - **Free**: permite que uma área alocada seja liberada.

Alocação de Memória

Processo de limpeza:

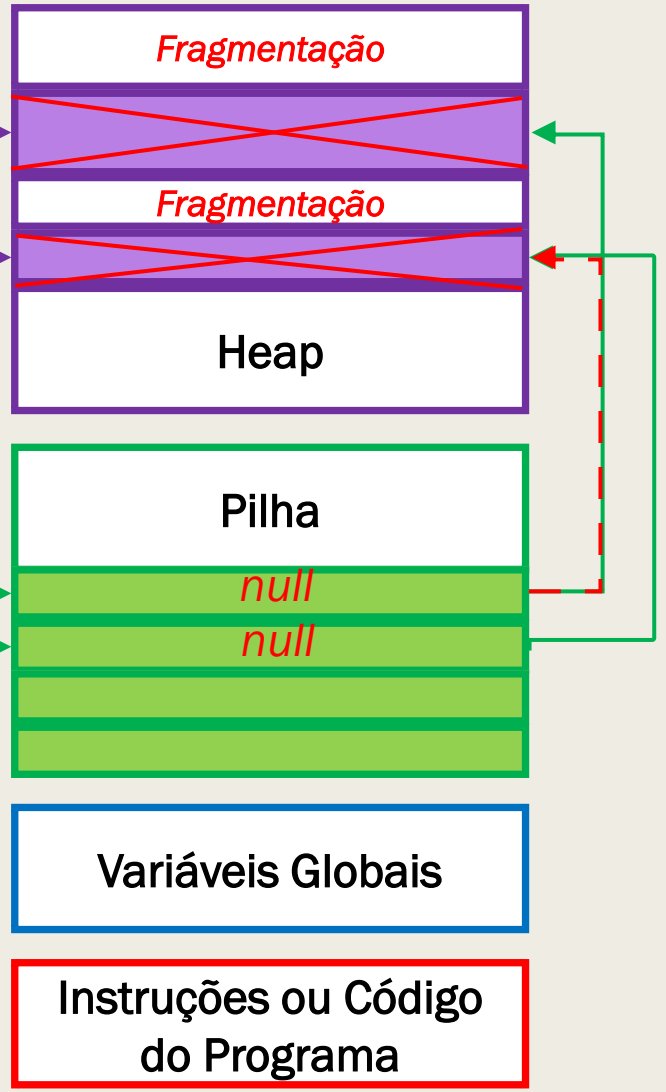
- Em Java e Python o programador não precisa gerenciar explicitamente a memória do sistema.
- Pois existem mecanismos automáticos de gerenciamento e limpeza da memória, conhecido como coletor de lixo (**Garbage Collector**).
- A técnica de Coleta de Lixo consiste na recuperação segura do espaço de memória ocupado por um objeto que não é mais referenciado dentro de uma aplicação.
- Em linguagens orientadas a objeto é importante que objetos que não são mais referenciados, sejam desalocados para liberar a memória ocupada.

Alocação de Memória

Processo de limpeza:

- **Exemplo:**

```
StringBuffer nome= new StringBuffer();
StringBuffer auxiliar= new StringBuffer();
        ⋮
nome= auxiliar;
        ⋮
nome= auxiliar = null;
```



Alocação de Memória

Garbage Collector:

- **Possui a função:**
  - Alocar a memória.
  - Assegurar que quaisquer objetos referenciados permaneçam na memória.
  - Recuperar a memória alocada pelos objetos que não são mais alcançáveis pelas referências do código em execução.

- Em geral, quando não há interferência do programador, a máquina virtual executa o Coletor de Lixo quando percebe que a memória está ficando sem espaço.

Alocação de Memória

Garbage Collector:

- De qualquer maneira, uma aplicação pode encerrar sua execução sem que o **Garbage Collector** seja executado uma única vez.
- Quando um objeto deixa de ser referenciado, o espaço de memória ocupado por ele **não é imediatamente desalocado**.
- O **Garbage Collector** pode ser configurado para priorizar diferentes comportamentos:
  - Diminuir a quantidade de intervenções utilizando assim mais memória.
  - Realizar limpezas mais constantes otimizando o uso de memória.

Alocação de Memória

Garbage Collector:

- Leia mais sobre Garbage Collector em Java: https://www.devmedia.com.br/introducao-ao-java-garbage-collection/30326

Alocação de Memória

Exercício

- Pesquise sobre **Garbage Collection,** execute o código abaixo e responda:
  - Com base na quantidade de memória utilizada e na quantidade de variáveis declaras.
  - *Qual o tamanho em bytes de cada variável do tipo **double** ?*

Alocação de Memória

```
public static void main(String[] args) {
        Runtime rt = Runtime.getRuntime();
        System.out.println("Memoria total da JVM: " + rt.totalMemory());
        double m1,m2;
        m1= rt.freeMemory();
        System.out.println("Memoria antes da criação dos objetos: " + rt.freeMemory());

        double vetor[]= new double[100000];
        for (int i = 0; i < 100000; i++) {
            vetor[i]=i;
        }
        m2= rt.freeMemory();
        System.out.println("Memória depois da criação dos objetos: " + rt.freeMemory());
        System.out.println("Memória utilizada: " + (m1- m2) + " bytes");
        rt.gc();
        System.out.println("Memória depois executar o gc: " + rt.freeMemory());
}
```





Recursão

- Recursão pode ser encontrada em matemática, computação e no cotidiano.
- Remetendo a ideia de repetição.
- Repetição de um objeto dentro dele mesmo.
- Equivalente a um *looping* ou laço continuo.
- Imagine um espelho de frente com outro espelho.
- Ou um sonho onde você sonha que está sonhando.

Recursão

- Algoritmos recursivos podem simplificar o entendimento de códigos complexos.
- Porém nem sempre um algoritmo recursivo terá o melhor desempenho.
- De forma geral problemas matemáticos naturalmente recursivos podem ser facilmente implementados utilizando funções recursivas.
- O grande potencial da recursão é a possibilidade de definir elementos com base em versões mais simples desses mesmo elementos.
- Em termos computacionais, trata-se do paradigma de dividir um problema maior em partes menores, que são resolvidos pela mesma função recursiva.

“Dividir para conquistar”.

Recursão

- **Para implementar uma função ou método recursivo é necessário estabelecer:**
  - **Uma condição de parada:** estabelecendo uma solução trivial que encerre a chamada recursiva.
  - **Uma mudança de estado:** estabelecendo uma diferença entre o estado inicial e o estado final após a nova chamada recursiva.
  - **Por exemplo:** o decremento de um parâmetro da função recursiva a cada interação e o estabelecimento de um condicional de parada, como ***n=1***, atende os dois requisitos de uma função recursiva.

Recursão

- *Um exemplo clássico de uma função matemática recursiva é o cálculo do **fatorial** de um número **n**.*

4! = 4 x 3 x 2 x 1 = 24 ➔ ***n! =***

- *Se n =0* ➔ *1*
- *Se n>=1* ➔ *n.(n-1).(n-2).(n-3) .....*

```
public int fatorial(int n) {
    // Se n for igual a 0 (zero) então retorna 1.
    if (n == 0) {
        return 1;
    }
    else{
    /* Para qualquer outro número, calcula o seu valor multiplicado pelo fatorial de seu antecessor. */
        return  n * fatorial(n - 1);
    }
}
```

Leia: http://www.universidadejava.com.br/java/java-fatorial/

Recursão

Atenção

- A maioria dos algoritmos recursivos consomem mais recursos computacionais que seus equivalentes interativos. Por isso é importante tomar cuidado com o uso de algoritmos recursivos.

- **Você deve utilizar a recursão quando**:
  - O problema é naturalmente recursivo.
  - A recursividade não gera aumento considerável do custo computacional.
  - Existe um condicional de parada bem definido que permite prever a quantidade de interações geradas pela função recursiva.

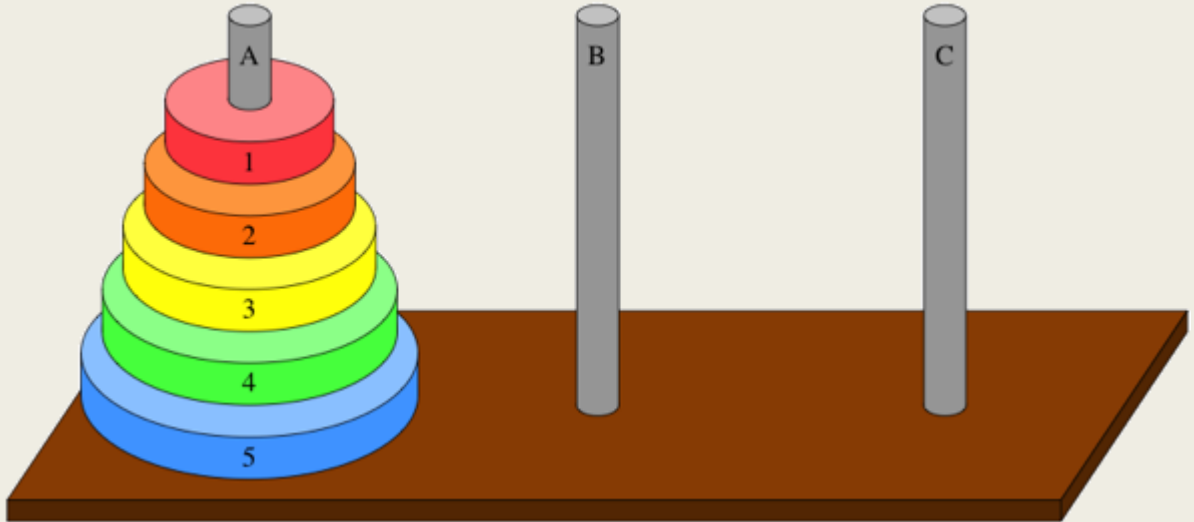
- **Você NÃO deve utilizar a recursão quando:**
  - A função recursiva é ineficiente, se comparada a funções interativas similares.
  - Não existe um condicional de parada.
  - Não é possível prever o número de interações até o estabelecimento do condicional de parada.

Recursão

Exemplo

- **Torre de Hanoi**
  - Consiste de três hastes **A (origem), B (destino) e C (auxiliar).**
  - O objetivo do jogo é transferir todos os discos da origem para o destino.

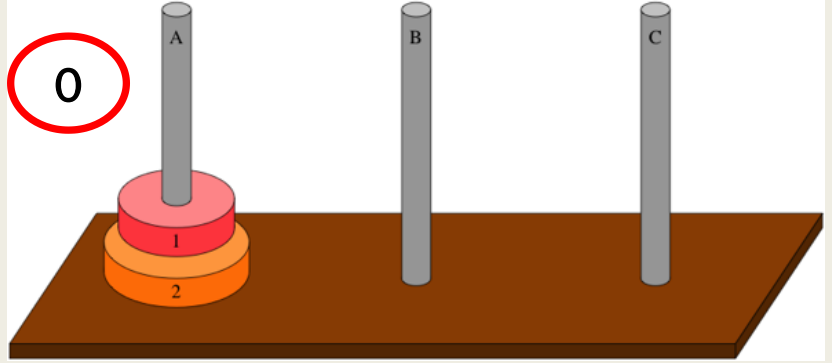
  - ***Respeitando as seguintes regras:***
    - Apenas um disco poderá ser movido por vez.
    - Um disco maior não pode ser movido em cima de um disco menor.

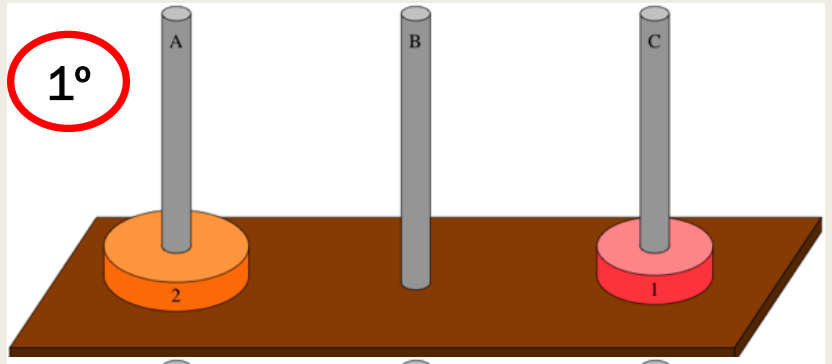


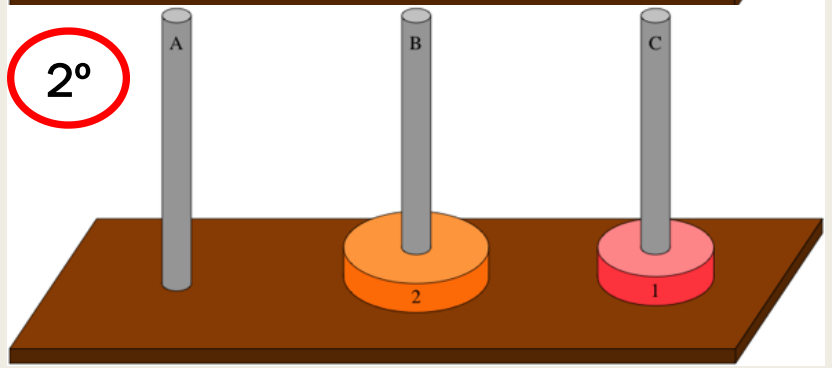
Recursão

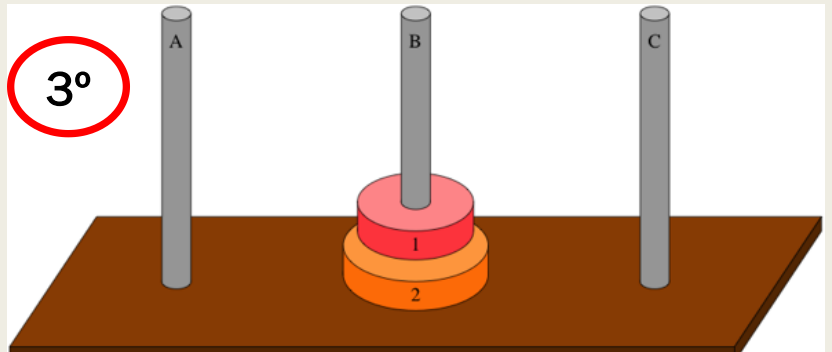
Torre de Hanoi

- **Passo a passo da resolução com 2 discos**
  I. Mova o disco 1 para o pino auxiliar C.
  II. Mova o disco 2 para o pino destino B.
  III. Mova o disco 1 para o pino destino B.









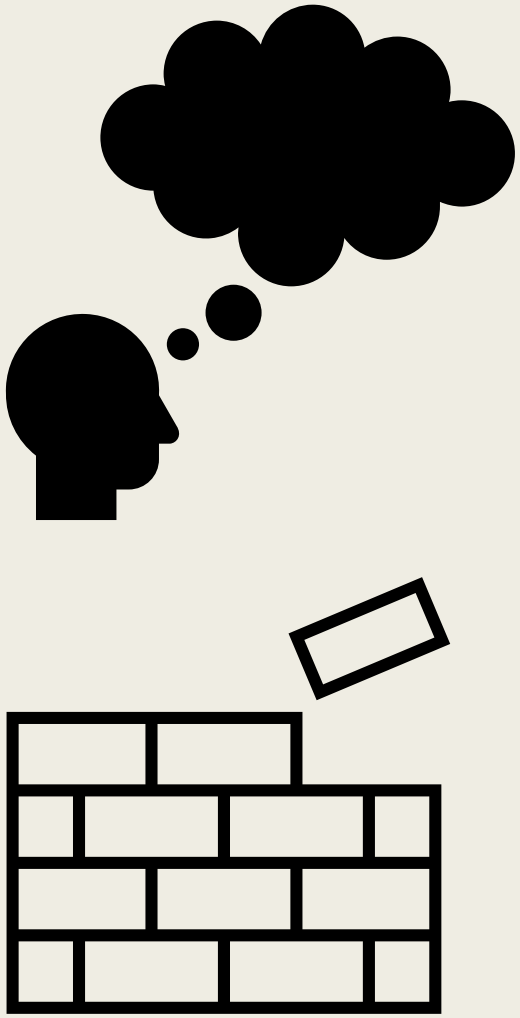
Recursão

- Leia mais sobre:
  - Recursividade: https://pt.khanacademy.org/computing/computer-science/algorithms/recursive-algorithms/a/recursion

  - Torre de Hanoi: https://pt.khanacademy.org/computing/computer-science/algorithms/towers-of-hanoi/a/towers-of-hanoi

  Assista: https://www.youtube.com/watch?v=Q2BooYpqS6g&t=428s

  Se você gostar de matemática: https://www.youtube.com/watch?v=CLouA-TA6nc&t=474s



Estruturas de Dados Elementares

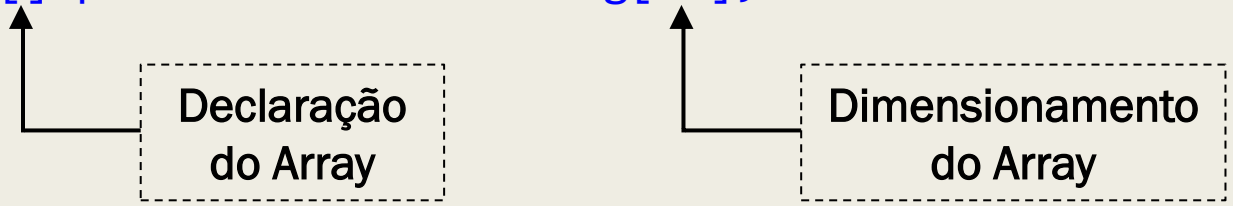
- Listas (Lists).
- Pilhas (Stack).
- Filas (Queues).
- Arvores (Tree)

Array e ArrayList

- Toda linguagem de programação possui uma estrutura de dados capaz de armazenar uma coleção de elementos, sendo acessados através de um índice, normalmente conhecidos como ***Array*** ou **Vetor.**

- A quantidade de elementos possíveis de serem armazenados em um ***Array*** é normalmente fixo, com algumas linguagens de programação como C e C++, permitindo redimensionar o tamanho do ***Array*** em tempo de execução.

- **Java não permite o redimensionamento de vetores.**
  - `String[] palavras = new String[10];`



- **Uma solução para contornar esta limitação é o uso de ArrayList ou Listas.**
  - `ArrayList<String> palavras = new ArrayList<String>();`



Quais as diferenças entre Array e ArrayList em java?

- Quando utilizamos um **array** em Java, estamos trabalhando com tamanhos fixos de coleções. Não sendo possível aumentar o tamanho da coleção sem criar uma nova coleção.
  - Exemplo:

    String[] palavra = new String[10];

    // Para aumentar o tamanho do vetor precisamos criar uma nova coleção.

    String[] palavra = new String[20];

    //Porém a nova coleção apaga a coleção antiga, sendo necessário realizar uma cópia dos valores da coleção antes de redimensionar o vetor.

- Isso não acontece com o **ArrayList**, porque a classe **ArrayList** é um agrupamento dinâmico de objetos.

- Podemos adicionar ou retirar elementos de um **ArrayList**, sem que seja necessário criar uma nova coleção.

ArrayList em java?

- **ArrayList** é membro da classe ***java.util***, por isso sendo necessário a declaração:
  - *import java.util.ArrayList;*

- **ArrayList** é uma coleção de objetos, **NÃO sendo possível** criar lista de **tipos primitivos,** como **int ou float.**

- A solução para isso é tratar os primitivos como objetos, usando o que chamamos **classes empacotadoras** (**wrapper**).

- Há uma classe **wrapper** correspondente para cada tipo primitivo em Java. Com a classe empacotadora encapsulando um único valor de um tipo de dado primitivo.

- Por exemplo, a classe **Integer** deve tratar um valor **int**.

- Uma outra situação em que as classes empacotadoras são úteis é quando precisamos **converter um objeto em um tipo de dado primitivo**.
  - Por exemplo: Integer.parserInt(“numero”);

ArrayList em java?

Primitivo	Classe Wrapper	Argumentos do construtor
boolean	Boolean	boolean ou String
Byte	Byte	byte ou String
Char	Character	char
double	Double	double ou String
Float	Float	float, double ou String
Int	Integer	int ou String
Long	Long	long ou String
short	Short	short ou String

- As classes ***wrapper*** são parte do pacote ***java.lang***, o qual é importado por default em todos os programas Java.

Interface List

- A interface ***List*** in Java oferece um caminho para **armazenar coleções ordenadas de objetos.**
- Com a interface ***List*** sendo uma subclasse de ***Collection***, oferecendo assim **diversos métodos herdados de Collection para manipulação de coleções.**
- Como ***List*** é uma interface, não é possível criar objetos do tipo ***List.*** Devendo ser instanciado objetos, através de uma classe concreta que implementa a interface ***List,*** como **ArrayList**, **LinkedList**, **Stack e Vector.**
- *Por exemplo:* ***List <Integer> lista = new ArrayList<Integer>();***



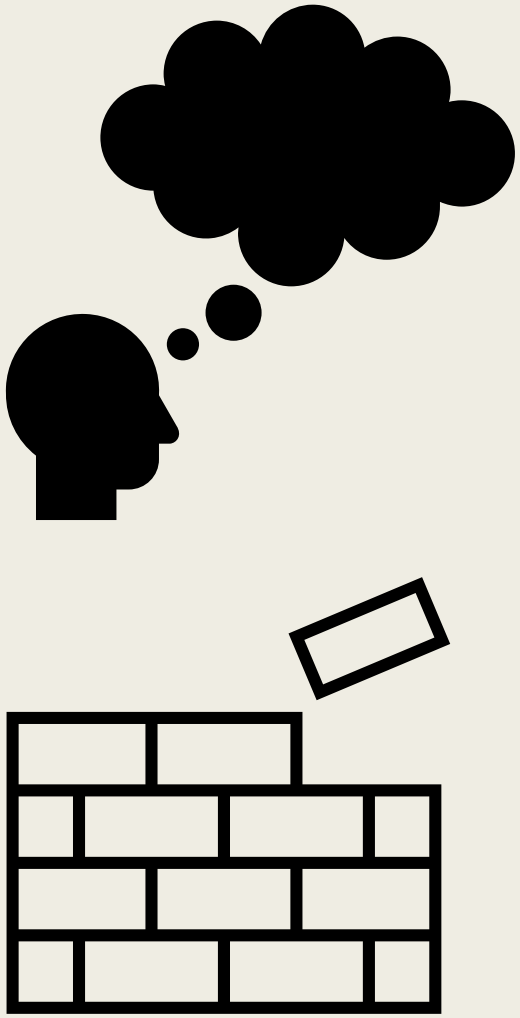
Principais métodos da Interface List

- *boolean* ***add(Object e);*** *// Adiciona o Objeto à coleção.*
- *boolean* ***remove(Object o);*** *// Remove o Objeto da coleção.*
- *boolean* ***contains(Object o);*** *// Retorna True se a o Objeto estiver na coleção.*
- *boolean* ***isEmpty();*** *// Retorna True se a coleção estiver vazia.*
- *int* ***size();*** *// Retorna a quantidade de objetos na coleção.*
- *int* ***indexOf(Object o);*** *// Retorna o índice do Objeto dentro da coleção.*
- *Object* ***get(int index);*** *// Retorna o Objeto referente ao index.*
- *void* ***add(int index, E element****); // Adiciona o objeto a posição referente ao index.*
- *E* ***set(int index, E element);*** *// Altera o objeto referente ao index.*
- *E* ***remove(int index);*** *// Remove o objeto referente ao index.*
- *void* ***sort (Comparator comp);*** *// Ordena a coleção dado o comparador utilizado.*

Interface List

Leia Sobre

- ***Wrapper*** *Classe:* *https://www.geeksforgeeks.org/list-interface-java-examples/*
- ***Arrays*** *em Java:* *https://www.geeksforgeeks.org/arrays-in-java/?ref=lbp*
- ***List*** *e seus métodos*: https://www.geeksforgeeks.org/list-interface-java-examples/



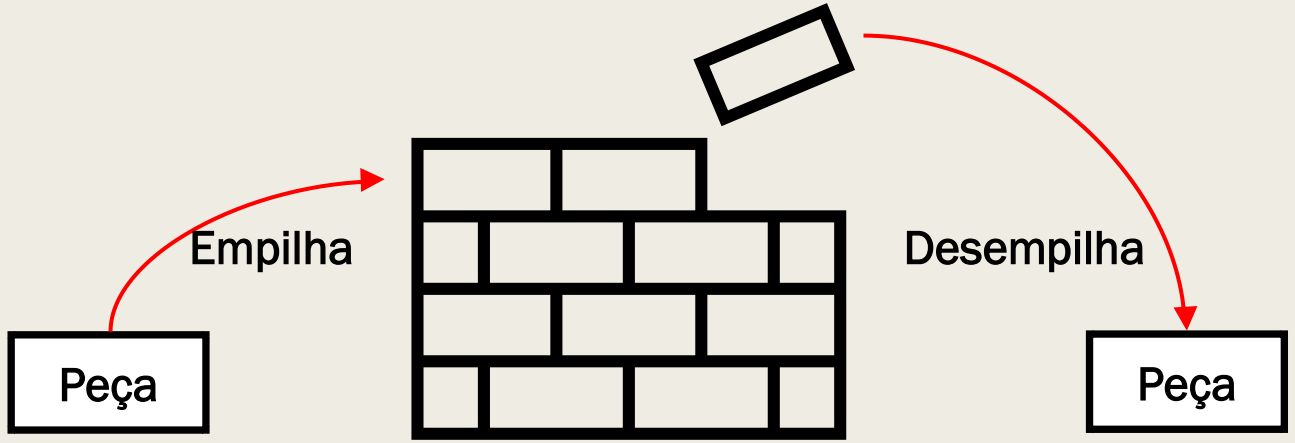
Pilhas

- Conceitos.
- Aplicação.

Pilhas

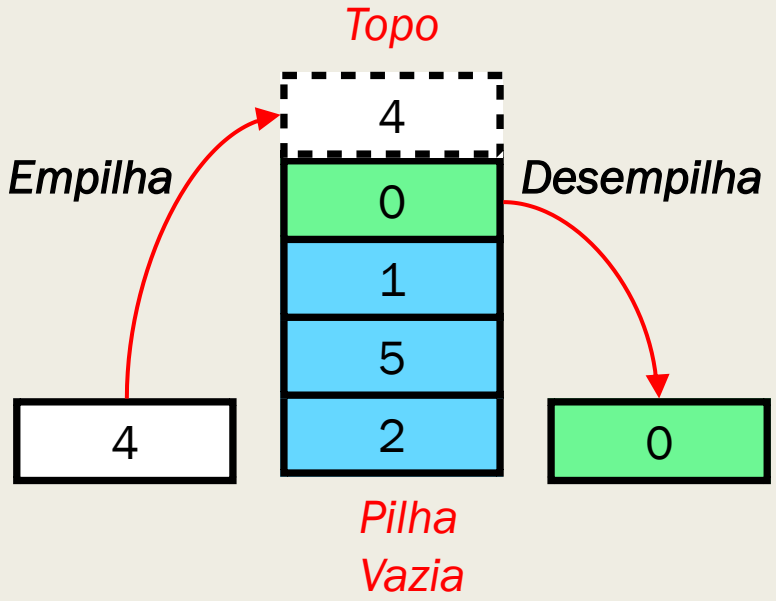
- Imagine um jogo de montar do tipo “Lego”, onde as peças devem ser montadas ou empilhadas em determinada ordem, e depois desempilhadas seguindo a ordem inversa.

- Como implementar um algoritmo capaz de realizar essa tarefa ???



Pilhas

- **Características de um Pilha**
  - Pilhas são estruturas de dados capazes de armazenar objetos de forma sequencial, similar a uma lista. Porém são mais restritivas em relação as operações permitidas, quando comparado a uma lista.
  - Permitindo o **acesso** apenas ao **último item adicionado**.
  - Conceito ***LIFO*** (***Last-in First-out***).



- **Interface de acesso a uma Pilha**
  - Uma pilha possui no mínimo 3 métodos públicos.
    - **Empilhar**: adiciona um objeto ao topo da pilha.
    - **Desempilhar**: retira o objeto do topo da pilha.
    - **Pilha Vazia**: informa que todos os objetos já foram removidos.

Aplicação de Pilhas

- **Pilhas** são essenciais em computação, sendo utilizadas nas mais diversas aplicações, como:
  - ***Controle de execução de instruções.***
    - Pilhas de instruções em sistemas operacionais, processadores, etc.
  - ***Compiladores.***
    - Pilhas para controle de sintática em linguagens de programação.
  - ***Interpretadores de expressões matemáticas, ortográficas e etc.***
    - Corretores ortográficos, planilhas de cálculo.
  - ***Algoritmos de busca em profundidade, como resolução de labirintos***.
    - Busca de rotas e registro de caminhos de retorno.
  - ***Controle de roteamento em redes de computadores***.
    - Controle de endereçamento de DNS, essencial para o funcionamento da internet.
  - **Etc……..**

Pilhas

- **Como implementar uma Pilha ?**
  - **Precisamos de duas variáveis:**
    - Uma estrutura capaz de armazenar objetos de forma sequência, podendo ser um vetor ou uma lista.
    - E um registrador para armazenar a posição referente ao topo da pilha.
  - **Precisamos de 3 métodos:**
    - **Empilha** que adiciona um novo item ao vetor ou lista e atualiza a posição do topo.
    - **Desempilha** que remove do topo da pilha.
    - **Pilha Vazia** que retorna verdadeiro se pilha está vazia ou falso se não estiver.

Interface pública de uma Pilha em JAVA

```
import java.util.ArrayList;
import java.util.List;

public interface InterfacePilha {

    public void empilhar(Object item);
    public Object desempilhar();
    public boolean pilhaVazia();
}
```

Em Java todos os objetos derivam da Classe ***Object***, portanto uma lista do tipo ***Object*** pode receber qual objeto.

Pilhas na API Java

- *Em Java temos a Classe **Stack** na biblioteca **java.util** que implementa a estrutura de dados de uma pilha.*
- ***Exemplo**: criando uma pilha de objetos do tipo Livro*

```
Stack<Livro> livrosParaLer = new Stack<Livro>();
livrosParaLer.push(new Livro("Data Structures and the Java Collections Framework"," William J. Collins ",2011));
livrosParaLer.push(new Livro("Data Structures and Problem Solving Using Java","Mark Allen Weiss",2009));
livrosParaLer.push(new Livro("Estrutura de Dados e Técnicas de Programação","Piva D.J.; Nakamiti, G. S.; Bianchi, F. et ",2014));
System.out.println(livrosParaLer.size());
While(!livrosParaLer.empty()){
Livro proximo = livrosParaLer.pop();
System.out.println(proximo.toString());
}
System.out.println(livrosParaLer.size());
```

Exercício

- Crie uma nova **Classe** in Java com o nome **PilhaArray**.
- Implemente os **empilhar**, **desempilhar, pilha está vazia e pilha está cheia**.
- Utilize na implementação um **Array de Object** que irá receber como parâmetro via construtor o tamanho do **Array**.
- Teste o objeto pilha empilhando uma lista sequencial de números e os desempilhando em seguida.

Utilize os métodos do objeto Array() :

- **length();** que retorna o tamanho do vetor
- Crie uma variável **int** com nome de **topo** para controlar o índice do topo da pilha.
- Verifique se a pilha já está cheia antes de adicionar um novo item, e se está vazia antes de tentar remover.

Exercício

- Crie uma **Classe Pilha** e implemente os métodos padrões.
- Implemente os métodos da interface para executar as funções de **empilhar**, **desempilhar** e informar que a **pilha está vazia**.
- Teste o objeto pilha empilhando uma lista sequencial de números e os desempilhando em seguida.

Utilize os métodos do objeto ArrayList() :

- **.add();** que permite adicionar itens a lista
- **.remove(index lista);** para remover itens.
- **.size();** que retorna o tamanho da lista.
- **.isEmpty();** que informa se a lista está vazia.

Interface pública de uma Pilha em JAVA

```
import java.util.ArrayList;
import java.util.List;

public interface InterfacePilha {

    public void empilhar(Object item);
     public Object desempilhar();
     public boolean pilhaVazia();
}
```

Em Java todos os objetos derivam da Classe ***Object***, portanto uma lista do tipo ***Object*** pode receber qualquer objeto.

Exercício

- Crie um método que recebe uma **String** e retorne **True** se a String é **Palíndromo.**
- Ou seja, o nome é lido da mesma forma nos dois sentidos.
- Para isso utilize uma pilha para inverter o nome, exemplo: “maria” ➔ “airam”. Depois compare o nome invertido com o original, se forem iguais é palíndromo.
- Utilize as classes **Pilha**, **PilhaArray** e **Stack** da API Java nos testes. E verifique se as três classes possuem o mesmo comportamento.

- **Utilize os métodos do objeto String :**
  - **.charAt(index);** para acessar os caracteres da String.
  - **.length():** para verificar a quantidade de caracteres da String.

Exercício

- Crie um método chamando **validaBalanceamento** que recebe uma **String** e verifica se uma sequência de caracteres contém parênteses balanceados. Retornando ***True*** ou ***False***.

- **Por exemplo:**
  - “{ [( ) ( )] }” ➔ Balanceado
  - “{ [( ))] }” ➔ Desbalanceado

- **Utilize a seguinte lógica:**
  - Quando encontrar um símbolo de abertura **{, [, (,** adiciona o símbolo de fechamento equivalente a pilha. Por exemplo: encontra **“{“** ➔ Empilha **“}”**
  - Quando encontrar um símbolo de fechamento **},],),** compara com o topo da pilha.
    - Se for verdadeiro remove o símbolo da pilha.
    - Se for falso retorna ***False***.
  - Se o método chegar ao final da **String** e a **pilha estiver vazia**, retorna ***True***.

Exercício

- **Agora pense, em um interpretador de expressões matemáticas que precisa resolver a seguinte expressão:**
  - Resultado = [(A + B) * C] / D

- Seria possível utilizar o conceito de pilhas para solucionar esse problema ?

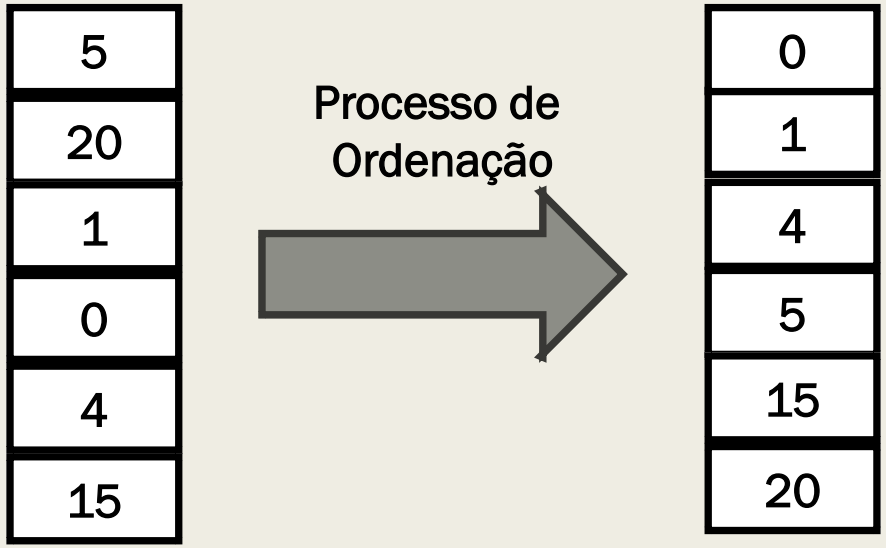
- Pesquise quais os passos para resolver as expressões matemáticas internas primeiro ?

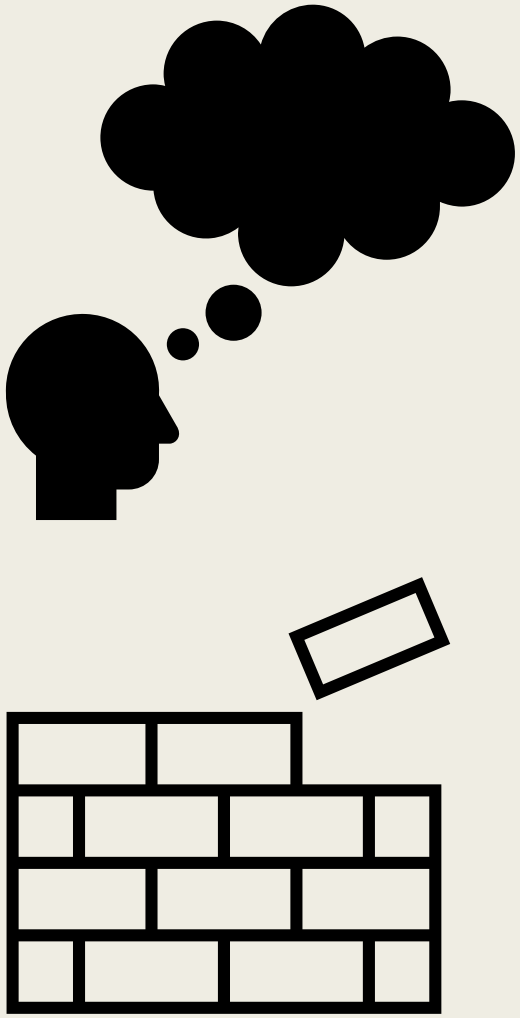
- Pesquise sobre notação **Infixa**, **Prefixa** e **Posfixa**.

- Pense em utilizar funções recursivas.

Exercício

- Considere uma pilha que armazene números, escreva uma função para ordenar os valores da pilha em ordem crescente, utilize pilhas auxiliares para realizar a ordenação.
- Por exemplo:



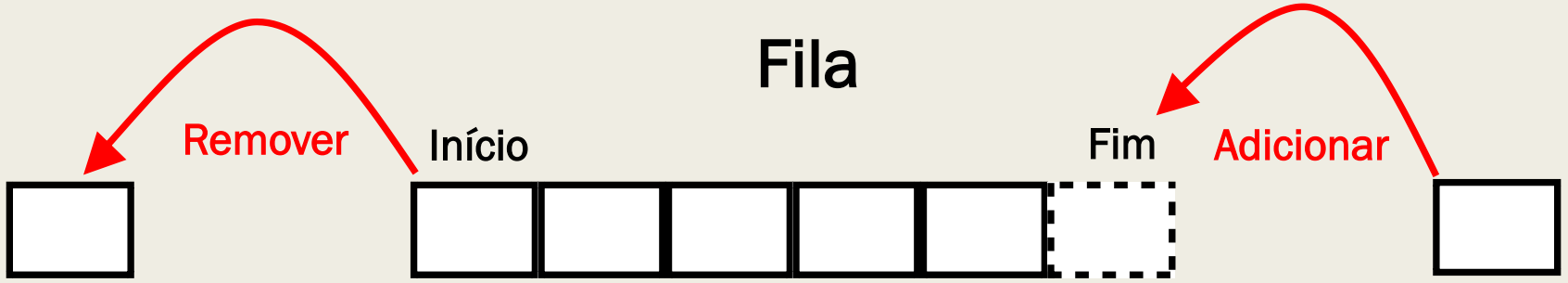


Filas

- Conceitos.
- Aplicação.

Filas

- Imagine uma fila de pessoas em um banco, se todos respeitarem as regras o primeiro a entrar será o primeiro a sair, conceito FIFO **(First-In First-out).**
- Assim como uma pilha a Fila pode ser representada como uma lista com algumas restrições.
- Novos elementos são adicionados sempre no final da fila e são removidos sempre do início da fila.



Aplicação de Filas

- **Filas** são utilizadas em diversas aplicações, que precisam garantir o controle de sequência de execução como:

  - ***Impressoras.***
    - *Controle da fila de documentos a serem impressos e a sequência de impressão de suas respectivas páginas.*

  - ***Roteadores***
    - *Controle de fluxo dos pacotes de dados, podendo implementar diversas técnicas de controle de prioridades.*

  - ***Controle de Acesso.***
    - *Filas de acesso a sistemas ou recursos de um dado sistema. Podendo implementar recurso avançados de prioridade.*

Filas

- **Como implementar uma Fila ?**
  - **Precisamos de quatro variáveis:**
    - Uma estrutura capaz de armazenar objetos de forma sequencial, podendo ser um vetor ou uma lista.
    - E registradores para armazenar a posição inicial e final da fila e um contador que registra a quantidade de elementos da fila.
  - **Precisamos de quatro métodos:**
    - **Adiciona** que adiciona um novo item ao vetor ou lista e atualiza a posição do final da fila.
    - **Remove** que retira o objeto do início da fila.
    - **ExibirInicio** retorna o primeiro item da fila sem remover.
    - **Tamanho** que retorna a quantidade de elementos na fila

Interface pública de uma Fila em JAVA

```
public class Fila {

   List fila= new ArrayList<Object>();

    public void adiciona(Object item);
    public Object remove()
    public Object exibirInicio();
    public int tamanho();}
```

A classe Fila pode incluir outros métodos públicos, como **toString()** que imprime todos os elementos da fila sem remove-los

Em Java todos os objetos derivam da Classe ***Object***, portanto uma lista do tipo ***Object*** pode receber qual objeto.

Filas na API Java

- Em Java temos a Classe **Queue** na biblioteca ***java.util*** que implementa a estrutura de dados de uma fila.
  - **Enqueue** – Coloca um item na fila;
  - **Dequeue** – Retira o primeiro item da fila e retorna uma referência;
  - **Peek** – Retorna o primeiro item.
- ***Exemplo:*** criando uma fila de objetos do tipo Livro.

```
Queue objFila = new Queue();

objFila.Enqueue("A");
objFila.Enqueue("B");
objFila.Enqueue("Item 3");

string primeiroItem = objFila.Peek().ToString();
primeiroItem = objFila.Dequeue().ToString();
```

Exercício

- Crie uma **Classe** in Java com o nome **Fila** e implemente os métodos padrões .
- Implemente os métodos da interface.
- Teste o objeto fila adicionando uma lista sequencial de números e os removendo em seguida.

Interface pública de uma Fila em JAVA

```
public class Fila {
   List fila= new ArrayList<Object>();

    public void adiciona(Object item);
    public Object remove()
    public Object exibirInicio();
    public int tamanho();
}
```

Utilize os métodos do objeto ArrayList() :

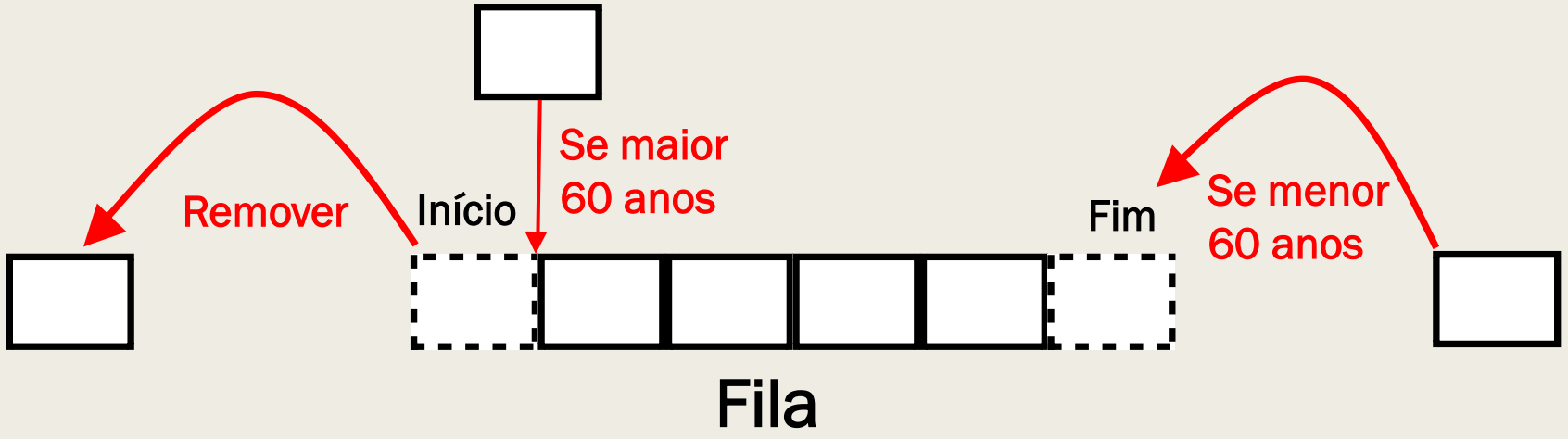
- **.add();** que permite adicionar itens a lista
- **.remove(index lista);** para remover itens.
- **.size();** que retorna o tamanho da lista.
- **.isEmpty();** que informa se a lista está vazia.

Em Java todos os objetos derivam da Classe ***Object***, portanto uma lista do tipo ***Object*** pode receber qual objeto.

Filas de Prioridades

- Imagine uma fila de pessoas em um banco, porém adicionamos uma nova regra de prioridade, na qual é definido que pessoas maiores de 60 anos entram no início da fila, independente da ordem de chegada.

- Neste caso o conceito FIFO **(First-In First-out**), não é mais válido, porém a adição de novas regras pode incrementar as funcionalidades da Fila .



Filas de Prioridades

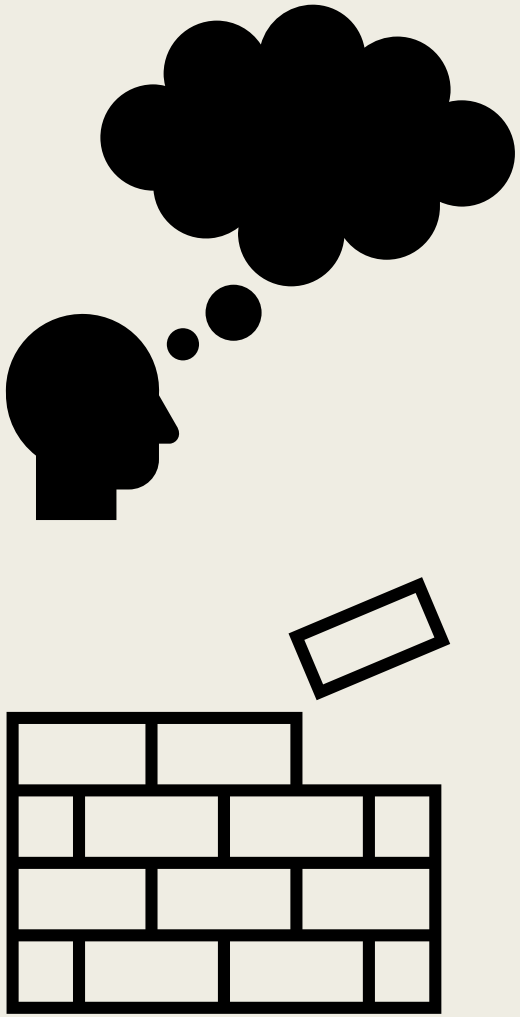
- A implementação da fila de prioridade utiliza os mesmos métodos da fila comum, porém com regras de prioridade que irão definir a inclusão dos itens.
- O método de Remoção da fila continua o mesmo, sempre é removido o primeiro item.
- As regras de prioridade devem ser definidas conforme a necessidade do projeto.
- Por exemplo: em uma fila de alunos, os alunos podem ser adicionado seguindo a ordem alfabética dos nomes ou por idade.

Filas de Prioridades

- **Como implementar uma fila de prioridades ?**
  - A utilização de listas ligadas na implementação de filas de prioridade é recomendada.
  - Utilize o método **.add(int index, String element)** para adicionar item em qualquer posição da lista.

- **Exercício**: crie uma classe **FilaPrioridade** de forma similar a classe **Fila**, porém modifique o método adicionar incluindo a seguinte regra:
  - Adicione os itens de menor valor no início da fila e de maior valor no final. Valores intermediários devem ser inseridos no meio da fila, seguindo uma ordenação com valores crescentes do início para o final da fila.

- Agora pense nos conceitos de herança de POO para reaproveitar o código da classe fila, e crie uma nova classe **FilaPrioridade**.



Listas

- Conceitos.
- Aplicação.

Lista Ligada

- **Vetores** são uma excelente **alternativa** ao uso direto de **arrays em Java**, porém para algumas tarefas eles podem **não ser eficientes**.

- Adicionar um **elemento na primeira posição de um Vetor**, por exemplo, consome **muito tempo**, pois temos de **deslocar** todos os outros elementos uma **posição para a frente.**

- A **performance** dessa operação **degrada** a medida que a **quantidade de elementos** do nosso vetor **cresce**: ela consome tempo linear em relação ao número de elementos.

- **Analogamente**, remover um **elemento da primeira posição** implica em **deslocar** todos os **outros elementos** que estão na sua frente para trás.

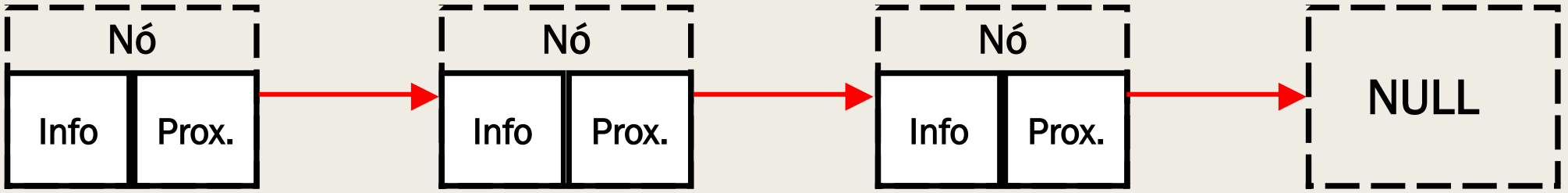
Lista Ligada

- Para manipular uma estrutura sem ter de alterar todos seus vizinhos, precisamos de algo que não seja fixo.

- A solução é o uso do conceito de **lista ligada**, onde cada elemento da lista é **representado por um Nó** que possui **dois campos**:

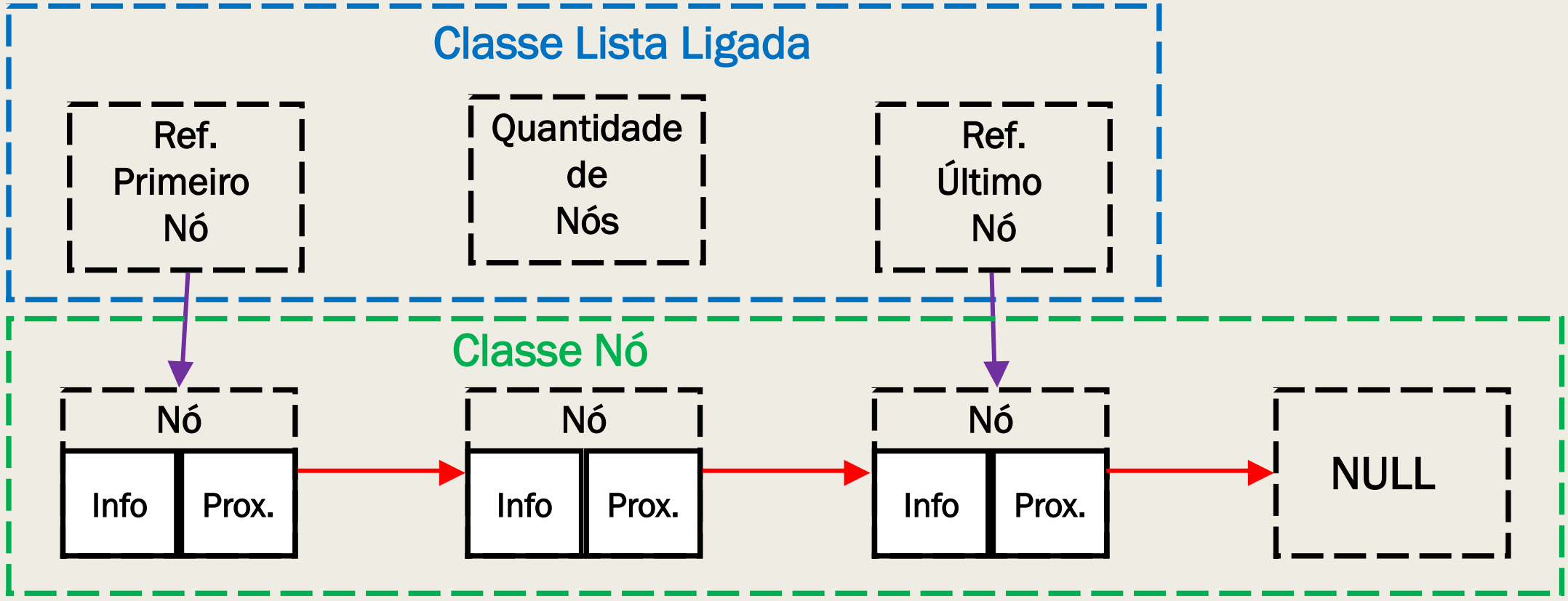
  - **Info**: que armazena as informações armazenadas na lista.

  - **Próximo**: um ponteiro para o próximo Nó da lista. Sendo que o ultimo Nó da lista irá apontar para um valor **Null**.



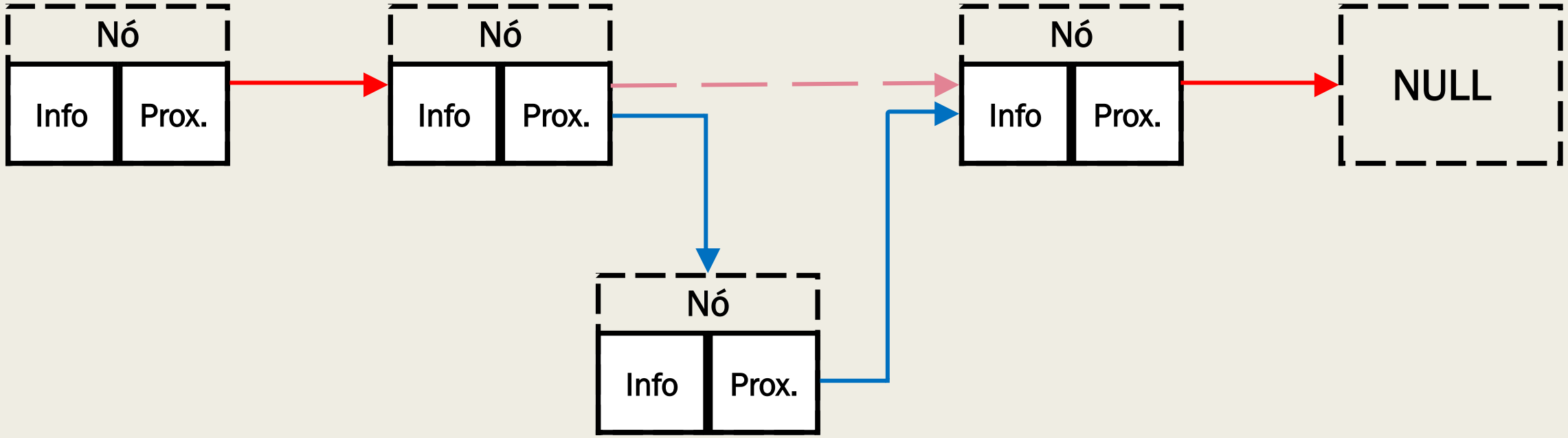
Lista Ligada

- Para **permitir** a operação da lista ligada, além da estrutura do Nó que em orientação a objeto será representada por uma **classe Nó**.
- Também é necessário: um **contador de elementos da lista** e uma referência para o **primeiro e último Nó da lista**, de forma a referenciar a lista na memória.



Lista Ligada

Adição ou remoção de novos elementos da lista.



Lista Ligada

Adição de novos elementos na lista.

1. *Verificar se o posição para inserir o elemento é válida. Utilizar um método interno para percorrer a lista partindo do primeiro Nó.*
2. *Se for válido, copiar a referência do atributo **próximoNó** da posição anterior a posição desejada no atributo **próximoNó** do Nó que está sendo inserido.*
3. *Atribuir a referência do novo Nó que foi criado ao atributo **próximoNó** da posição anterior.*
4. *Incrementar o contador de elementos da lista.*

Adição de novos elementos nas posições inicial ou final da lista.

1. Realizar os procedimentos anteriores, atualizando os atributos **primeiroNo** ou **ultimoNo** respectivamente. Após realizar a adição do novo elemento.

Lista Ligada

Remoção de novos elementos na lista.

1. *Verificar se o posição para remover o elemento é válida. Utilizar o contador de elementos da lista para validação.*
2. *Se for verdadeiro, atribuir a referência do objeto Nó que está após a posição desejada ao atributo* ***proximoNó*** *do objeto que está antes da posição desejada.*
3. *Decrementar o contador de elementos da lista.*

Remoção de novos elementos nas posições inicial ou final da lista.

1. Realizar os procedimentos anteriores, atualizando os atributos **primeiroNo** ou **ultimoNo** respectivamente. Após realizar a remoção do nó.

Lista Ligada

Exercício: crie uma nova classe com o nome de **No** que contenha dois atributos: **info do tipo Object** e **proximoNo do tipo No** e seus respectivos métodos ***getter*** e ***setter***. O construtor dessa classe deverá receber como parâmetros **info** e **proximoNo**.

Classe No

(-) info: Object
(-) proximoNo: No

---

public No (info,proximoNO);

//Getter e Setters dos atributos

Lista Ligada

Continuação: Crie uma nova classe com o nome de **ListaLigada** que contenha os atributos: **primeiroNo do tipo No** e **ultimoNo do tipo No** e **quantidadeNos** do tipo Inteiro e seus respectivos métodos **getter** e **setter**. O construtor dessa classe deverá receber como parâmetros a informação que será armazenada na lista, e adicionar a informação no primeiro nó da lista.

Métodos Públicos:

void adicionar(Object elemento); *// cria um novo nó no final da lista.*

void adicionar(Object elemento, int posicao); *// cria um novo nó e insere na posição determinada.*

void remover(int posição); *//remove o nó definido pela posição da lista*

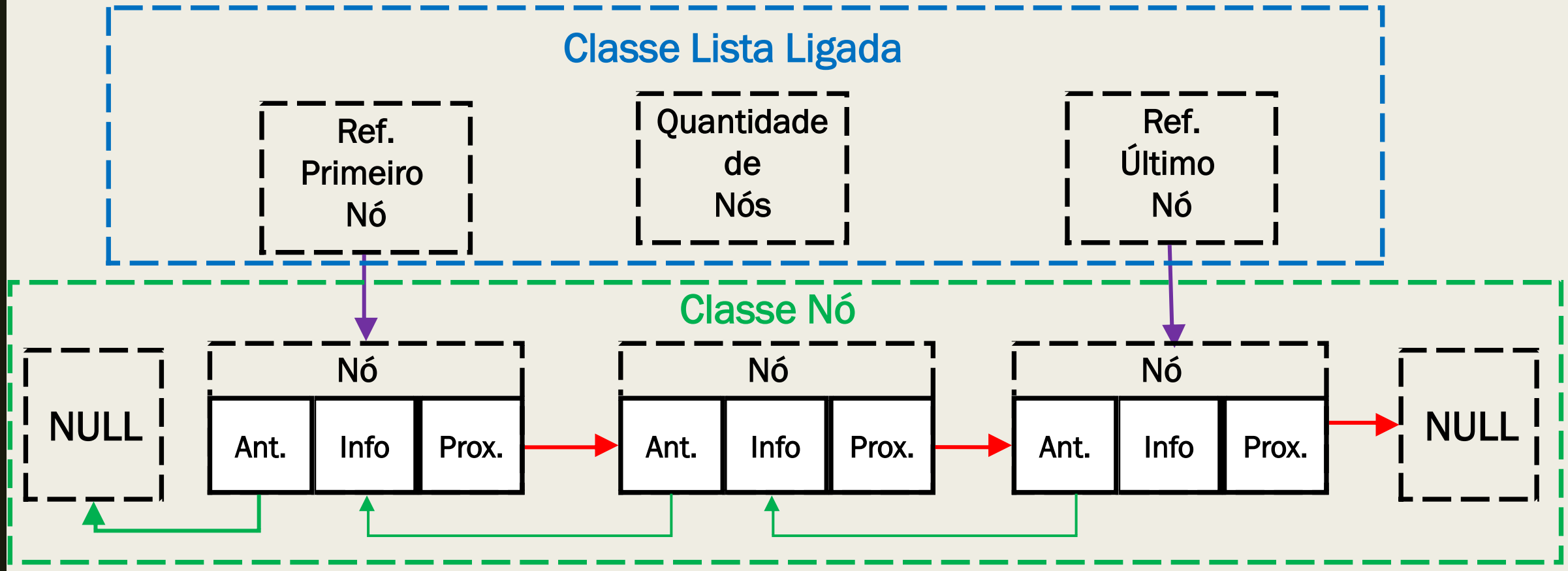
Object getLista(int posição); *// retorna a informação do nó definido pela posição.*

int tamanhoLista(); *//retorna a quantidade de elementos da lista.*

String imprimirLista*();// retorna uma string com as informações do primeiro ao último item concatenado.*

Lista Duplamente Ligada

- Uma lista **duplamente ligada**, possui o mesmos métodos e mecanismos de funcionamento da lista ligada, no entanto **o Nó possui dois ponteiros** um apontando para o **próximo Nó** e outro apontando para o **Nó anterior**.



Lista Duplamente Ligada

Exercício: Crie uma nova classe **ListaDuplamenteLigada,** e uma classe **NoDuplo** com dois ponteiros para o Nó anterior e posterior, implemente:

Métodos Públicos:

void adicionar(Object elemento); *// cria um novo nó no final da lista.*

void adicionar(Object elemento, int posicao); *// cria um novo nó e insere na posição determinada.*

void remover(int posição); *//remove o nó definido pela posição da lista*

Object getLista(int posição); *// retorna a informação do nó definido pela posição.*

int tamanhoLista(); *//retorna a quantidade de elementos da lista.*

String imprimirLista*();// retorna uma string com as informações do primeiro ao último item concatenado*

Exercícios

Exercício: Implemente um método chamado **adicionaOrdenado** nas classes **ListaLigada** e **ListaDuplamenteLigada**, que permita adicionar valores numéricos em ordem crescente ou decrescente.
Teste o método adicionando uma sequência de números aleatórios, ordenados de forma crescente e decrescente e depois utilize o método **imprimirLista** para validar.

```
void adicionarOrdenado(int valor, TipoOrdenacao tipo);
```

Crie uma classe **Enum TipoOrdenacao** com duas constantes, crescente e decrescente, utilize o tipo Enum para escolher o tipo de ordenação desejada.

```
public enum TipoOrdenacao {
    CRESCENTE,DECRESCENTE;
}
```

Lista Circular Duplamente Ligada

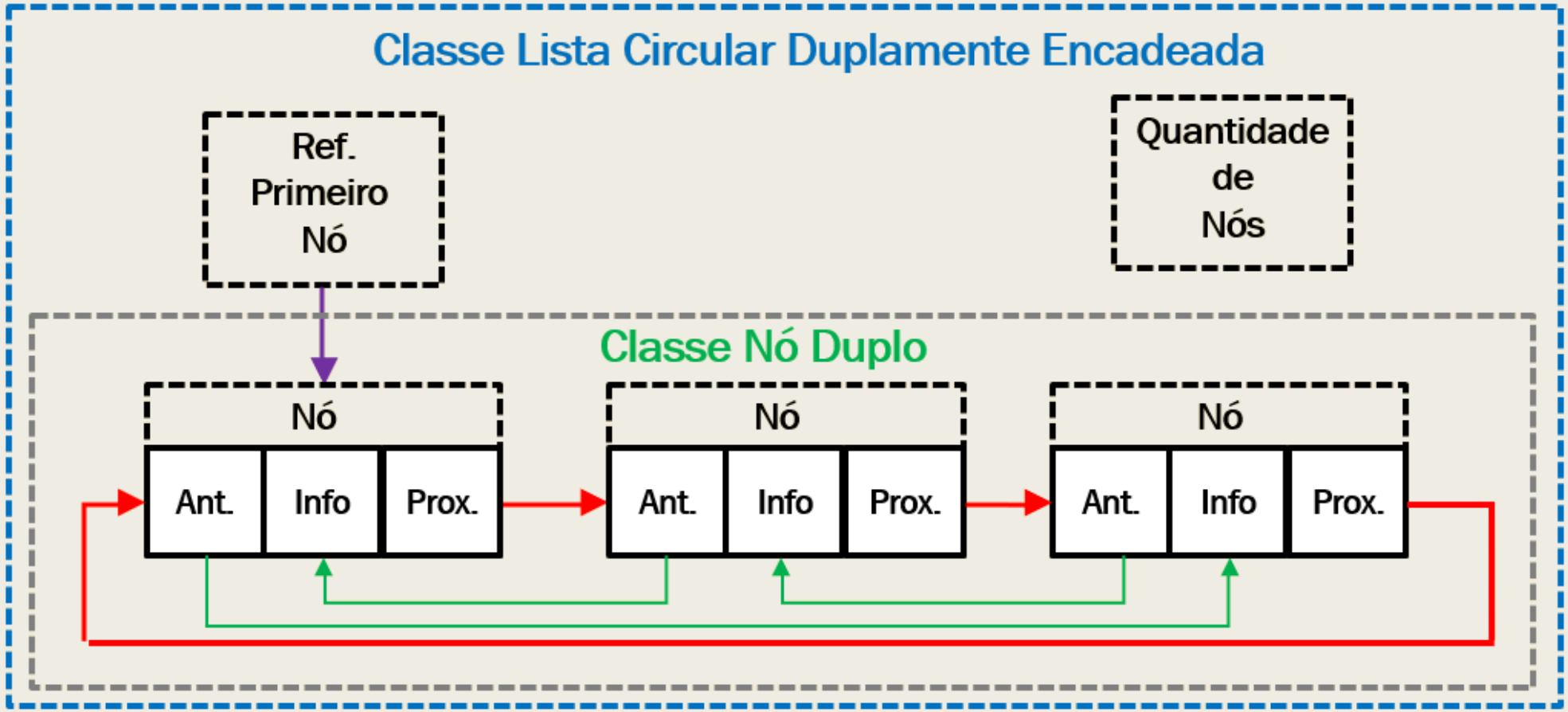
A Lista Circular Duplamente Ligada utiliza nós duplos com dois ponteiros que apontam para o próximo item e para o item anterior.

Com a diferença se comparado a Lista Duplamente Ligada, sendo o fato do início e fim da lista não estarem apontando para NULL.

Ao invés disso o último item da lista aponta para o primeiro e o primeiro para o último.

Formando assim um círculo, vindo daí o nome “Circular”.

Lista Circular Duplamente Ligada



Lista Circular Duplamente Ligada

Para referenciar a lista na memória é utilizado apenas um único nó denominado “Primeiro Nó”, que sempre deverá apontar para o início da lista.

Também há um contador de nós que auxiliar nos processos de adição e remoção dos nós.

Com a adição e remoção de nós sendo similar a Lista Duplamente Ligada, contudo é necessário tomar o cuidado de atualizar a referência do “Primeiro No”, sempre que for adicionado ou removido itens na posição 0 do índice.

Garantindo assim que a lista manterá sua consistência.

Exercício Lista Circular

1) Complete o código do método ***adicionar(Object item, int indice)***, para permitir adicionar novos nós em qualquer posição da lista, incluindo a primeira e última posições.
2) Complete o código do método ***remover(int indice)***, para permitir remover os nós em qualquer posição da lista, incluindo a primeira e última posições.
3) Complete o código do método ***imprimirLista(String resposta, NoDuplo auxiliar, int contador)*** de forma a permitir a caminhada recursiva pela lista circular, registrando na variável resposta o valor dos nós e retornando no final da execução do método.



Árvores

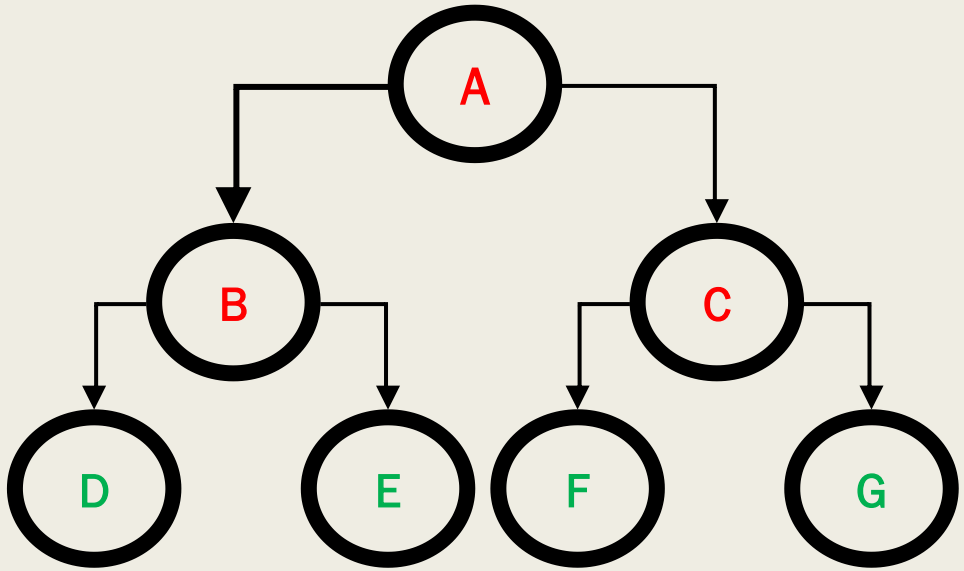
- Conceitos.
- Aplicação.



Conceito de árvores

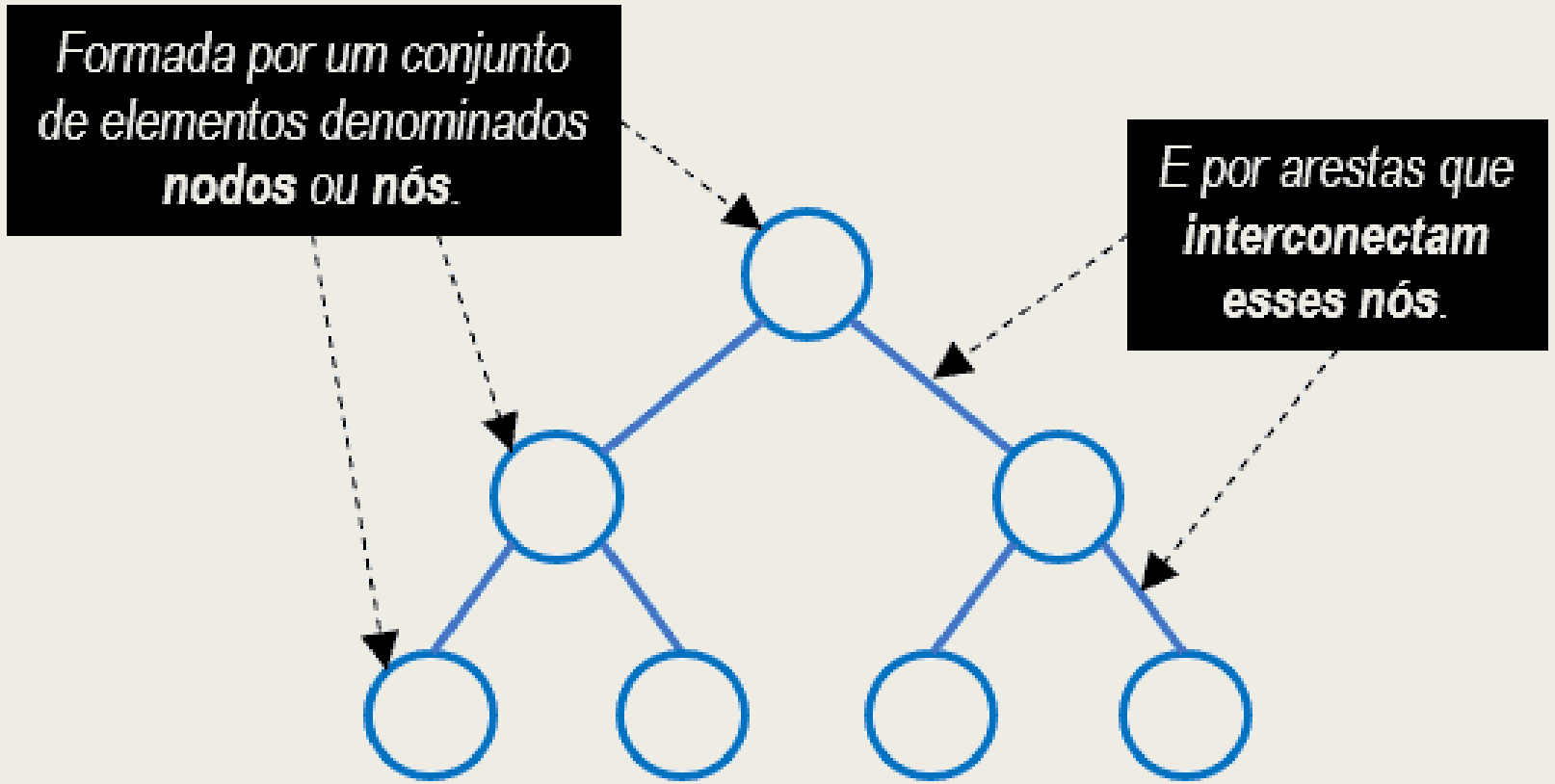
- As árvores representam um dos tipos de estruturas de dados mais importantes da computação.

- A estrutura de uma árvore define uma organização de elementos dispostos de forma hierárquica.



Estrutura de uma árvore

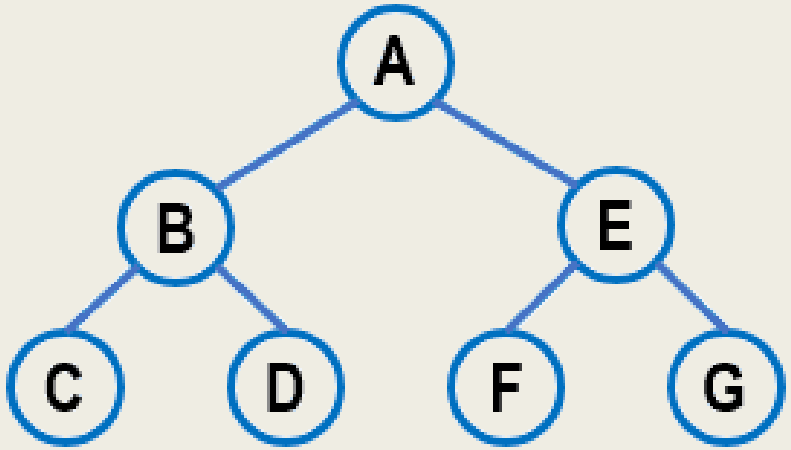
- Toda árvore é formada por dois elementos básicos denominados:
  1. **Nós** (ou nodos)
  2. **Arestas** que interconectam estes nós.



Estrutura de uma árvore

- A grande diferença de uma árvore para outras estruturas de dados como: listas, filas e pilhas é que as árvores não são organizadas em uma sequência.
- Em uma árvore não podemos determinar o próximo elemento e o elemento anterior, porque as suas informações não se organizam de maneira enfileirada.
- Por este motivo, as árvores são denominadas **estruturas não lineares**. E isto a torna uma estrutura bastante especial, com diversas propriedades e características diferenciadas.

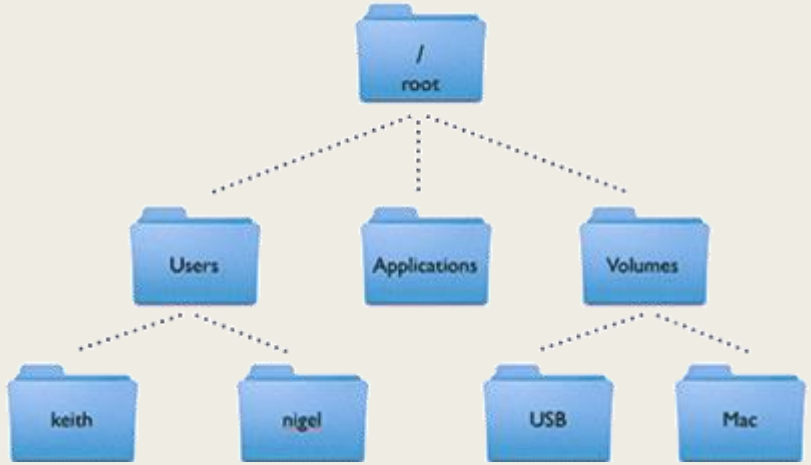
A	B	C	D	E	F	G
0	1	2	3	4	5	6

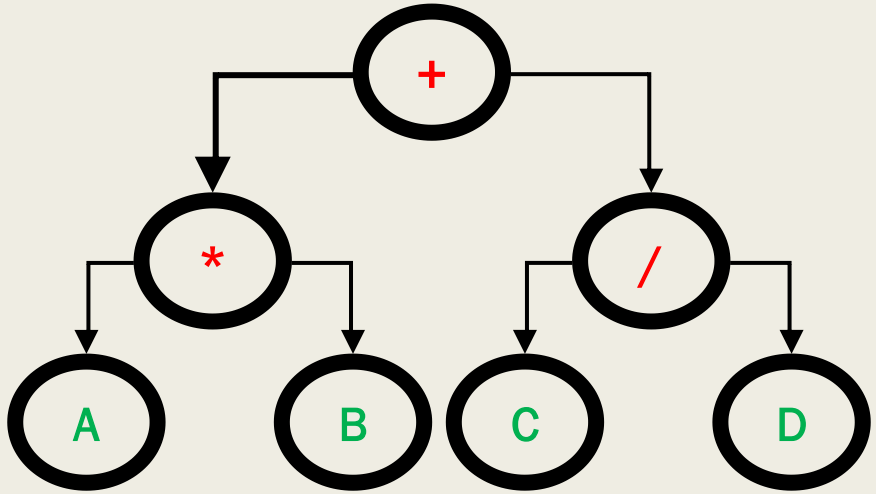


Conceito de árvores

- Os nós de uma árvore representam espaços onde podem ser armazenados diversos tipos de informações, tais como:
  - Um telefone, uma data, um nome, uma lista de e-mails, um perfil de um usuário em uma rede social ou uma função matemática, para citar alguns.

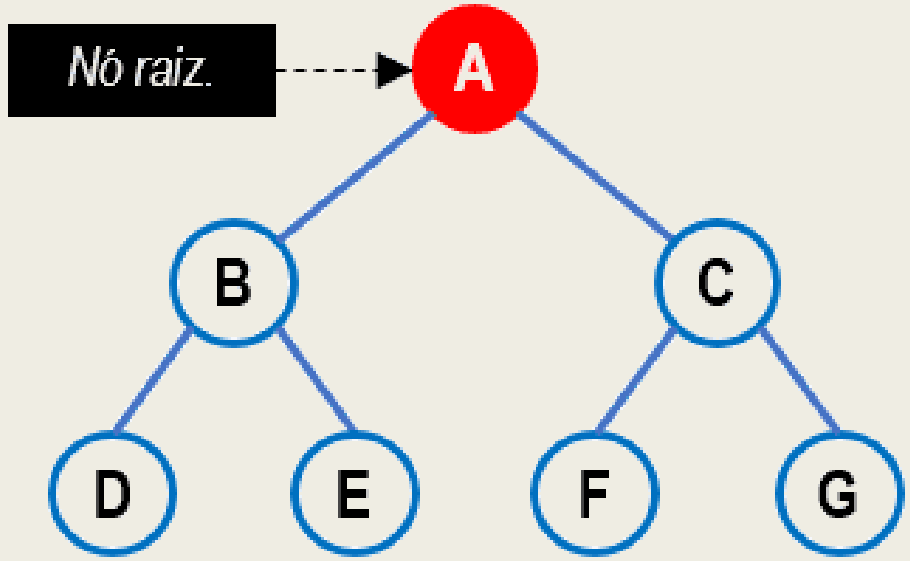
- As arestas, representam relacionamentos entre esses nós. Representam a forma com a qual os nós se conectam, e essa relação depende totalmente da forma como a árvore é utilizada e do tipo de informação que os nós da árvore armazenam.





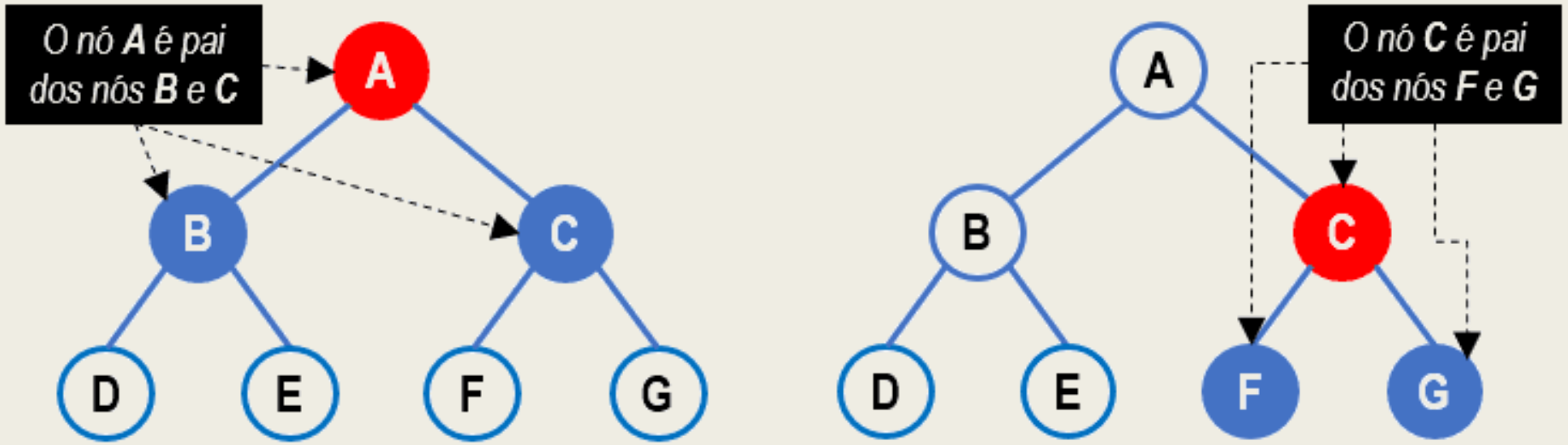
Propriedades hierárquicas de uma árvore

- **Nó raiz**
  - Toda a árvore possui um nó principal, denominado **raiz**. A raiz representa o nó inicial de todos os tipos de árvore.



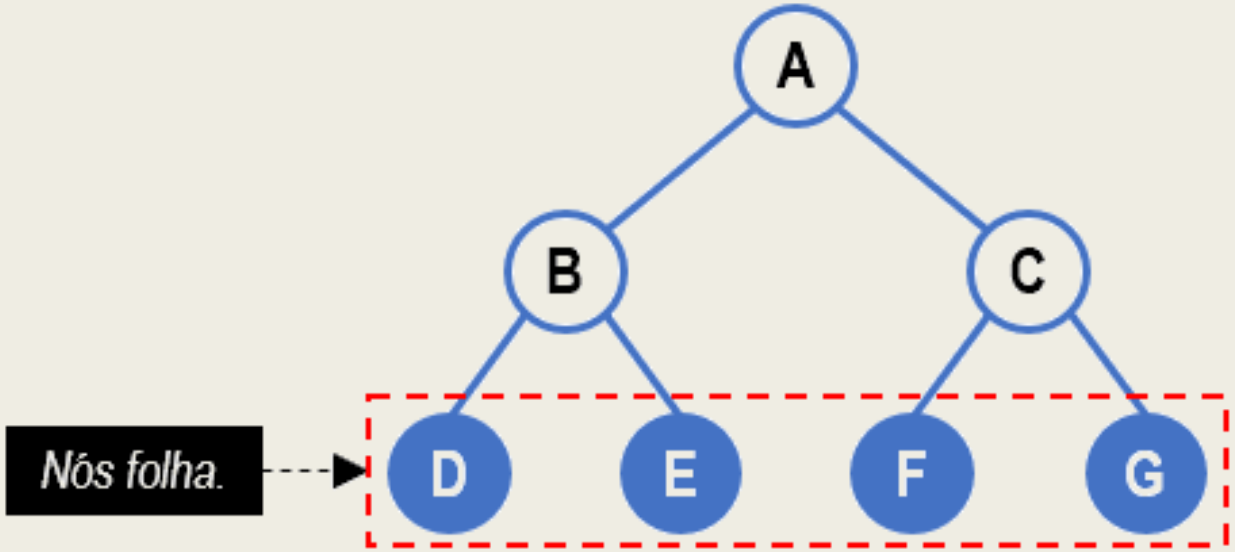
Propriedades hierárquicas de uma árvore

- **Nós filhos e nós pais**
  - A partir da raiz, todos os nós inferiores ligados a um nó superior são denominados **nós filhos**. Todos os nós superiores, que geram nós inferiores, são chamados de **nós pais**.



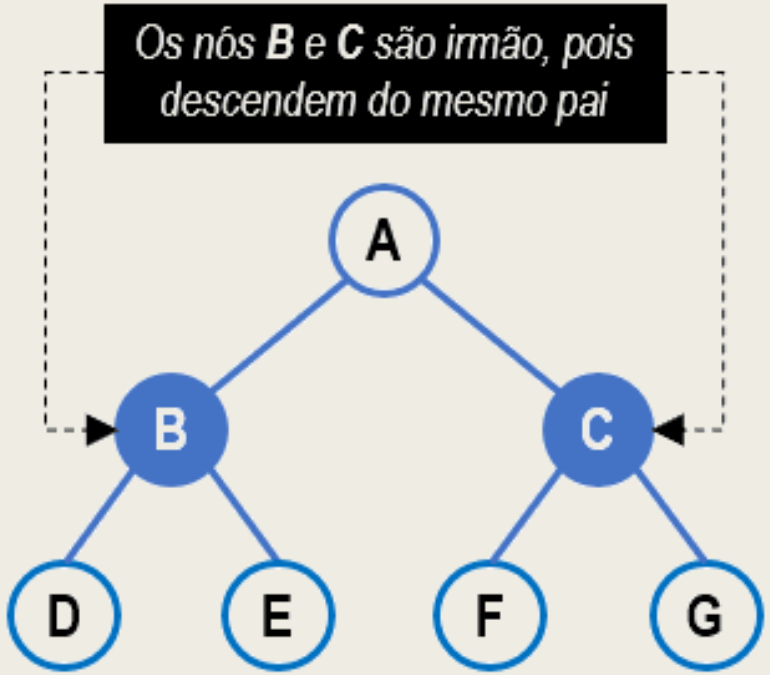
Propriedades hierárquicas de uma árvore

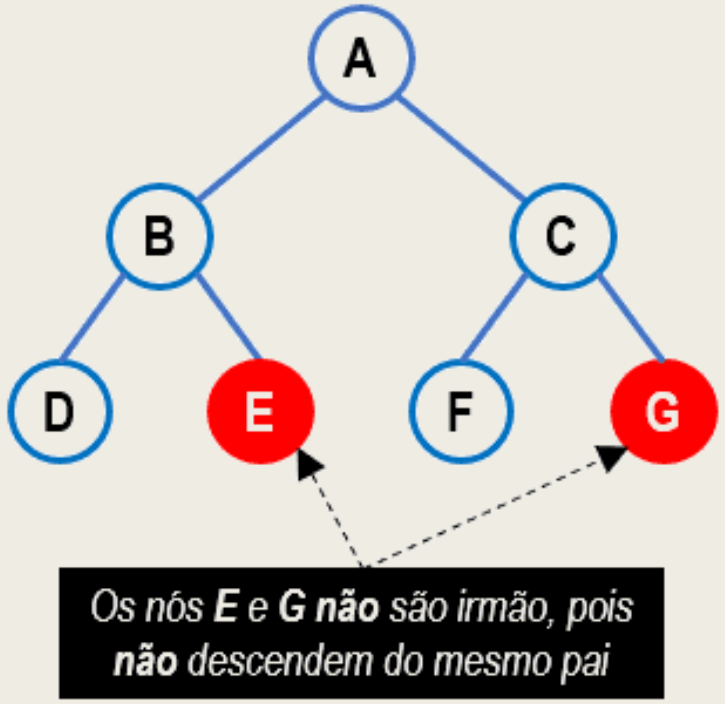
- **Nó folha**
  - Os nós que não possuem filhos são denominados nós folha.
  - O termo *"nó folha"* faz alusão às folhas de uma árvore, já que elas representam o final de um galho. A imagem abaixo ilustra essa metáfora.



Propriedades hierárquicas de uma árvore

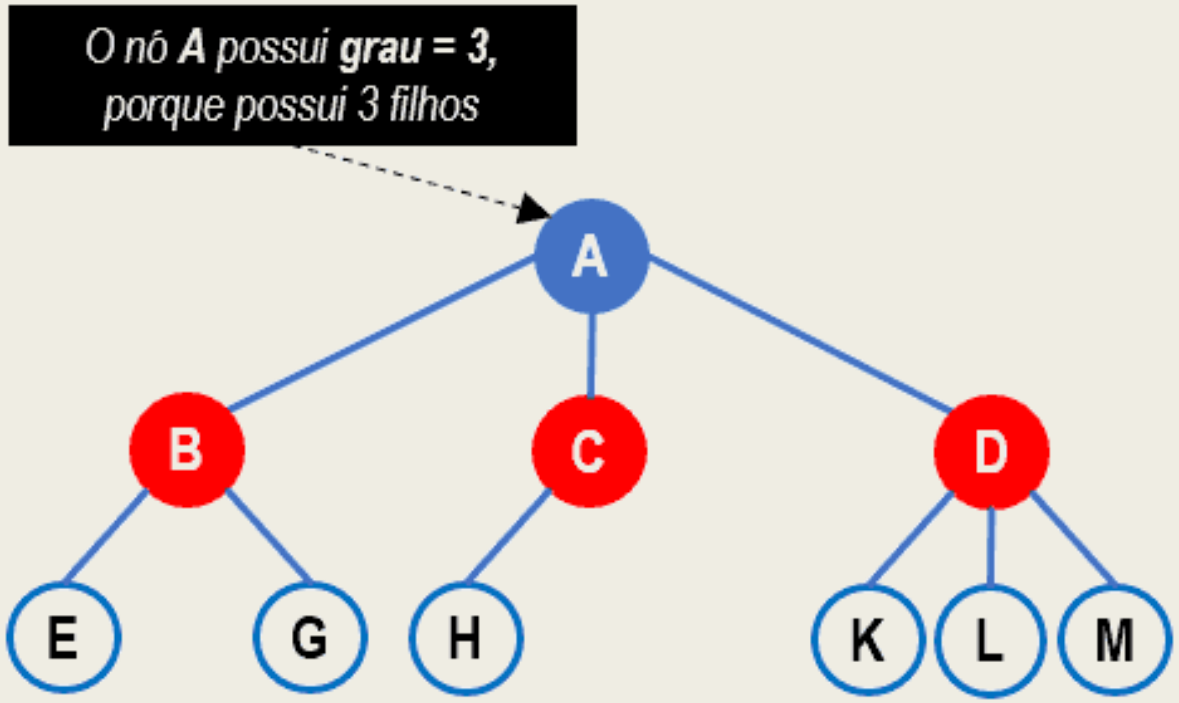
- **Nós irmãos.**
  - Outro conceito importante é o conceito de irmandade entre os nós. Dois nós são irmãos se eles descendem diretamente do mesmo nó pai.





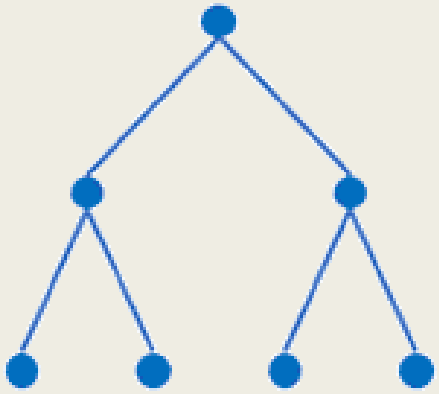
Propriedades hierárquicas de uma árvore

- **Graus de uma árvore.**
  - O grau é a propriedade que qualifica os nós de uma árvore, ela define a quantidade de filhos que cada nó de uma árvore possui.

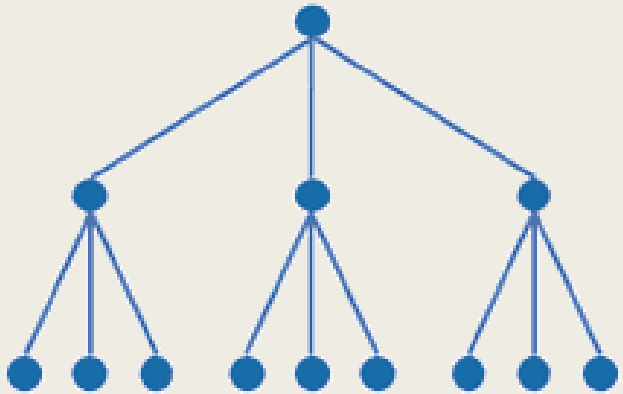


Propriedades hierárquicas de uma árvore

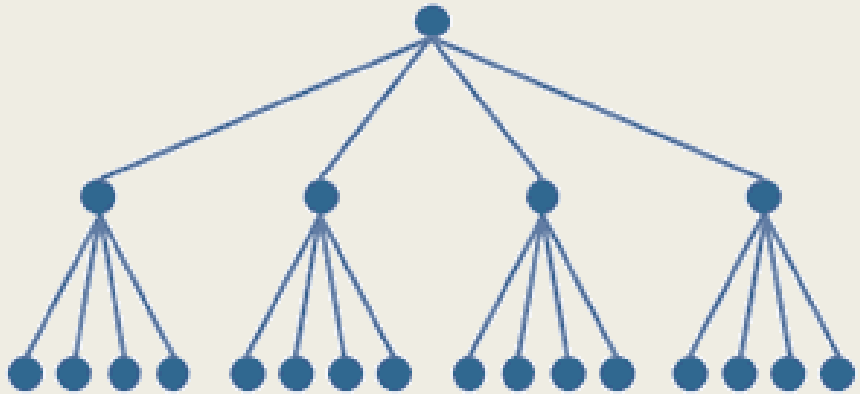
- **Ordem de uma árvore.**
  - A ordem é um conceito que qualifica toda a árvore. A ordem de uma árvore é definida pelo seu nó de maior grau. Ex.: se o maior grau da árvore é 3, então a árvore possui ordem 3.



Árvore de **ordem 2**



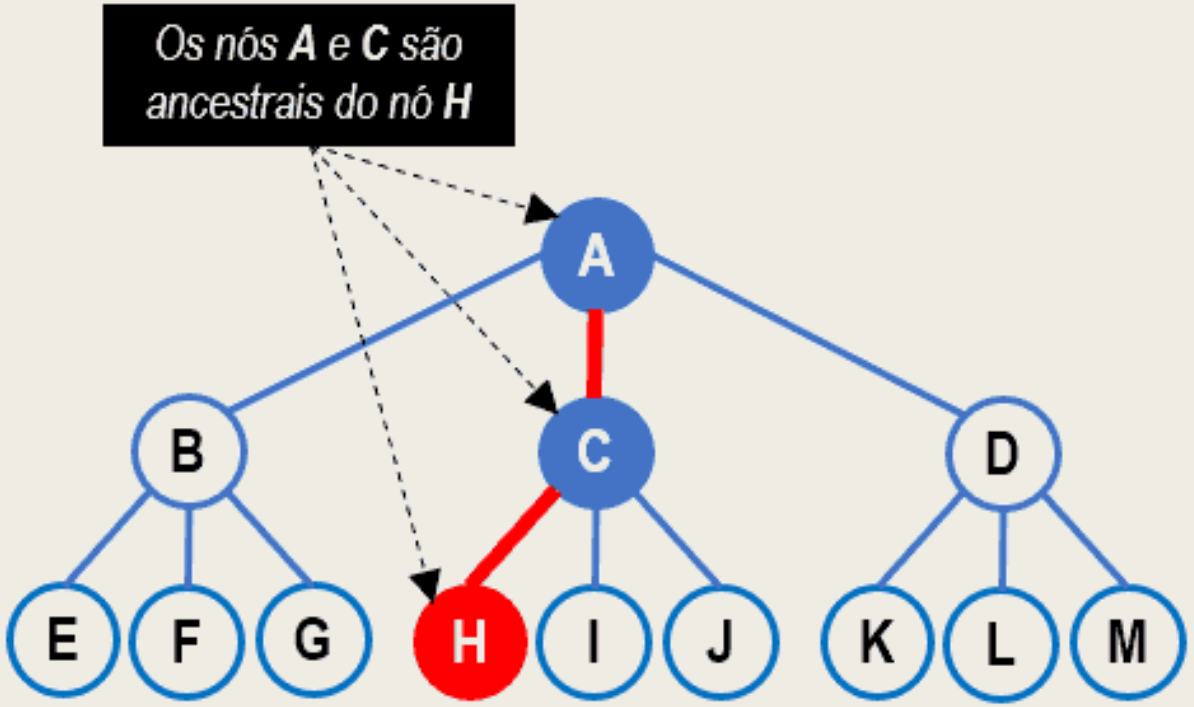
Árvore de **ordem 3**



Árvore de **ordem 4**

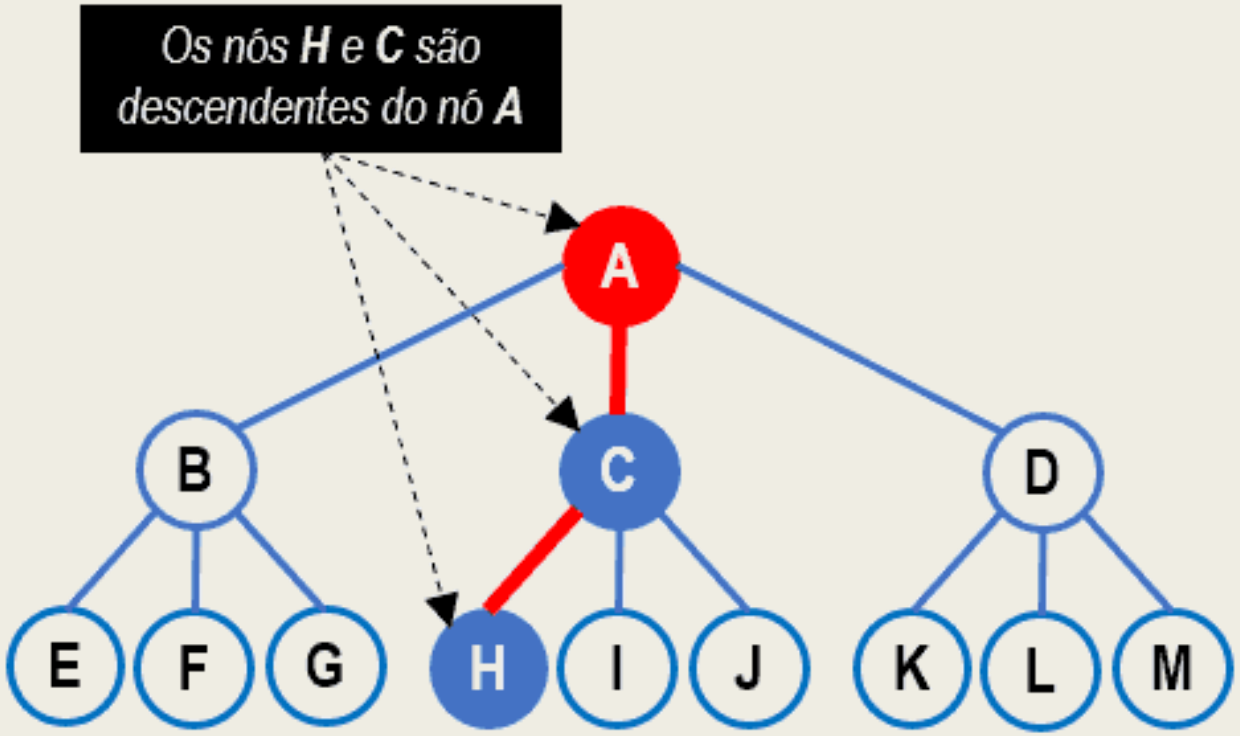
Propriedades hierárquicas de uma árvore

- **Ancestralidade.**
  - A ancestralidade é um conceito que qualifica nós superiores em relação a nós inferiores. Quando existe um conjunto de arestas que ligam um nó superior A a um nó inferior B, dizemos que A é ancestral de B.



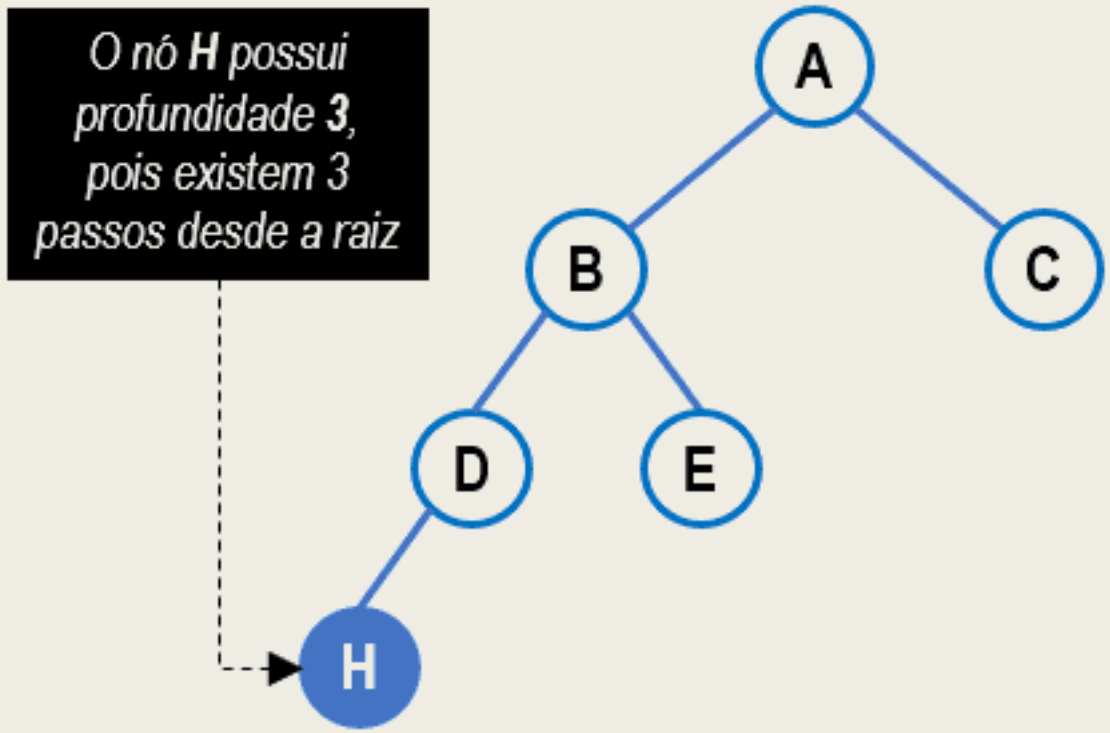
Propriedades hierárquicas de uma árvore

- **Descendência.**
  - A descendência é um conceito que qualifica nós inferiores em relação a nós superiores. Quando existe um conjunto de arestas que ligam um nó inferior B a um nó superior A, dizemos que B é descendente de A.



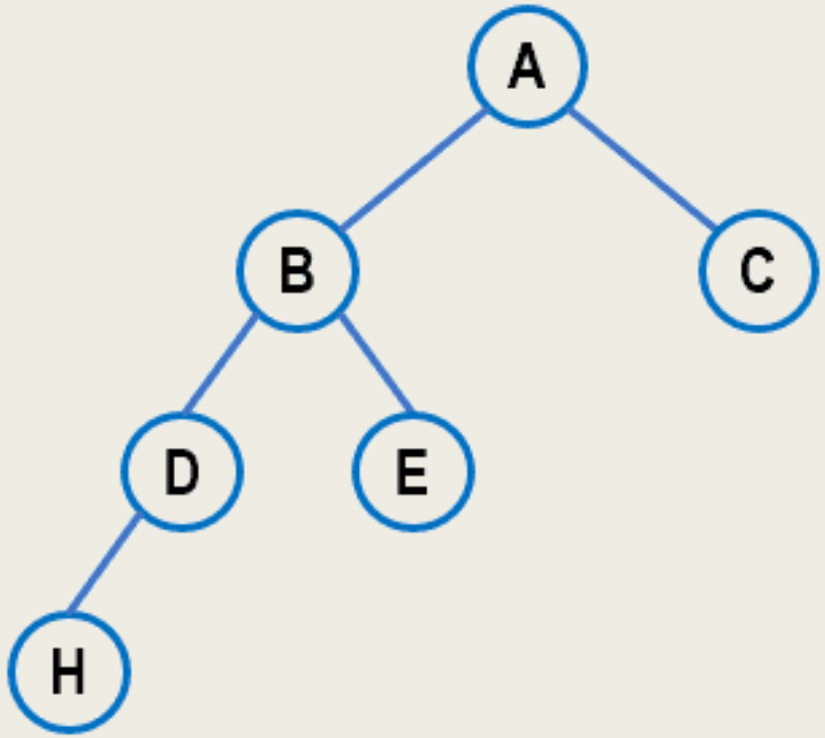
Propriedades hierárquicas de uma árvore

- **Profundidade de um nó**
  - A profundidade é um conceito que qualifica um nó de uma árvore. Ela define a distância de um determinado nó até a raiz.



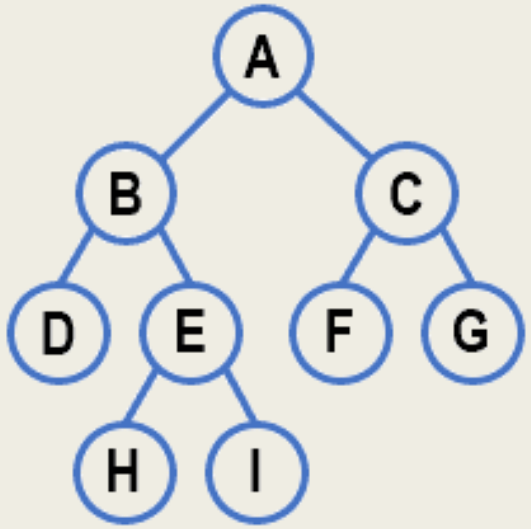
Propriedades hierárquicas de uma árvore

- **Nível de uma árvore**
  - O nível define as fatias de profundidade de uma árvore, de maneira que os filhos de um nó sempre são colocados em níveis posteriores.

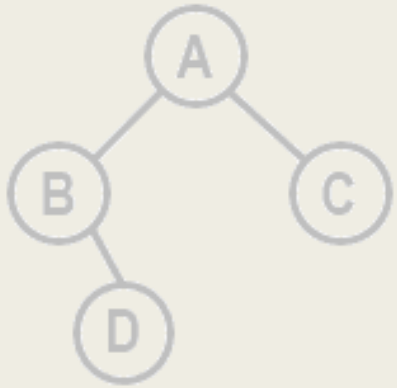


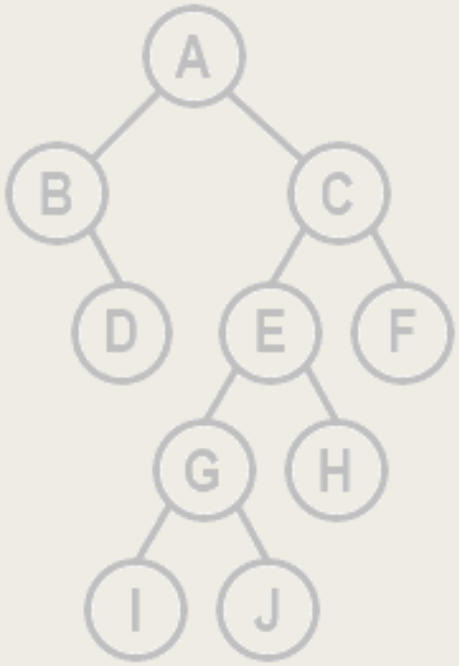
Propriedades hierárquicas de uma árvore

- **Altura de uma árvore**
  - Representa a maior profundidade da árvore. Representa a distância entre a raiz e um nó folha do maior nível da árvore.



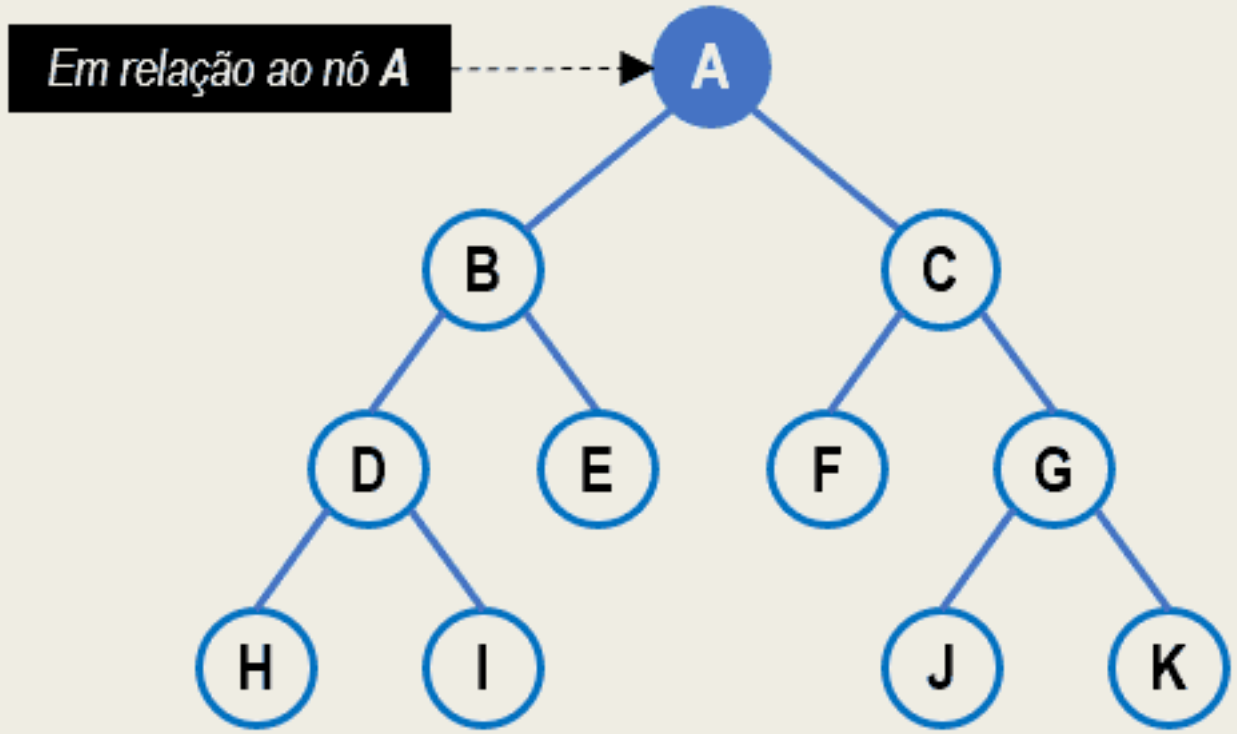
Altura = 3





Propriedades hierárquicas de uma árvore

- **Conceito de sub-árvore**
  - A partir da raiz, cada novo filho representa uma **sub-árvore**, uma nova árvore que se redefine de forma recursiva.



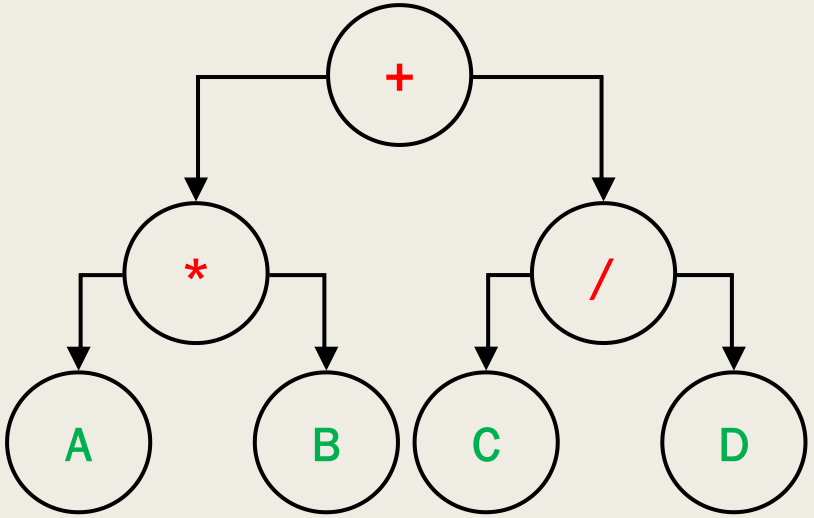
Árvore Binária

- Uma árvore binária é uma estrutura de dados mais geral que uma lista encadeada.
- É o tipo de árvore mais utilizado na computação.
- A principal utilização de árvores binárias são as árvores binárias de busca.
- Os nós de uma árvore binária possuem graus zero, um ou dois. Um nó de grau zero é denominado folha.

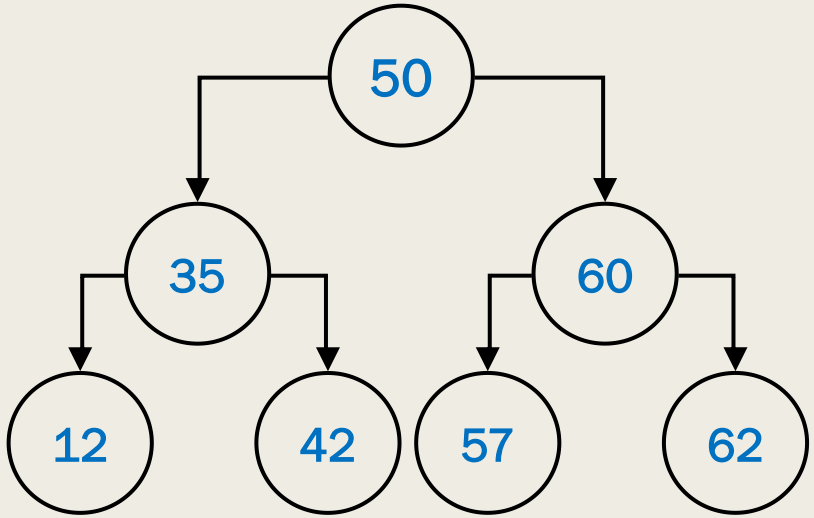
Aplicações de Árvores Binárias

Cálculo de expressões matemáticas

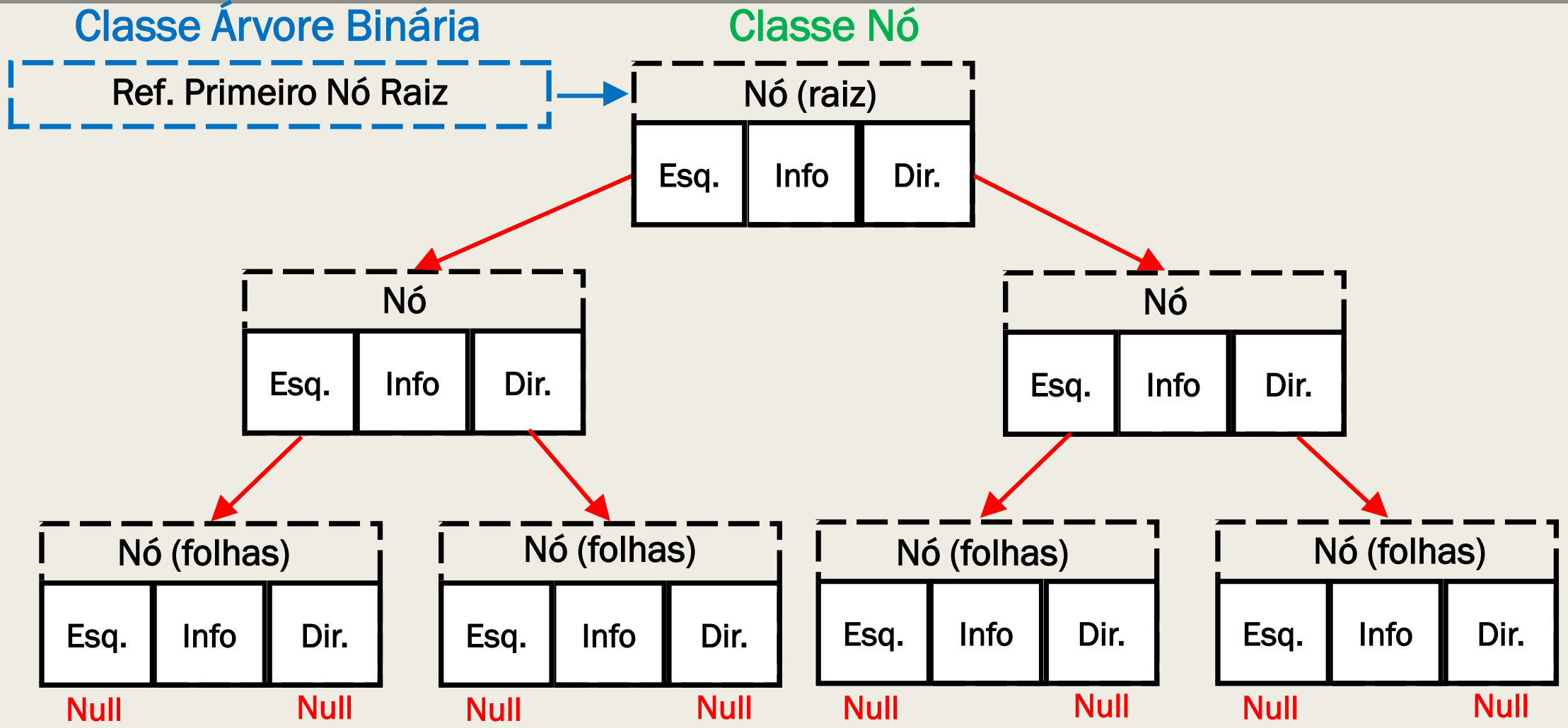
= (A * B) + (C /D)



Ordenação e Busca



Estrutura de Árvores Binárias



Exercício Árvores

Exercício 1.

- Crie uma classe Árvore Binária que armazene valores inteiros em seus nós de forma ordenada a esquerda valores menores e a direita valores maiores que o Nó Pai.

- Crie um método que busca um determinado nó na árvore retornando a profundidade do nó ou -1 se o nó procurado não existir.

- Crie um método que percorra toda a árvore imprimindo o valor e profundidade dos nós.

Exercício Árvores

Exercício 2.

- Crie uma classe Árvore capaz de armazenar qualquer quantidade de nós filho.
- Crie um método capaz de somar os valores armazenados em todos os nós filhos de um Nó Pai, retornando o valor da soma.

Percursos em Árvores Binárias

Há três métodos utilizados para percorrer uma árvore binária:

- **Pré-ordem:** os nós são visitados na seguinte ordem, primeiro o nó **pai**, em seguida o filho **esquerdo** e, por último, o filho **direito**. Essa técnica é útil para **criar uma cópia da árvore** ou para realizar operações de pré-processamento antes de visitar os nós filhos.

- **Em ordem** (Simétrico): a árvore é percorrida de forma que os nós sejam visitados na seguinte ordem: primeiro o filho **esquerdo**, depois o nó **pai** e, por fim, o filho **direito**. Essa técnica é comumente utilizada para obter os elementos de uma árvore binária de **busca em ordem crescente.**

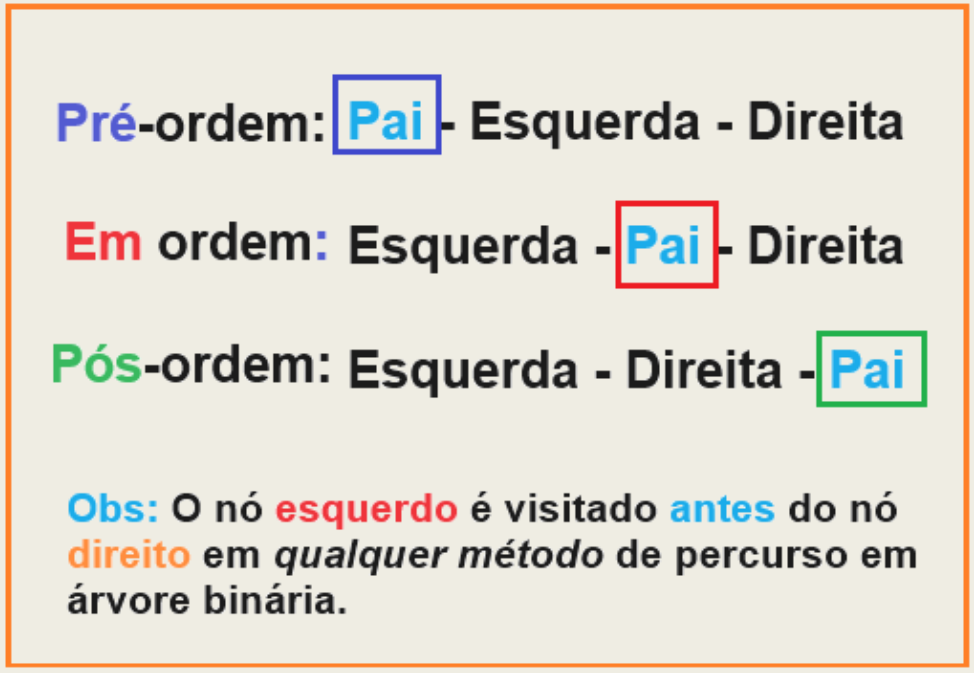
- **Pós-ordem:** envolve a visita aos nós na seguinte ordem: primeiro o filho **esquerdo**, depois o filho **direito** e, por último, o nó **pai**. Essa técnica é frequentemente empregada em cálculos que requerem informações dos nós filhos antes de processar o nó pai, como a avaliação de expressões aritméticas.

Resumo

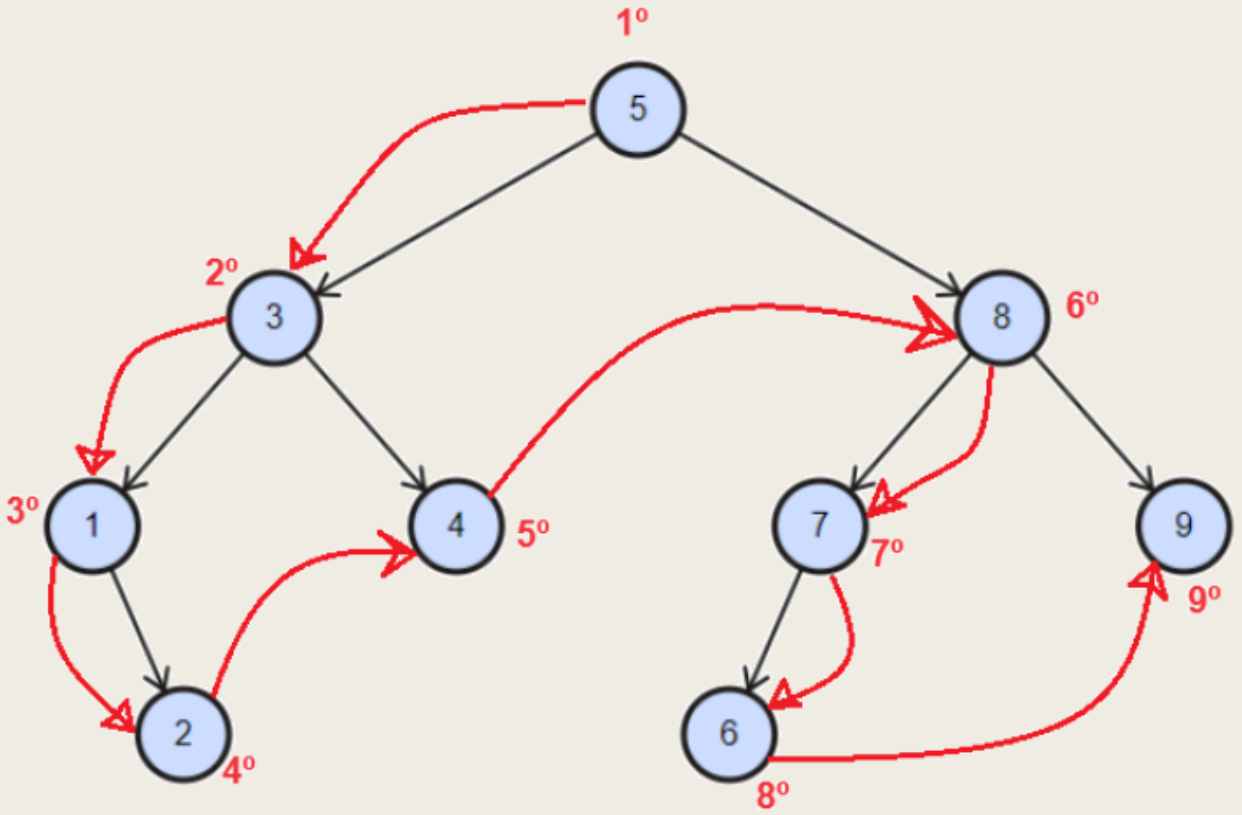
- O nó filho da **esquerda** é sempre visitado **antes** que o da **direita**:

- O nome do método se refere ao **momento** em que o **nó pai é visitado**, ou seja:

  - **Pré-ordem**: o pai é visitado antes dos filhos, ou seja, **Pai**-Esquerda-Direita;
  - **Em ordem**: o pai é visitado entre os filhos, ou seja, Esquerda-**Pai**-Direita;
  - **Pós-ordem**: o pai é visitado após os filhos, ou seja, Esquerda-Direita-**Pai**;

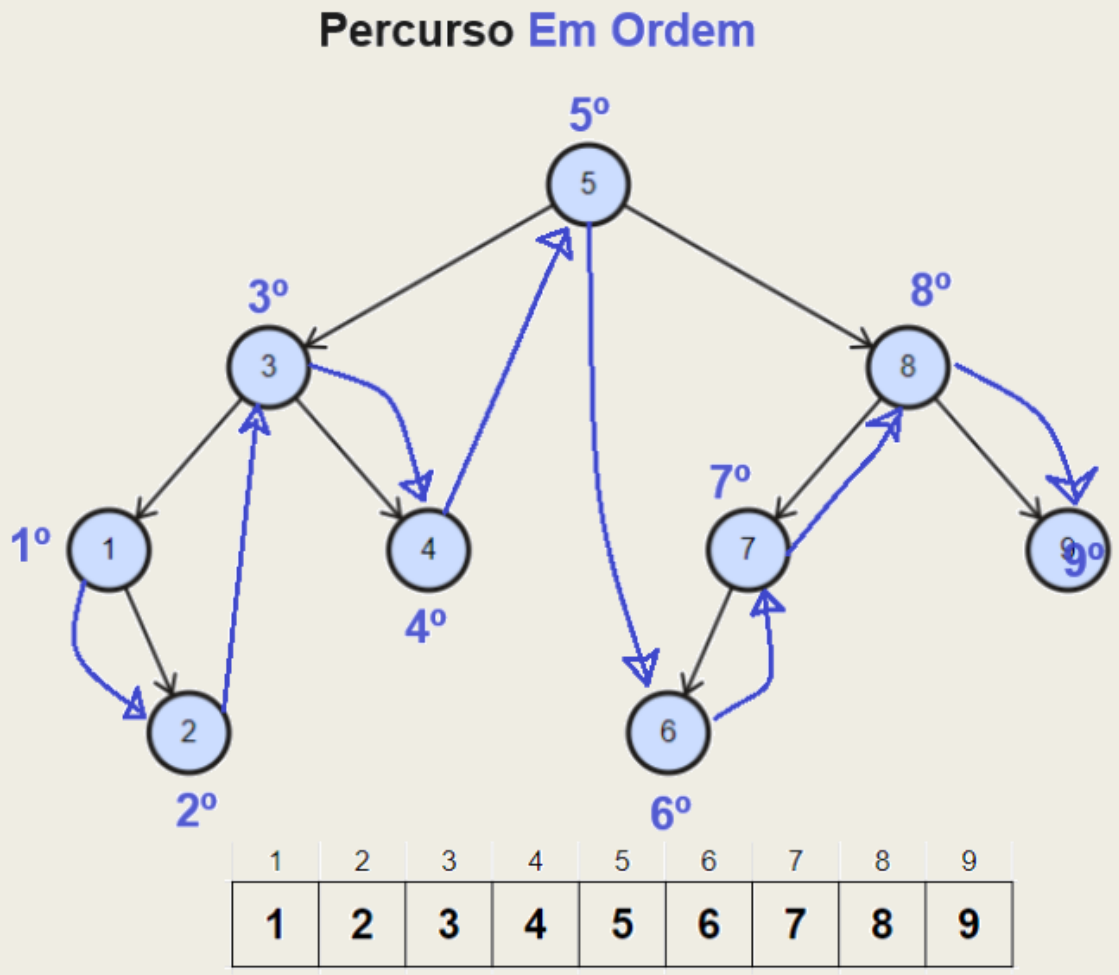


Percursos Pré ordem

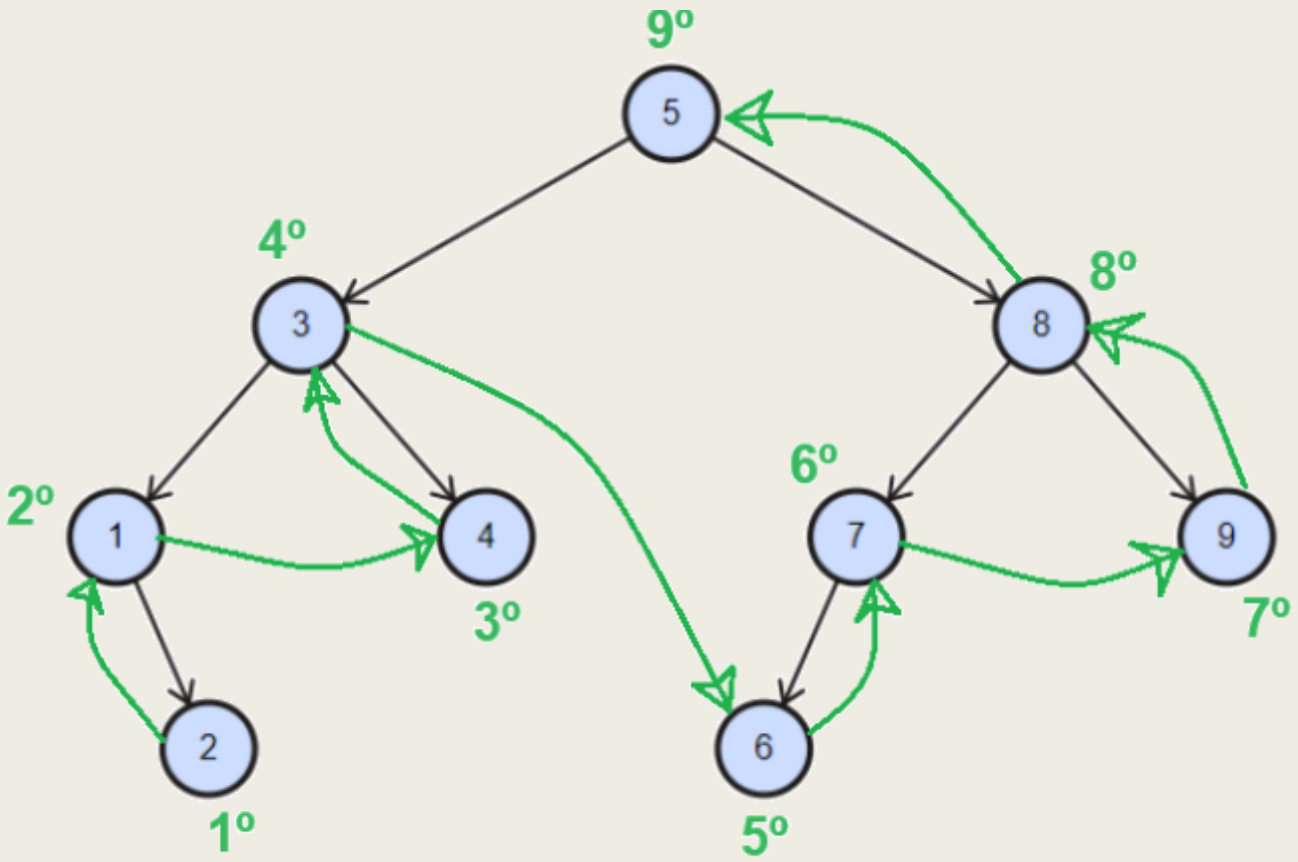


1	2	3	4	5	6	7	8	9
5	3	1	2	4	8	7	6	9

Percursos em ordem



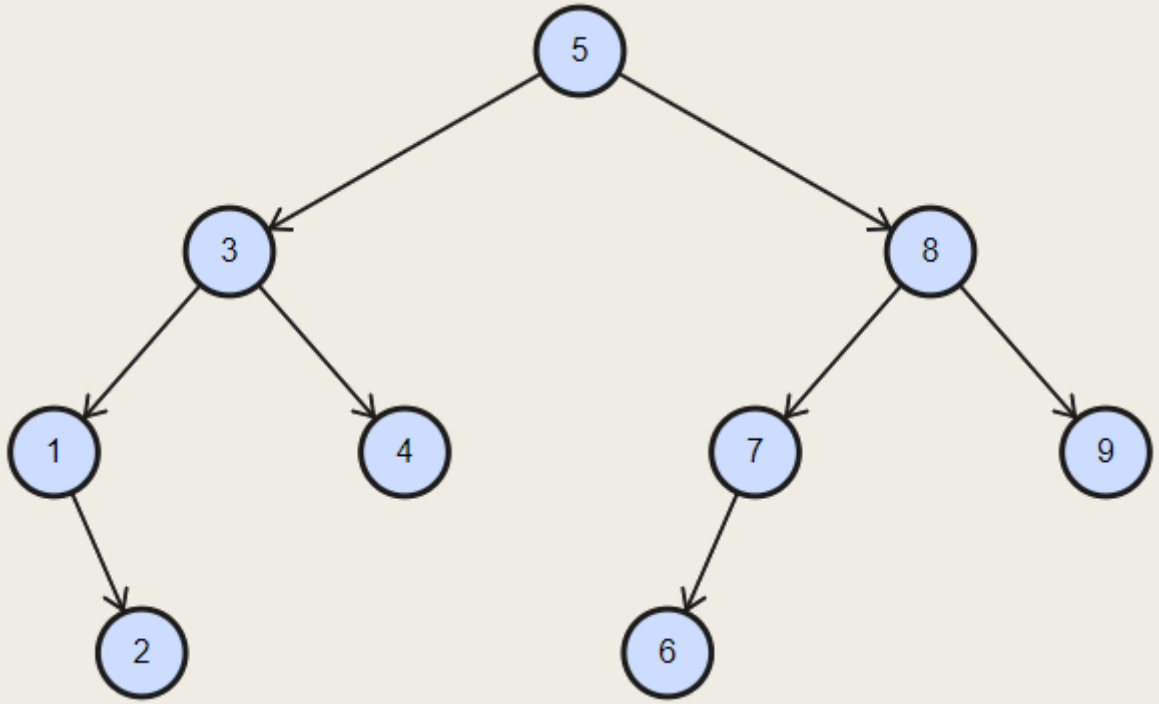
Percursos pós ordem

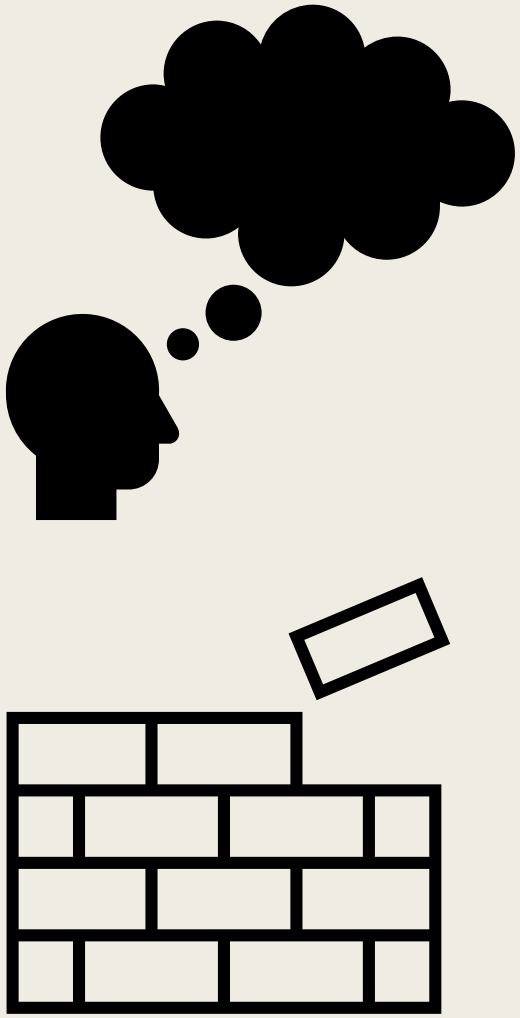


1	2	3	4	5	6	7	8	9
2	1	4	3	6	7	9	8	5

Exercício

1. Crie três funções que imprimam os valores dos nós de uma arvore binária em ***pré-ordem, em ordem e pós-ordem,*** respectivamente. Utilize a arvore dos slides anteriores como modelo para teste.





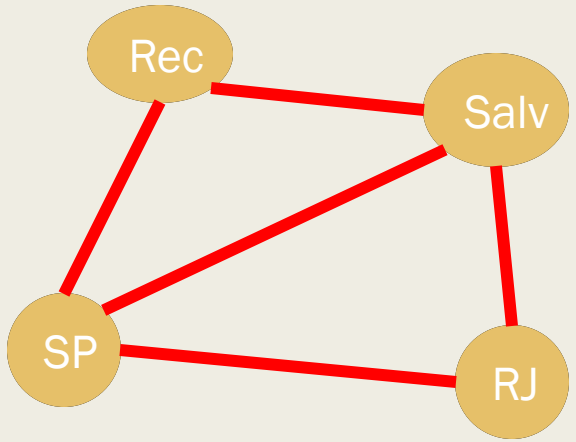
Grafos

- Conceitos.
- Aplicação.

Grafos

Grafos são formados por vértices (ou nós) e arestas, que ligam os vértices:

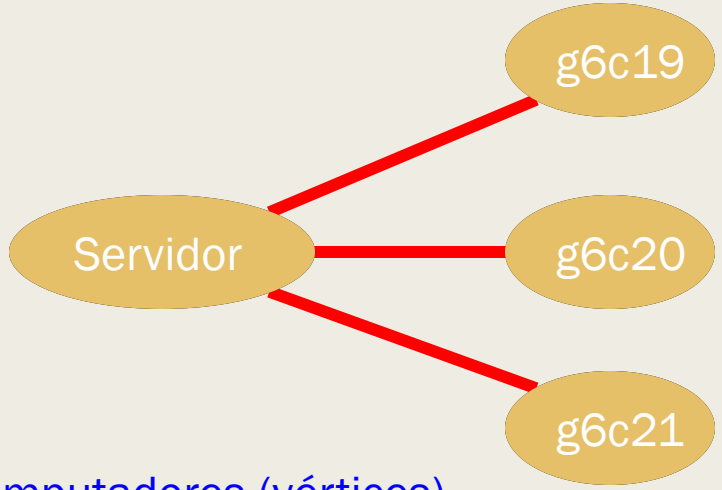
Ex: Rotas de Avião



Cidades (vértices)

Rotas (arestas)

Ex: Computadores conectados



Computadores (vértices)

Conexões (arestas)

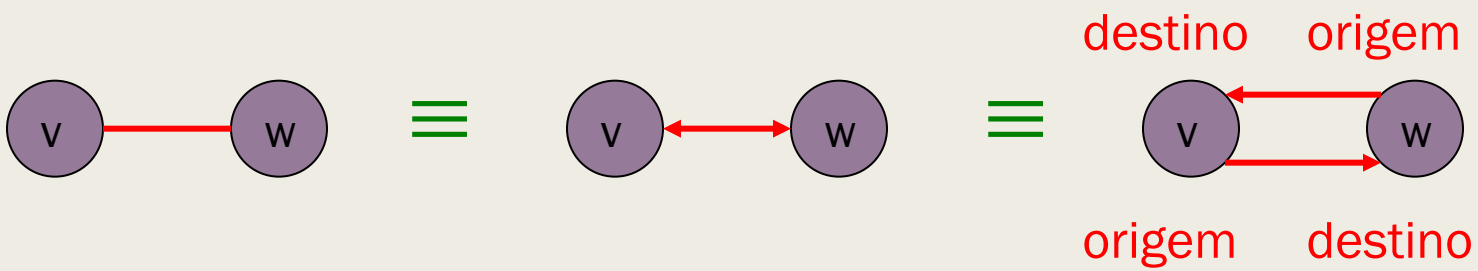
Arestas

Há dois tipos de arestas, segundo sua orientação:

- Direcionadas (*mão única*), tais que (v,w) ≠ (w,v)

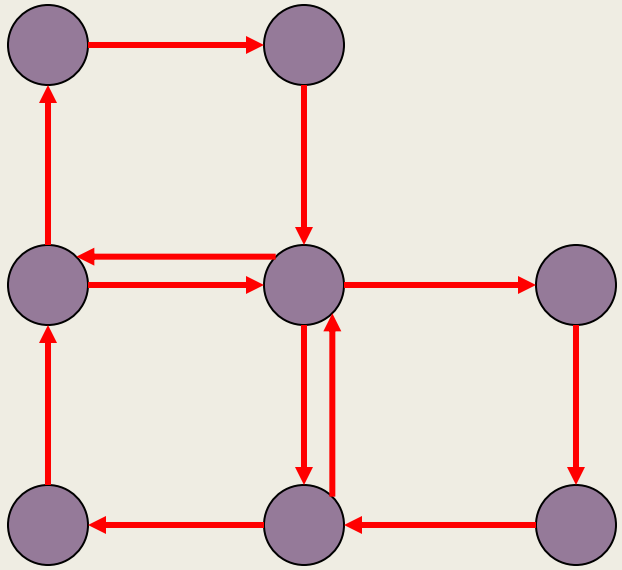


- Não-direcionadas (*mão dupla*), tais que (v,w) = (w,v)

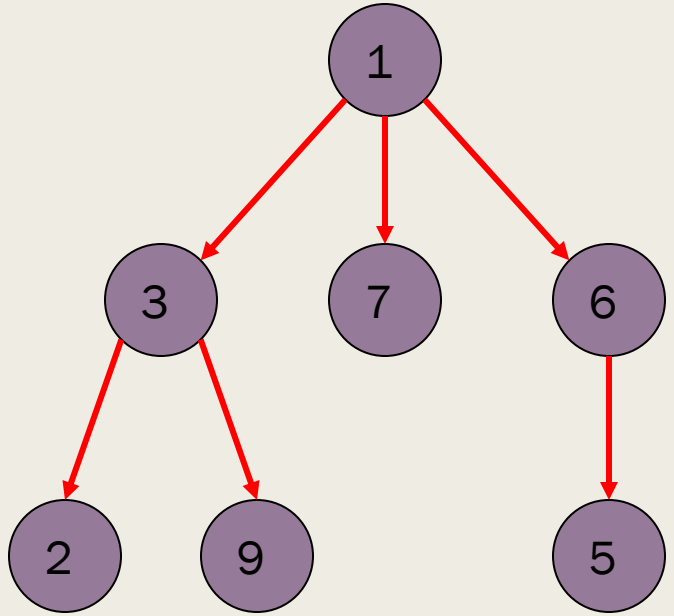


Grafos Direcionados

Cada aresta tem um sentido único, partindo do vértice de origem e chegando ao vértice de destino:



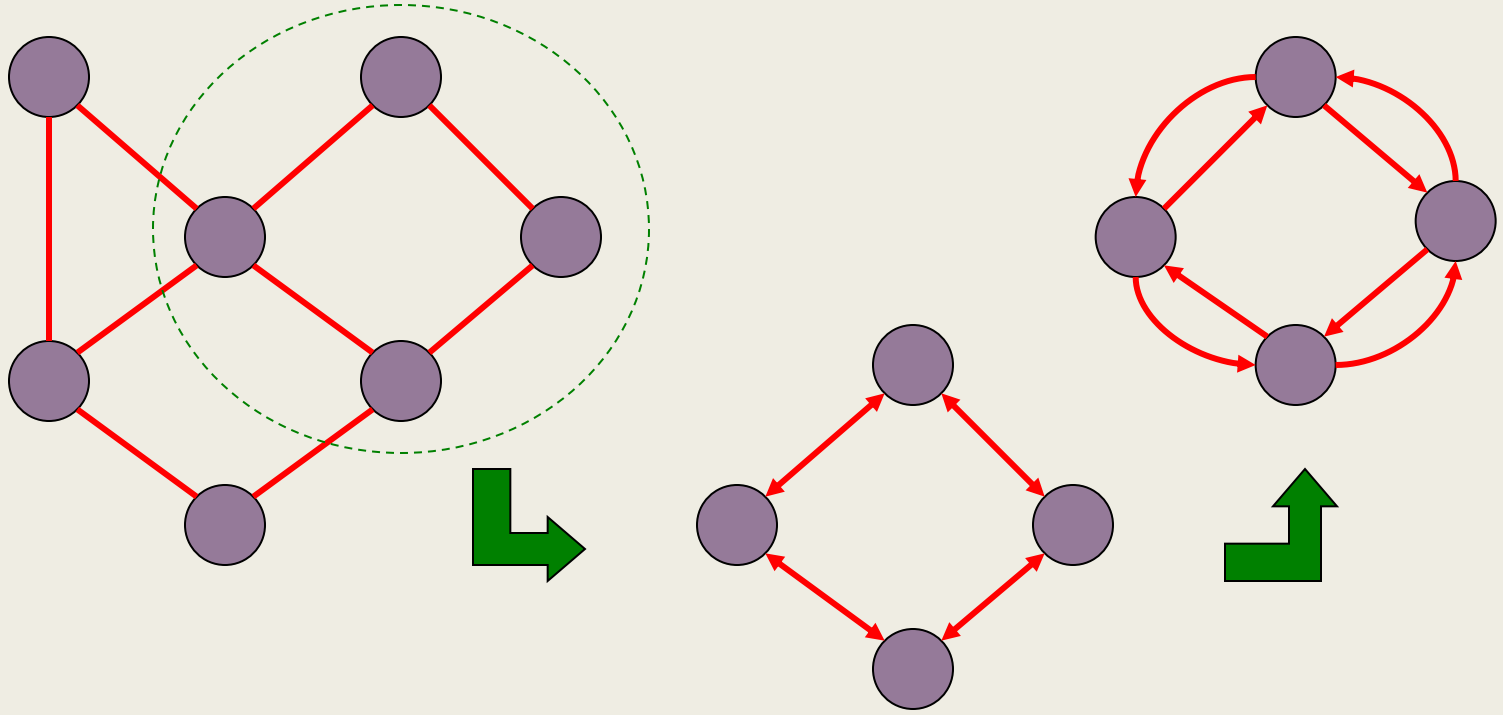
Ex: Ruas de uma cidade



Ex: Árvore (arestas pai-filho)

Grafos Não-Direcionados

Todas as arestas são de mão dupla, ou seja, não importa o sentido escolhido, e (v,w) = (w,v):



Representando Grafos

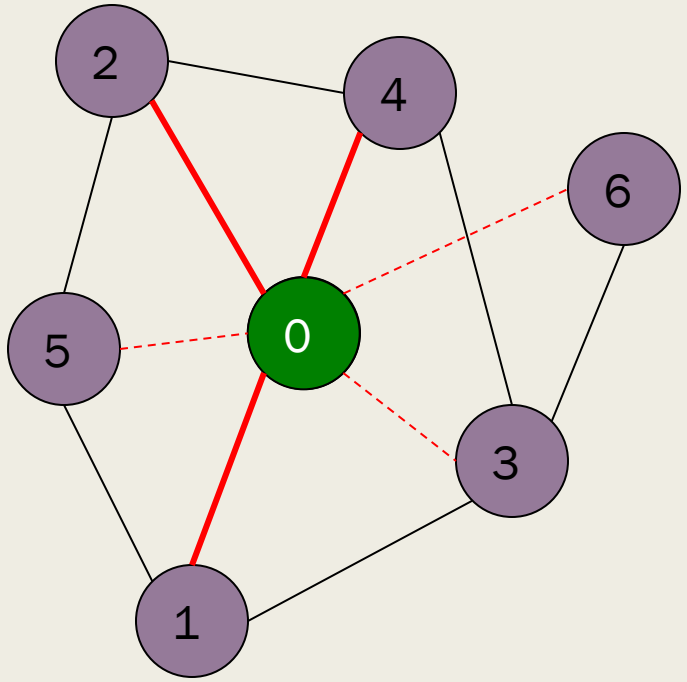
Há várias maneiras de representar um grafo. As mais comuns são:

- ***Matriz de Adjacências***
  - Relação entre pares: se dois vértices são vizinhos
  - Espaço fixo: desperdício
- ***Lista de Incidências***
  - Lista ligada de vizinhos de cada vértice
  - Acesso sequencial: ideal para buscas

Matriz de Adjacências

Se são vizinhos valor igual a 1, senão 0.

São necessárias n^2 posições:

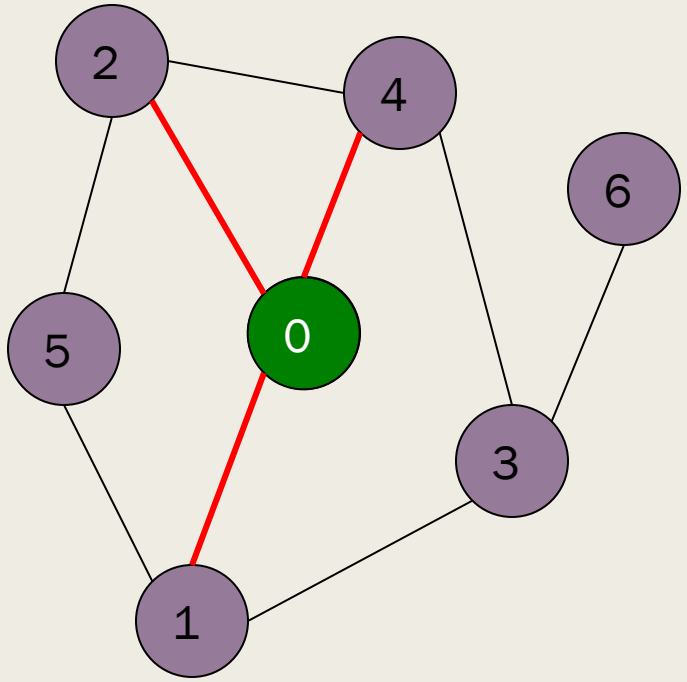


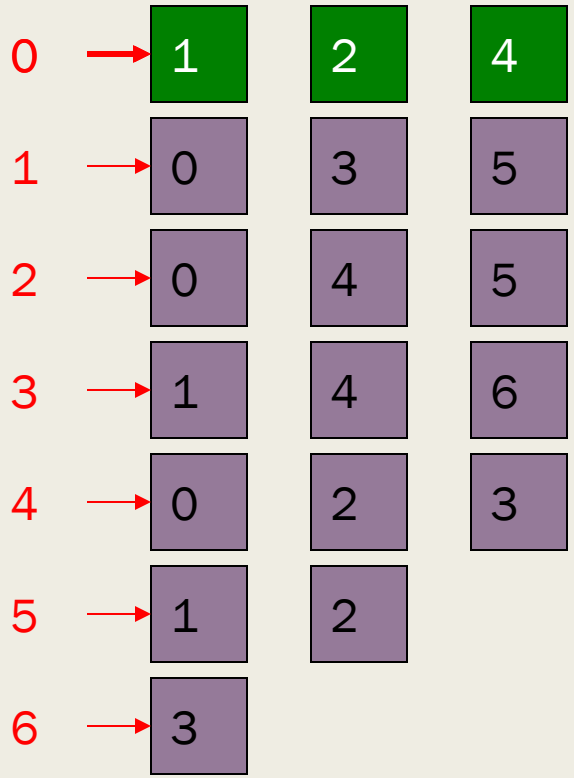
	0	1	2	3	4	5	6
0	0	1	1	0	1	0	0
1	1	0	0	1	0	1	0
2	1	0	0	0	1	1	0
3	0	1	0	0	1	0	1
4	1	0	1	1	0	0	0
5	0	1	1	0	0	0	0
6	0	0	0	1	0	0	0

Uma linha inteira para cada vértice

Lista de Incidências

Para cada vértice, uma lista contendo apenas os vértices ligados a ele:

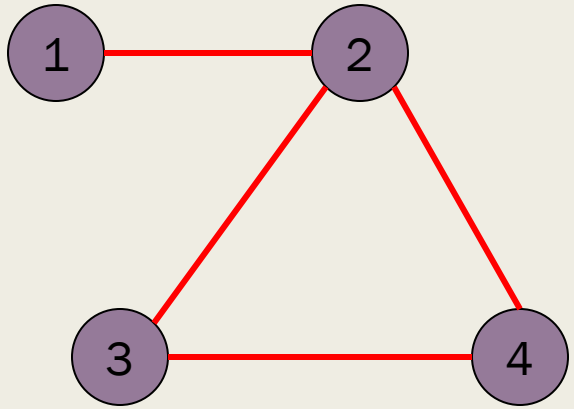




Arestas com Pesos

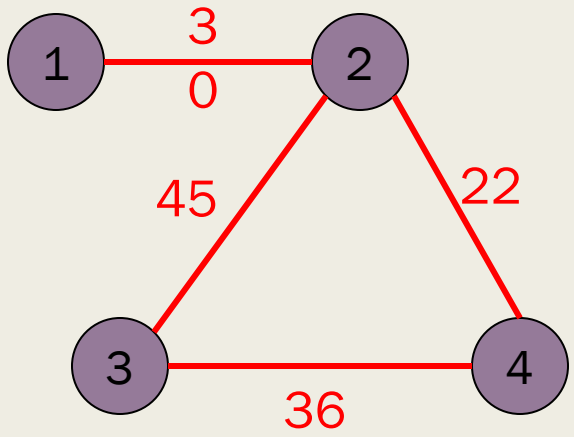
Há outra classificação para arestas:

- Arestas sem peso, com custo uniforme



Dois vértices são vizinhos?

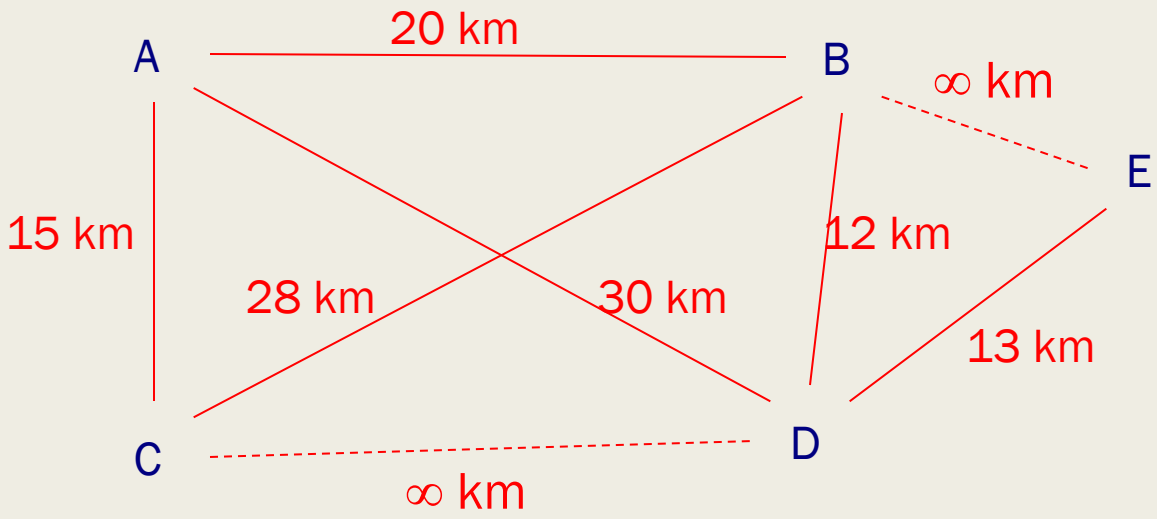
- Arestas com peso
  Ex: distâncias



Se são, qual o custo da ligação?

Grafos com Pesos

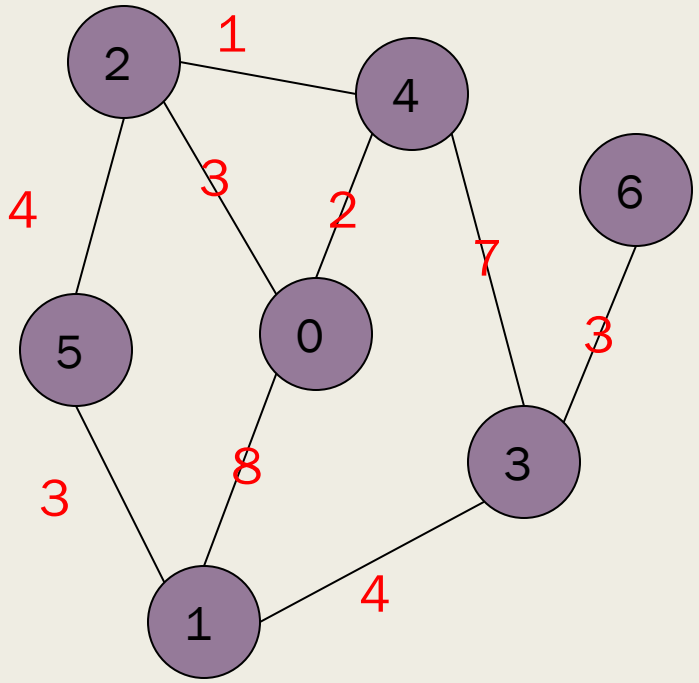
Cada aresta tem um custo (peso) associado a ela.
Ex: Mapa de uma cidade



Arestas inexistententes são arestas com custo infinito.

Matriz de Adjacências com Pesos

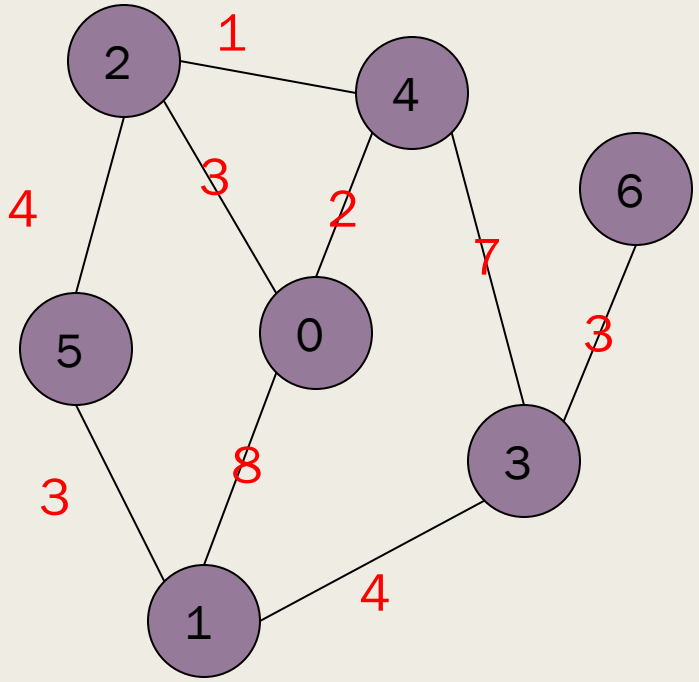
Para cada par, o custo da aresta se são vizinhos, infinito em caso contrário:

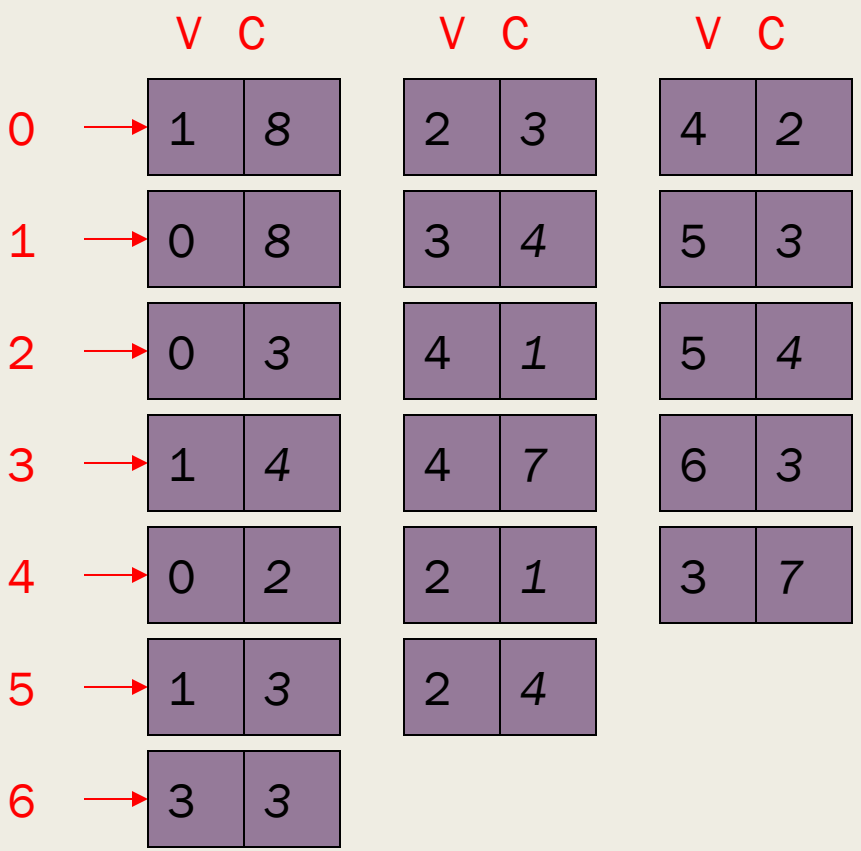


	0	1	2	3	4	5	6
0	∞	8	3	∞	2	∞	∞
1	8	∞	∞	4	∞	3	∞
2	3	∞	∞	∞	1	4	∞
3	∞	4	∞	∞	7	∞	3
4	2	∞	1	7	∞	∞	∞
5	∞	3	4	∞	∞	∞	∞
6	∞	∞	∞	3	∞	∞	∞

Lista de Incidências com Pesos

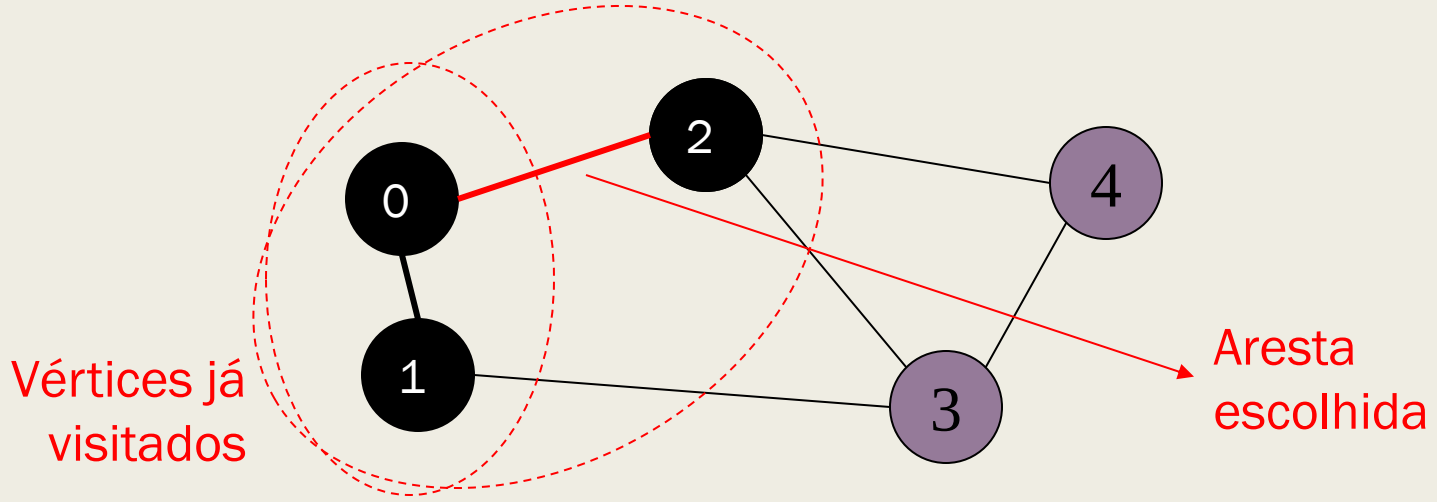
Cada nó da lista contém o vértice vizinho e o custo da aresta (sem infinitos):





Busca em Grafos

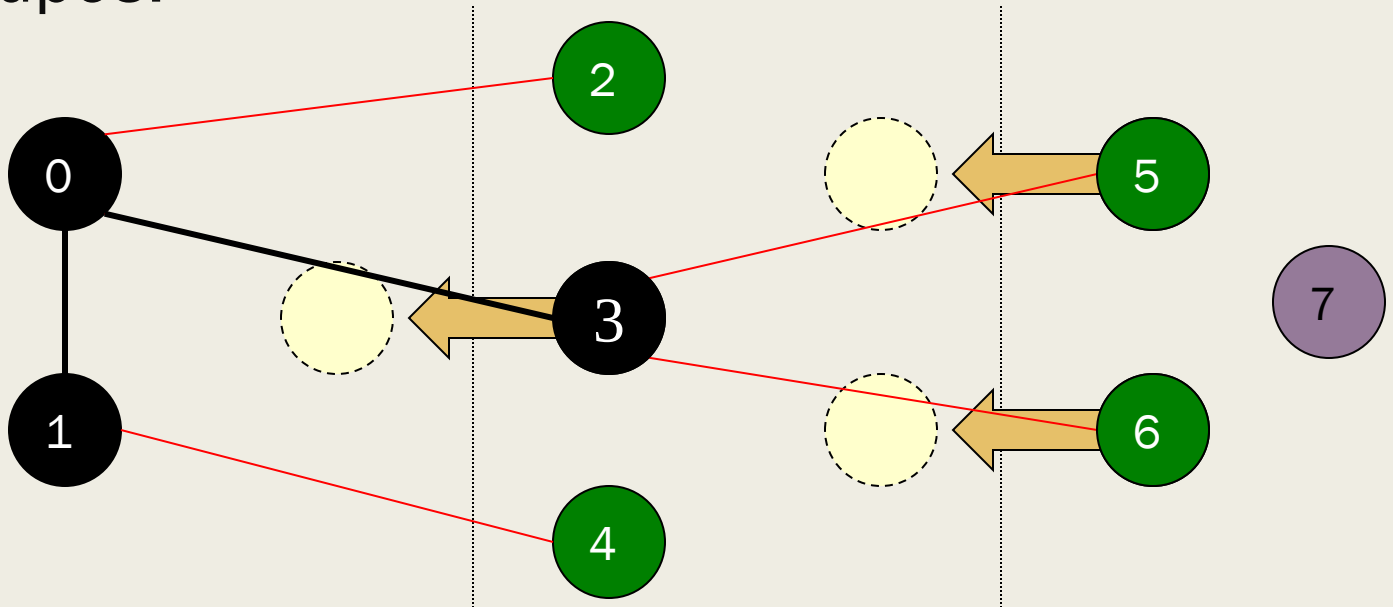
Escolhendo um vértice inicial, é possível visitar os vértices seguindo uma determinada ordem.



A cada iteração, escolhemos uma aresta que parte de um vértice já visitado.

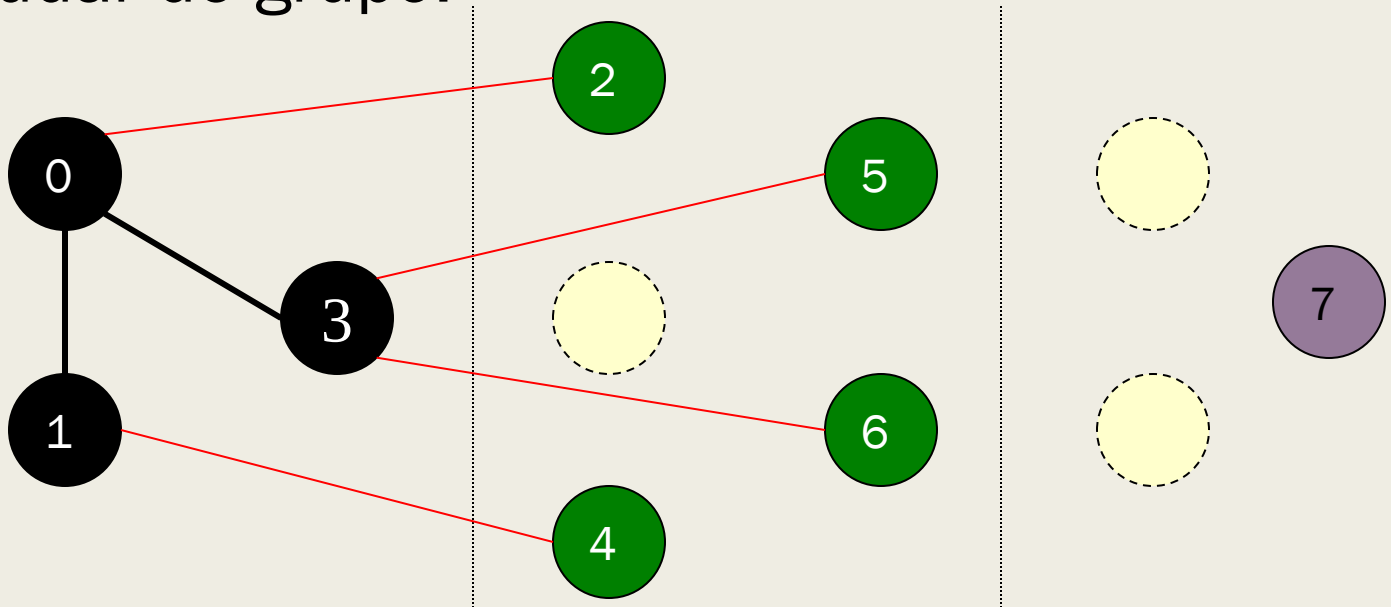
Busca em Grafos

A cada passo, os vértices são divididos em três grupos:



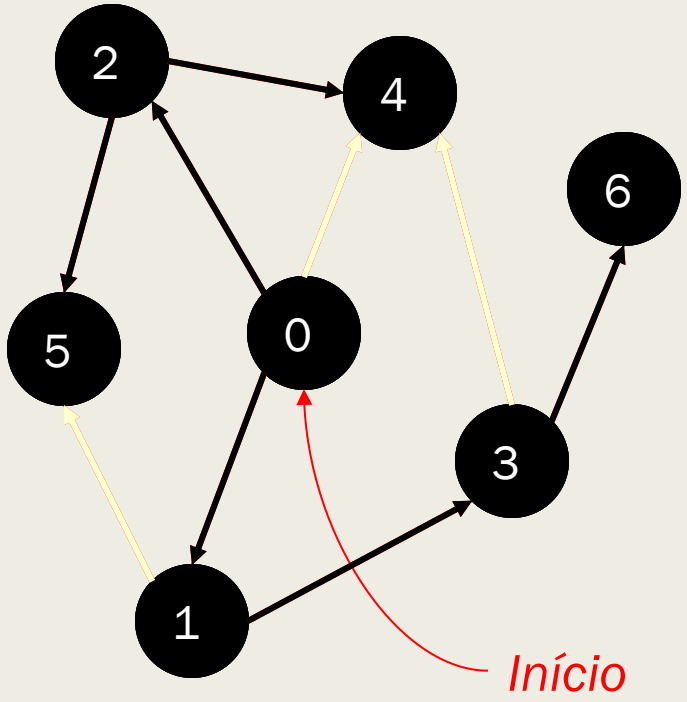
Busca em Grafos

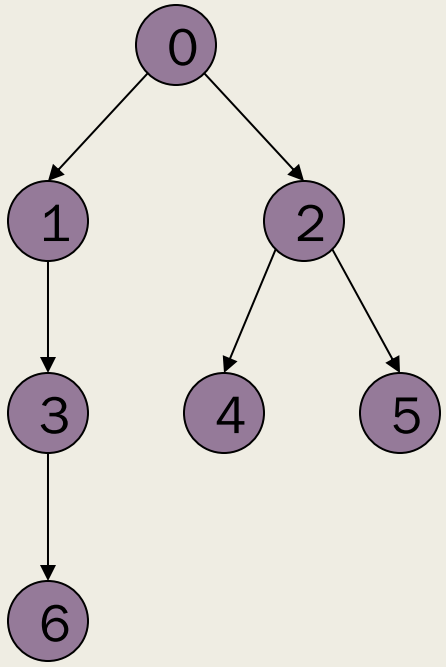
Quando uma aresta é escolhida, os vértices podem mudar de grupo:



Busca em Grafos

Utilizando as arestas escolhidas na ordem da busca, é possível montar uma árvore de busca:





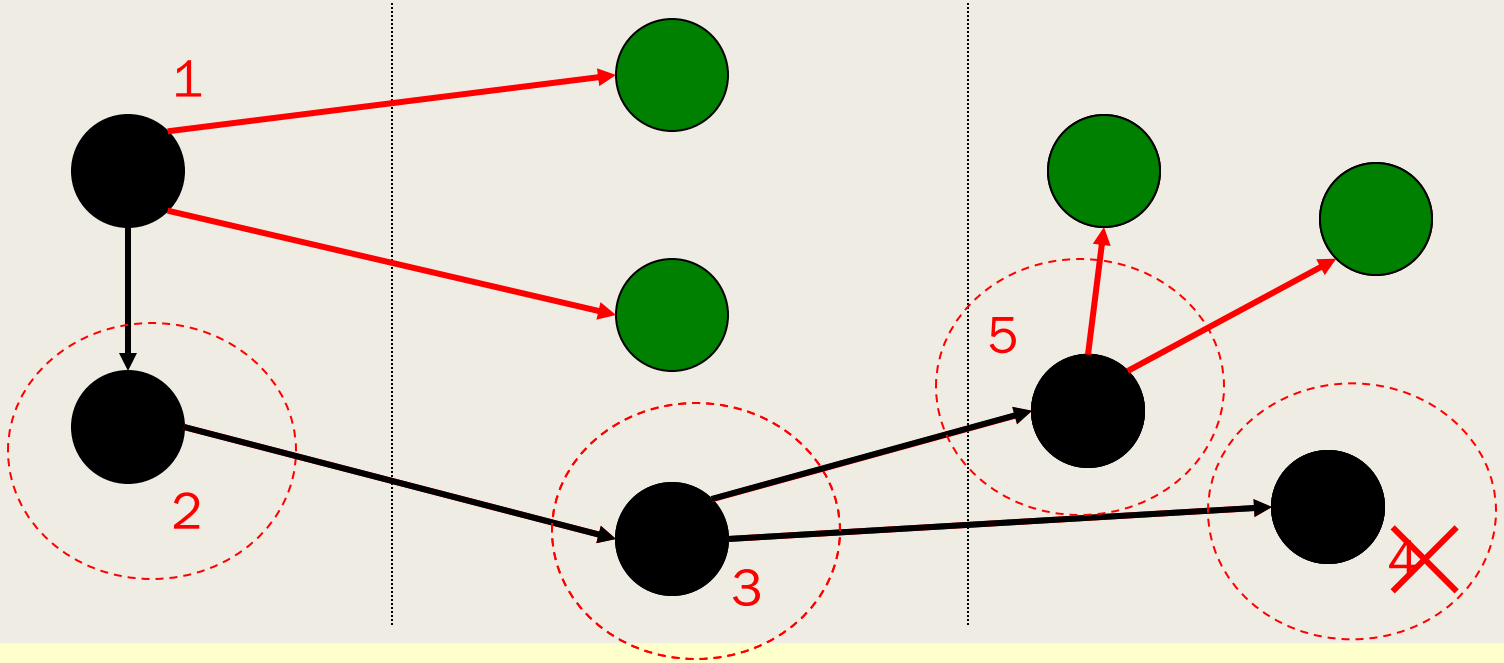
Busca em Grafos

As buscas mais comuns são:

- ***Busca em Profundidade (Depth-First Search)***
  Escolhe arestas que partem do vértice mais recente
- ***Busca em Largura (Breadth-First Search)***
  Escolhe arestas que partem do vértice mais antigo
- ***Buscas Gulosas (Greedy Search)***
  Escolhe arestas com menor custo, menor caminho, ...

Busca em Profundidade

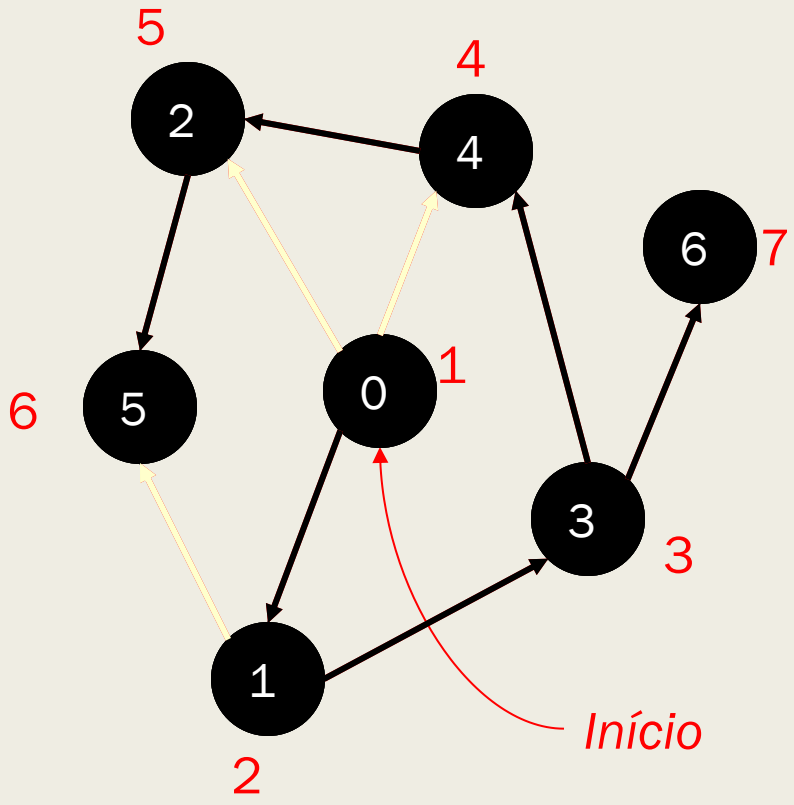
A cada passo, é escolhida uma aresta que parte do vértice escolhido que foi visitado por último:

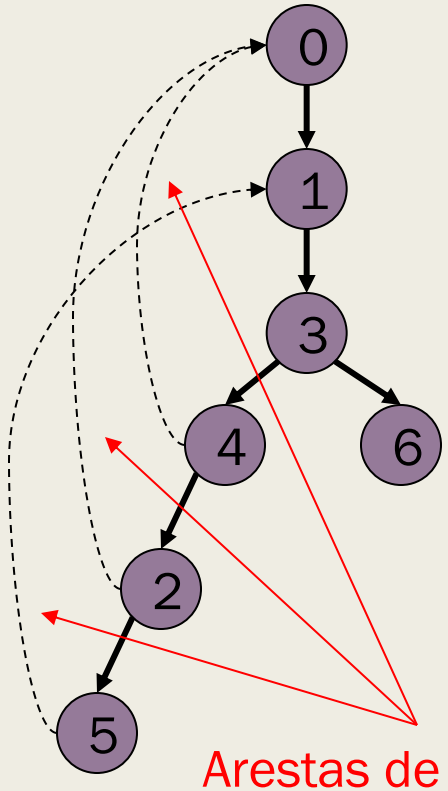


5, 6, ..., até que todos os nós tenham sido escolhidos.

Busca em Profundidade

Ex: Busca a partir do zero

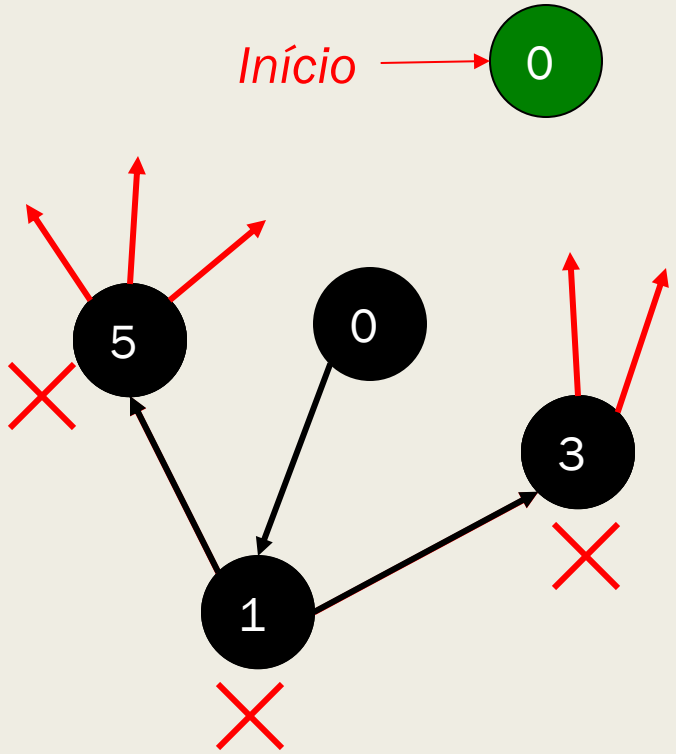




Arestas de retorno:
Retiradas do grafo para formar a árvore

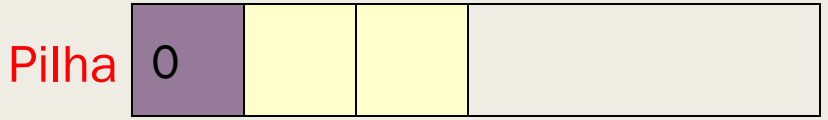
Busca em Profundidade

Pode ser implementada com recursão (pilha):



É feita uma chamada
recursiva para o vértice inicial

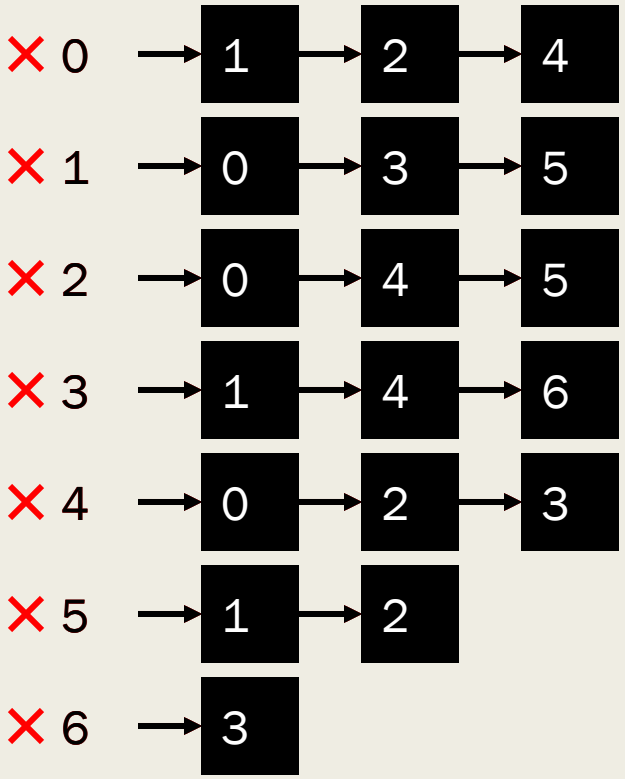
Para cada chamada, se o vértice
não estiver marcado:

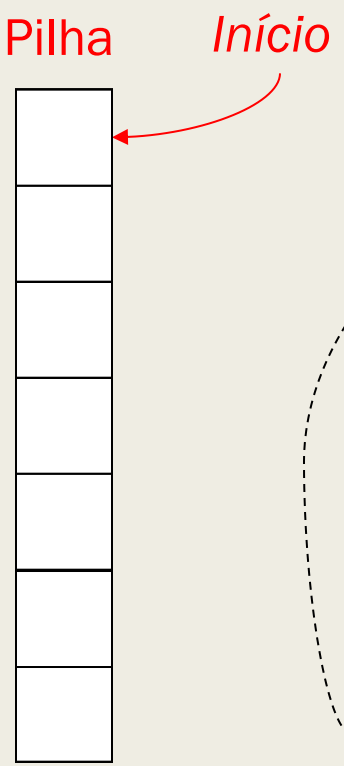


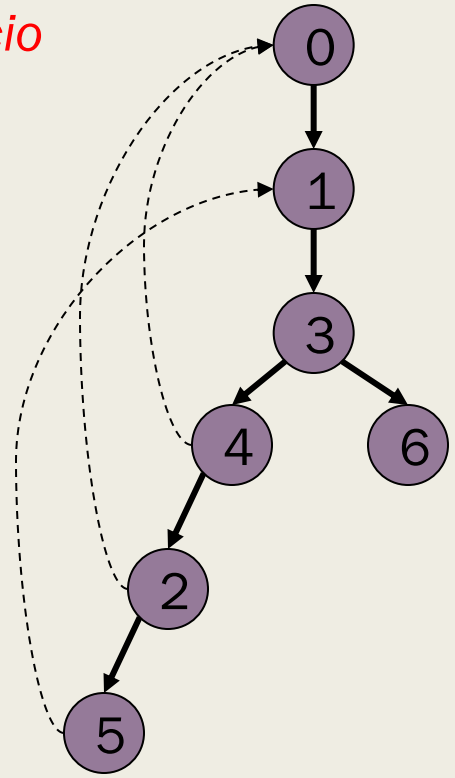
- Marcar o vértice

- Para cada vizinho não-marcado,
chamar a busca recursivamente

Busca em Profundidade







Busca em Profundidade

```
void dfs(int inicio) {
  dfsRec(inicio);
}
```

Chamada recursiva para o vértice inicial

```
void dfsRec(int v) {
  if (mark[v])
    return;

  mark[v] = true;
  NoLista temp = adj[v].inicio;
  while (temp != null) {
    dfsRec(temp.vertice);
    temp = temp.next;
  }
}
```

Se o vértice já foi visitado, retorna

Marca o vértice como visitado

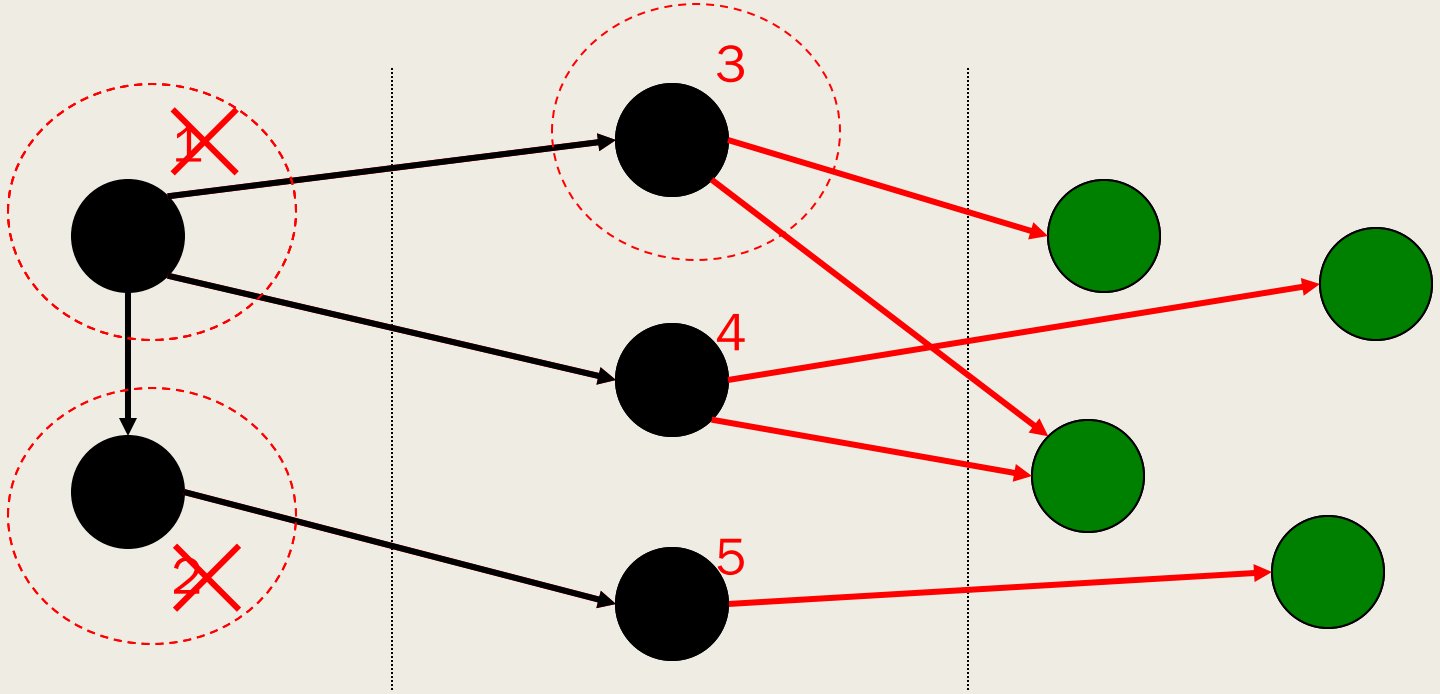
Lista de adjacências é percorrida

Busca recursiva no vizinho

Visita o próximo vizinho

Busca em Largura

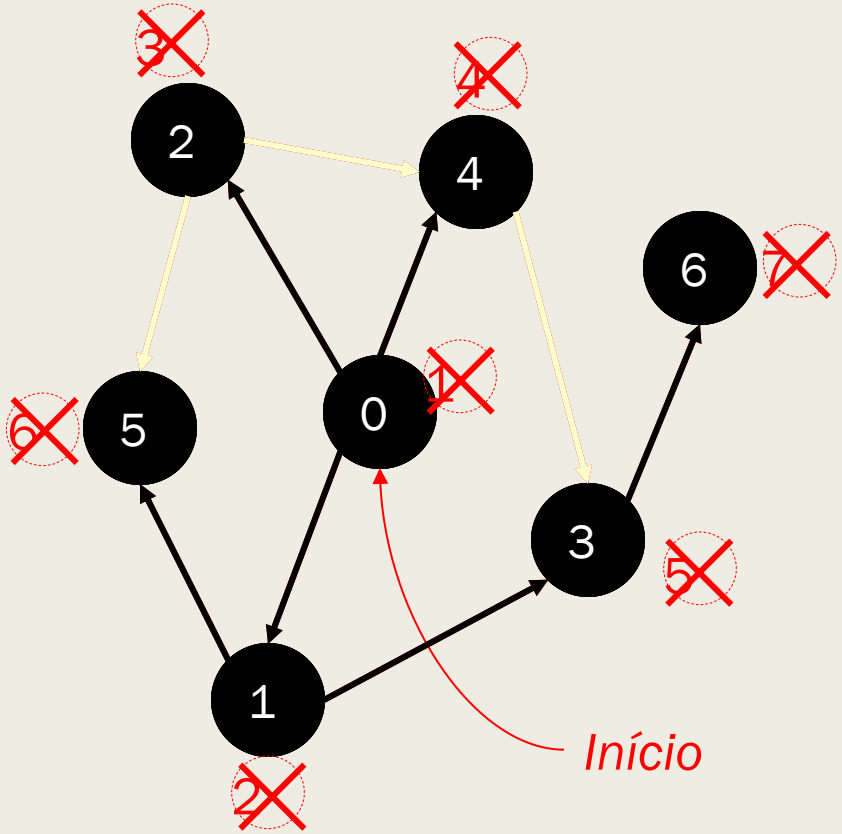
A cada passo, é escolhida uma aresta que parte do vértice escolhido que foi visitado primeiro:

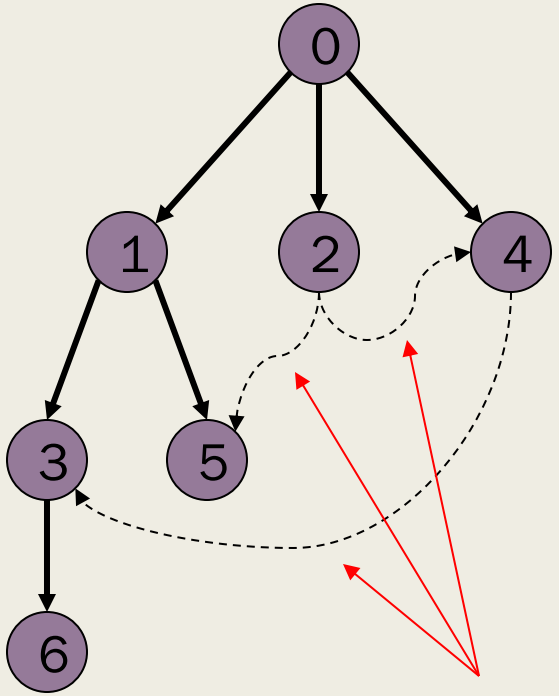


3, 4, 5, ... até que todos os nós tenham sido escolhidos.

Busca em Largura

Ex: Busca a partir do zero



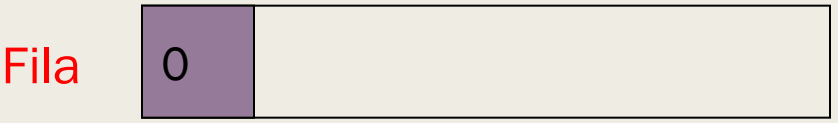


Arestas de cruzamento:
Retiradas do grafo para formar a árvore

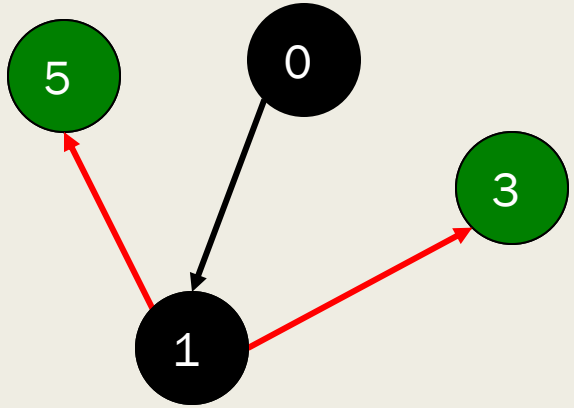
Busca em Largura

Pode ser implementada com ajuda de uma fila:





O vértice inicial é colocado na fila



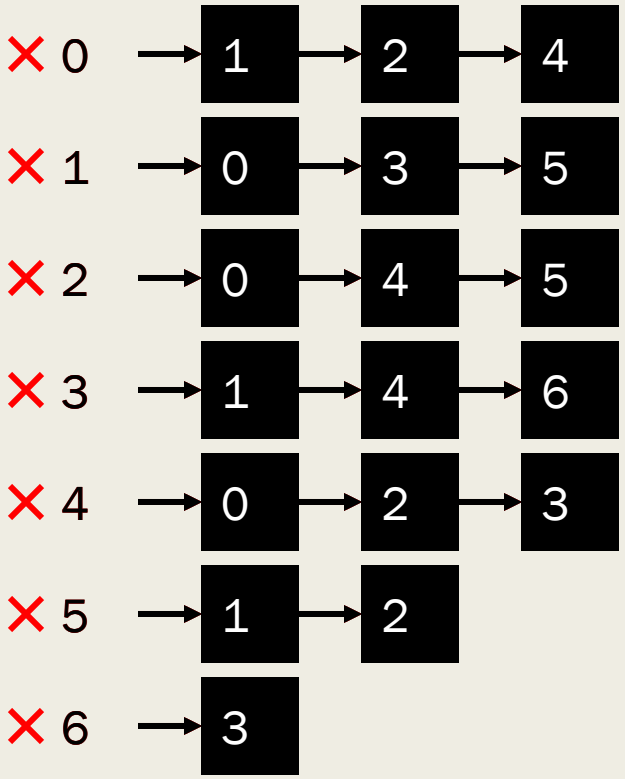
Enquanto a fila não estiver vazia...

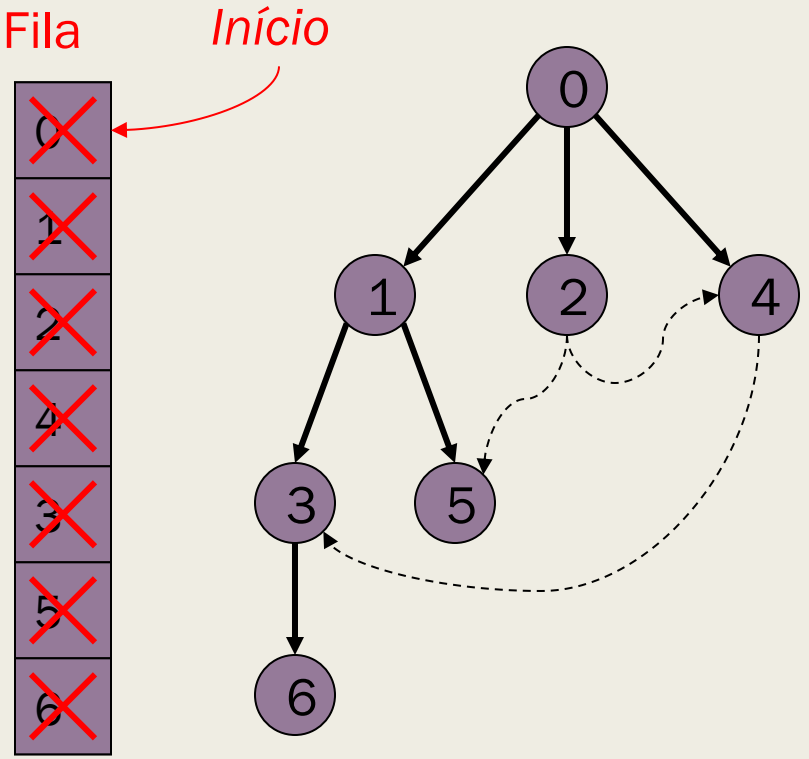


- Remover o primeiro vértice da fila

- Para cada vizinho não-visitado, visitá-lo e colocá-lo na fila

Busca em Largura





Busca em Largura

```
void bfs(int inicio) {
  fila.inserir(inicio);
  mark[inicio] = true;
  while (!fila.vazia()) {
    int v = fila.remover();
    NoLista temp = adj[v].inicio;
    while (temp != null) {
      int w = temp.vertice;
      if (!mark[w]) {
        fila.inserir(w);
        mark[w] = true;
      }
      temp = temp.next;
    }
  }
}
```

Vértice inicial é inserido na fila

Vértice inicial é marcado

Enquanto a fila não estiver vazia:

Primeiro da fila é removido

Lista de adjacências é percorrida

Se o vizinho não estiver marcado:

1. inserir na fila

2. marcar como visitado

Visita o próximo vizinho

Exercícios

- 1) Construa uma classe GMA (Grafos Matriz de Adjacências). O construtor receberá a quantidade de Vértices(Nós) do grafo e o tipo de arestas(direcional ou bidirecional) do grafo e os seguintes métodos:
  - *Adicionar Vertice(Vertice); //Adiciona um novo Vértice ao grafo.*
  - *Remover Vertice(Vertice); // Remove o vértice do grafo.*

    Verificar se aresta do grafo é direcional ou não, antes de setar a matriz de adjacências.
  - *Adicionar Arestas com Peso (Vertice Origem, Vertice Destino, Peso);*
  - *Adicionar Arestas sem Peso (Vertice Origem, Vertice Destino).*
  - *Imprime Matriz de Adjacencias(); // Imprime a matriz de adjacências*

Exercícios

- 2) Construa uma classe GLI (Grafos Lista de Incidências). O construtor receberá o tipo de arestas(direcional ou bidirecional) do grafo e os seguintes métodos:
  - *Adicionar Vertice(Vertice); //Adiciona um novo Vértice ao grafo.*
  - *Remover Vertice(Vertice); // Remove o vértice do grafo.*

    Verificar se aresta do grafo é direcional ou não, antes de setar a lista de incidências.
  - *Adicionar Arestas com Peso (Vertice Origem, Vertice Destino, Peso);*
  - *Adicionar Arestas sem Peso (Vertice Origem, Vertice Destino).*
  - *Imprime Lista de Incidências(); // Imprime a lista de incidências*

Complexidade em Algoritmos

Complexidade em Algoritmos

- **Por que se importar?**
  - Algoritmos são a espinha dorsal da tecnologia. Sua eficiência não é um detalhe, mas um pilar para a performance e usabilidade de qualquer software.
  - Desde apps de GPS que recalculam rotas em segundos até sistemas que processam "Big Data", a escolha do algoritmo certo e sua complexidade impactam diretamente a experiência do usuário e a viabilidade de um produto.

Complexidade em Algoritmos

- **Custo Computacional**
  - *Algoritmos são avaliados pelo tempo de execução e uso de memória em função da quantidade de entrada, N.*
  - *Com o número de operações básicas (comparações, atribuições) variando em função de N.*
  - *Com a função de tempo* $\boldsymbol{f(N)}$ *pondendo ser utilizada para representar o comportamento do algoritmo.*
  - *Por exemplo:* $\boldsymbol{f(N) = 3N^2 + 10N + 4}$*representa o tempo consumido por um algoritmo, dado um conjunto de entradas* $\boldsymbol{N}$*.*

Complexidade em Algoritmos

- **Análise Assintótica**
  - A função $\boldsymbol{f(N) = 3N^2 + 10N + 4}$, *pode ser simplificada ignorando constantes e termos de menor ordem, pois o termo dominante dita a taxa de crescimento.*
  - Dessa forma na função $\boldsymbol{f(N)}$ o termo de crescimento relevante será $\boldsymbol{N^2}$.
  - Importante observar que a função $\boldsymbol{f(N)}$ *não calcula um tempo determinado, mas sim oferece um termo de comparação entre diferentes algoritmos.*

Complexidade em Algoritmos

- Notação Assintótica
  - $\boldsymbol{BIG}$ ($\boldsymbol{O}$)**:** *Descreve o pior caso (limite superior), garantindo que o algoritmo não levará mais tempo do que* $\boldsymbol{O(f(N))}$**.**
  - Ô$\boldsymbol{mega}$ (**Ω):** *Descreve o melhor caso (limite inferior), garantindo que o algoritmo não levará menos tempo do que* $\boldsymbol{\Omega(f(N))}$**.**
  - $\boldsymbol{Theta}$ (**Θ):** *Descreve o valor médio entre o pior e o melhor caso.*
  - *Na prática, o foco é no* **Big O (pior caso)** *para garantir robustez.*

Complexidade em Algoritmos

O(1) - Constante
"Sempre rápido!" Ex: Acessar array por índice.

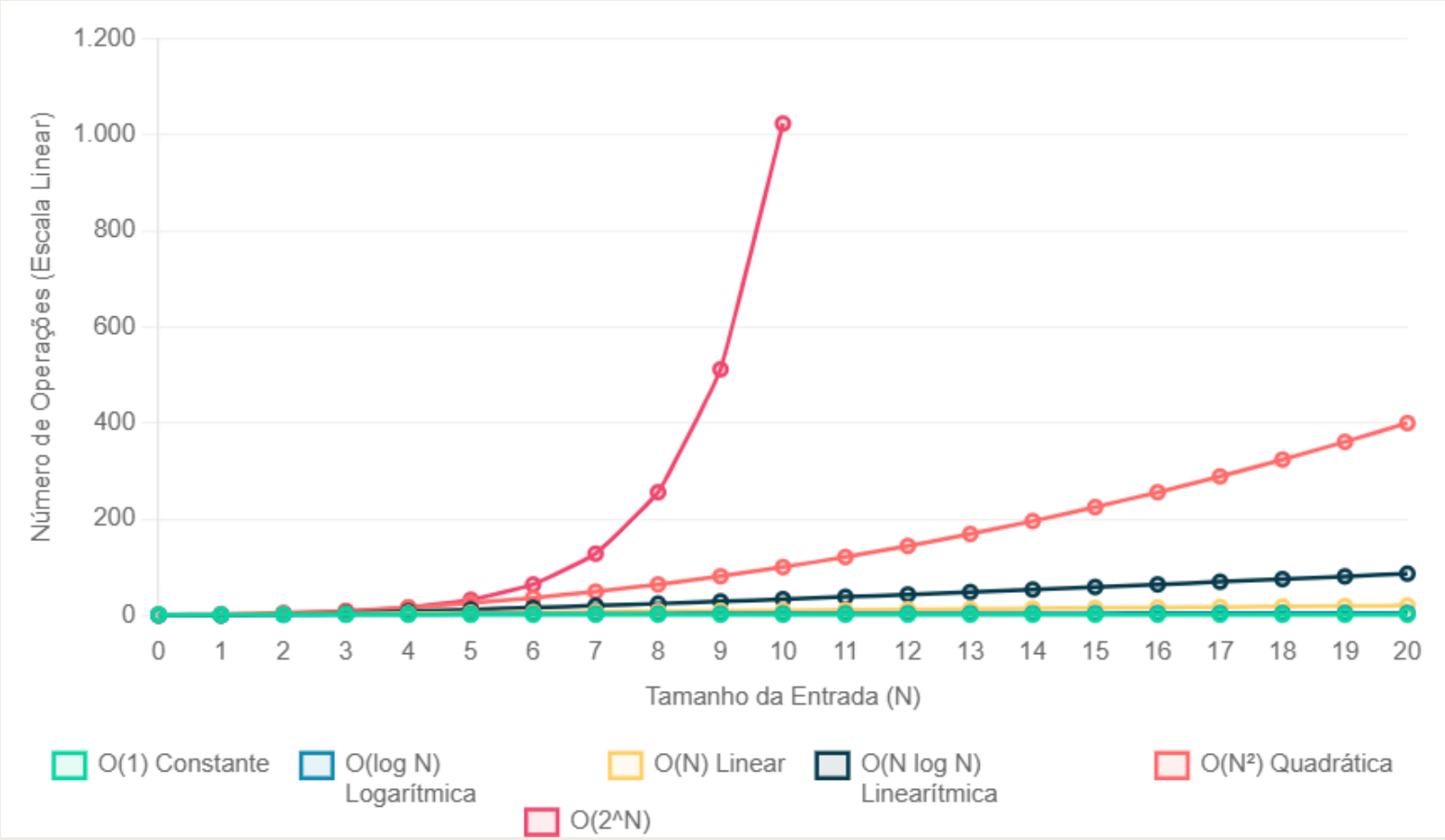
O(log N) - Logarítmica
"Divide e conquista." Ex: Busca Binária.

O(N) - Linear
"Um passo por item." Ex: Busca Linear.

O(N log N) - Linearítmica
"Ordenação esperta." Ex: Merge Sort.

O(N^2) - Quadrática
"Item com todos." Ex: Bubble Sort.

O(2^N) - Exponencial
"Crescimento explosivo!" Ex: Fibonacci recursivo ingênuo.



Métodos de Busca



Busca Linear



Busca Binária

Busca Linear

- A forma mais simples de se consultar um vetor em busca de um item particular é, a partir do seu início.
- Ir examinando cada um de seus itens até que o item desejado seja encontrado ou então que seu final seja atingido.

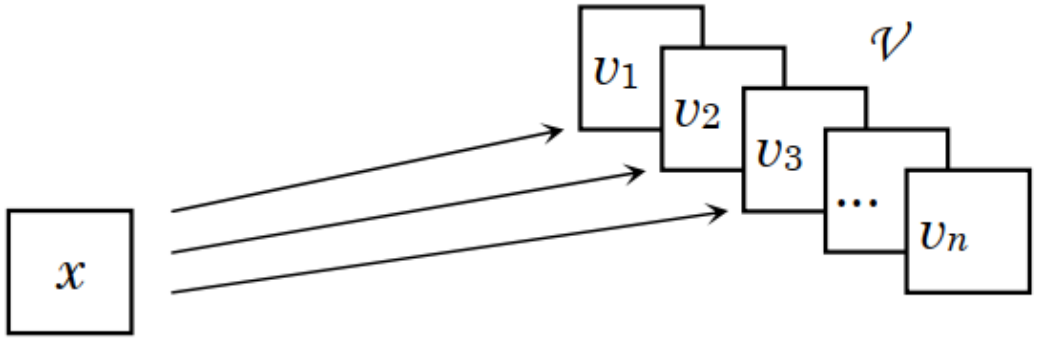


Figura 1 – *O método de busca linear*

Busca Linear

- A **vantagem** da busca linear é que ela sempre funciona, independentemente do vetor estar ou não ordenado.
- A **desvantagem** é que ela é geralmente muito lenta; pois, para encontrar um determinado item x, a busca linear precisa examinar todos os itens que precedem x no vetor.
- Dizemos então que o tempo gasto por esse algoritmo é da ordem de ***n***, isto é, ***O(n).***

Busca linear (*linear search*) num vetor de inteiros.

```
funcao buscaLinear(lista, itemProcurado):
  para cada elemento em lista:
    se elemento == itemProcurado: retorne POSICAO
  retorne "não encontrado"
```

Busca Binária

- Esse método é semelhante àquele que usamos quando procuramos uma palavra num dicionário:
  - *Primeiro abrimos o dicionário numa página aproximadamente no meio;*
  - *se tivermos sorte de encontrar a palavra nessa página, ótimo;*
  - *senão, verificamos se a palavra procurada ocorre antes ou depois da página em que abrimos e então continuamos*

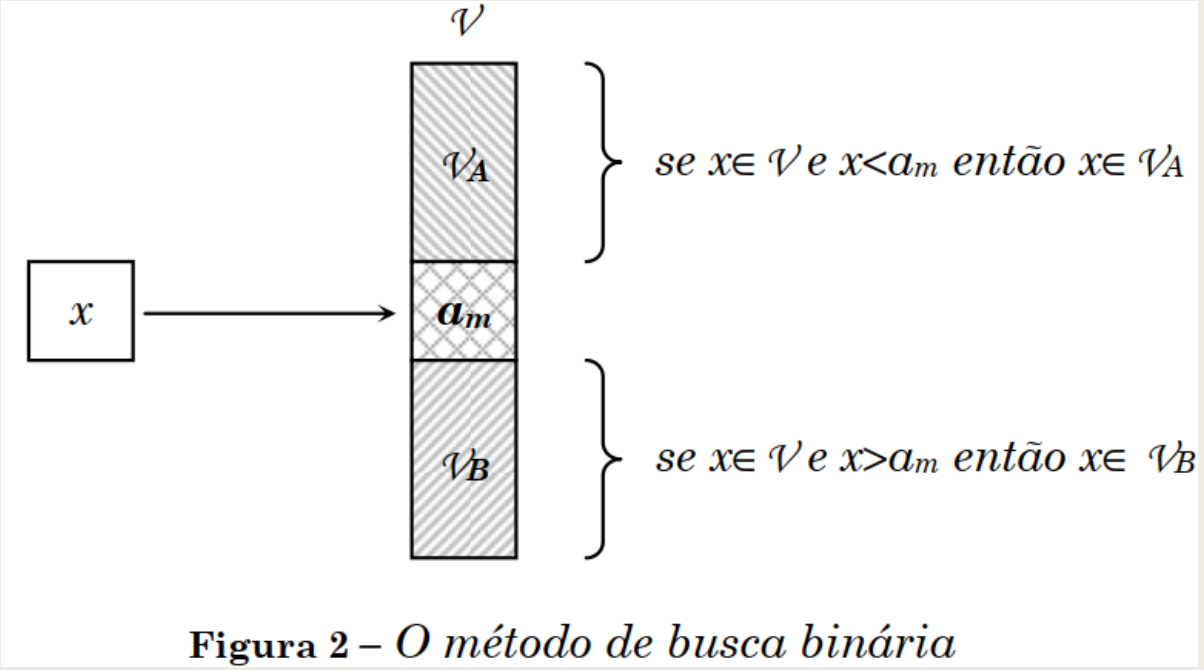
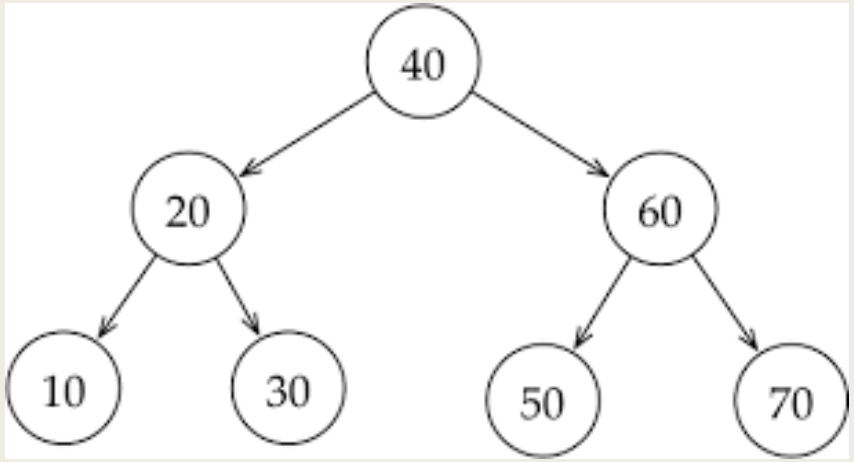


Figura 2 – *O método de busca binária*

Busca Binária

- A busca Binária exige que o vetor esteja ordenado para garantir a eficiência no processo de busca.
- Como a cada comparação realizada o espaço de busca reduz-se aproximadamente à metade.
- A quantidade de comparações necessária para realizar a busca é reduzido.



Busca Binária

Busca binária (*binary search*) num vetor de inteiros.

```
funcao buscaBinaria(listaOrdenada, itemProcurado):
  inicio = 0
  fim = tamanho_da_lista - 1
  enquanto inicio <= fim:
    meio = (inicio + fim) / 2 // Calcula o índice do meio
    se listaOrdenada[meio] == itemProcurado:
      retorne a POSIÇÃO meio // Item encontrado
    senão se listaOrdenada[meio] < itemProcurado:
      inicio = meio + 1 // Item está na metade direita
    senão:
      fim = meio - 1     // Item está na metade esquerda
  retorne "não encontrado" // Item não existe na lista
```

Algoritmos de Ordenação

- Ordenação por Trocas *(bubble sort)*.
- Ordenação por Seleção *(selection sort)*
- Ordenação por Inserção *(insert sort)*

Ordenação por Trocas (*bubble sort)*

Talvez a estratégia mais simples para ordenar um vetor seja comparar pares de itens consecutivos e permutá-los, caso estejam fora de ordem.

Se o vetor for assim processado, sistematicamente, da esquerda para a direita, um item máximo será deslocado para sua última posição.

Exemplo

$v[1]$	$v[2]$	$v[3]$	$v[4]$	$v[5]$	$v[6]$	$v[7]$	$v[8]$	$v[9]$
71	**63**	46	80	39	92	55	14	27
63	**71**	**46**	80	39	92	55	14	27
63	46	**71**	**80**	39	92	55	14	27
63	46	71	**80**	**39**	92	55	14	27
63	46	71	39	**80**	**92**	55	14	27
63	46	71	39	80	**92**	**55**	14	27
63	46	71	39	80	55	**92**	**14**	27
63	46	71	39	80	55	14	**92**	**27**
63	46	71	39	80	55	14	27	**92**

Ordenação por Trocas

À medida em que o vetor vai sendo processado, cada número vai sendo deslocado para a direita, até que seja encontrado outro maior.

Evidentemente, no final do processo, um valor máximo estará na última posição do vetor.

Exemplo

$v[1]$	$v[2]$	$v[3]$	$v[4]$	$v[5]$	$v[6]$	$v[7]$	$v[8]$	$v[9]$
71	**63**	46	80	39	92	55	14	27
63	**71**	**46**	80	39	92	55	14	27
63	46	**71**	**80**	39	92	55	14	27
63	46	71	**80**	**39**	92	55	14	27
63	46	71	39	**80**	**92**	55	14	27
63	46	71	39	80	**92**	**55**	14	27
63	46	71	39	80	55	**92**	**14**	27
63	46	71	39	80	55	14	**92**	**27**
63	46	71	39	80	55	14	27	**92**

Ordenação por Trocas

A variável i indica a fase da ordenação e j indica a posição do par de itens consecutivos que serão comparados e, eventualmente, permutados.

Apesar de simples esse método é pouco eficientes, não sendo recomendável para ordenação de grandes vetores.

Com sua complexidade em anotação *BigO* sendo $O(n^2)$.

Exemplo

fase	*i*	*j*	*v*[1]	*v*[2]	*v*[3]	*v*[4]	*v*[5]
1º	1	1	**46**	**39**	55	14	27
		2	39	**46**	**55**	14	27
		3	39	46	**55**	**14**	27
		4	39	46	14	**55**	**27**
			39	46	14	27	**55**
2º	2	1	**39**	**46**	14	27	**55**
		2	39	**46**	**14**	27	**55**
		3	39	14	**46**	**27**	**55**
			39	14	27	**46**	**55**
3º	3	1	**39**	**14**	27	**46**	**55**
		2	14	**39**	**27**	**46**	**55**
			14	27	**39**	**46**	**55**
4º	4	1	**14**	**27**	**39**	**46**	**55**
			14	**27**	**39**	**46**	**55**

Ordenação por Trocas

```
funcao bubbleSort(lista):
 n = tamanho_da_lista
 para i de 0 até n-2:
   para j de 0 até n-i-2:
     se lista[j] > lista[j+1]:
       troque lista[j] com lista[j+1]
 retorne lista
```

Ordenação por Inserção *(insert sort)*

- Para ordenar uma sequência de números, o método de inserção a divide em duas partes: uma ordenada (inicialmente vazia) e outra desordenada.
- A cada passo, um número é retirado da parte desordenada e inserido na parte ordenada, na posição correta para que ela continue ordenada.
- À medida em que o processo se desenvolve, a sub sequencia desordenada vai diminuindo, enquanto a ordenada vai aumentando.

Ordenação por Inserção

- Inicialmente, a parte ordenada ⟨34⟩ e a desordenada ⟨17, 68, 29, 50, 47⟩.
- Então, o item 17 é removido da parte desordenada e a posição liberada fica disponível no final da parte ordenada.
- Em seguida, o item 17 é comparado ao 34 que, sendo maior, é deslocado para a direita.
- Como não há mais itens, o 17 é inserido na posição liberada pelo item 34.
- Ao final dessa fase, a parte ordenada tem um item a mais e a desordenada, um item a menos...

Ordenação de um vetor v usando inserção.

i	v_1	v_2	v_3	v_4	v_5	v_6
1	**34**	**17**	68	29	50	47
2	**17**	**34**	**68**	29	50	47
3	**17**	**34**	**68**	**29**	50	47
4	**17**	**29**	**34**	**68**	**50**	47
5	**17**	**29**	**34**	**50**	**68**	**47**
	17	**29**	**34**	**47**	**50**	**68**

Ordenação por Inserção

- A complexidade de tempo da ordenação por inserção é $O(n^2)$.
- Entretanto, ao contrário do que acontece com os métodos de trocas e de seleção onde o número de comparações é fixo, na seleção ele é variável.
- Isso quer dizer que, no caso médio, o método da inserção pode ser um pouco mais eficiente que os outros dois.

Ordenação de um vetor v usando inserção.

i	v_1	v_2	v_3	v_4	v_5	v_6
1	**34**	**17**	68	29	50	47
2	**17**	**34**	**68**	29	50	47
3	**17**	**34**	**68**	**29**	50	47
4	**17**	**29**	**34**	**68**	**50**	47
5	**17**	**29**	**34**	**50**	**68**	**47**
	17	**29**	**34**	**47**	**50**	**68**

Ordenação por Inserção

```
funcao insertionSort(lista):
 n = tamanho_da_lista
 para i de 1 até n-1:
   chave = lista[i]
   j = i - 1
   enquanto j >= 0 e chave < lista[j]:
     lista[j+1] = lista[j]
     j = j - 1
   lista[j+1] = chave
 retorne lista
```

Ordenação por Seleção *(selection sort)*

- A estratégia básica desse método é selecionar o item de menor valor ainda não ordenado e permutá-lo com aquele que ocupa a sua posição na sequência ordenada.

Ordenação por Seleção

Em cada Fase o elemento com menor valor, apontado pelo índice ***k*** é selecionado para ser substituído pelo elemento apontado por ***i.***

Ordenação da seqüência ⟨46, 55, 59, 14, 38, 27⟩ usando seleção.

Fase	*i*	*k*	a_1	a_2	a_3	a_4	a_5	a_6
1º	1	4	**46**	55	59	**14**	38	27
2º	2	6	**14**	**55**	59	46	38	**27**
3º	3	5	**14**	**27**	**59**	46	**38**	55
4º	4	4	**14**	**27**	**38**	**46**	59	55
5º	5	6	**14**	**27**	**38**	**46**	**59**	**55**
			14	**27**	**38**	**46**	**55**	59

Ordenação por Seleção

O fato do espaço de busca da rotina de *SelecionaMenorValor* diminuir a cada interação torna a complexidade desse algoritmo menor que a Ordenação por Troca.

Sendo $\boldsymbol{O(n)}$

Ordenação da seqüência ⟨46, 55, 59, 14, 38, 27⟩ usando seleção.

Fase	i	k	a_1	a_2	a_3	a_4	a_5	a_6
1º	1	4	**46**	55	59	**14**	38	27
2º	2	6	**14**	**55**	59	46	38	**27**
3º	3	5	**14**	**27**	**59**	46	**38**	55
4º	4	4	**14**	**27**	**38**	**46**	59	55
5º	5	6	**14**	**27**	**38**	**46**	**59**	**55**
			14	**27**	**38**	**46**	**55**	59

Ordenação por Seleção

```
funcao selectionSort(lista):
  n = tamanho_da_lista
  para i de 0 até n-2:
    indice_menor = i
    para j de i+1 até n-1:
      se lista[j] < lista[indice_menor]:
        indice_menor = j
    troque lista[i] com lista[indice_menor]
  retorne lista
```

Exercício Busca e Ordenação

1) Implemente o algoritmo de Busca Linear.
2) Implemente o algoritmo de Busca Binária.
3) Implemente o algoritmo de Ordenação por Troca.
4) Implemente o algoritmo de Ordenação por Inserção.
5) Implemente o algoritmo de Ordenação por Seleção.
6) Compare o desempenho dos diferentes algoritmos.

Bibliografia

- **Estrutura de Dados e Técnicas de Programação**
  - **Piva D.J.; Nakamiti, G. S.; Bianchi, F. et (2014)**
- **Histórico da Computação e Principais Componentes Computacionais**
  - https://materialpublic.imd.ufrn.br/curso/disciplina/4/14/1
- **Estruturas de Dados em Java (Apostila)**
  - Prof. Dr. Paulo Roberto Gomes Luzzardi (2010).
- **Estruturas de Dados Abertas(Livro)**
  - Pat Morin e Joao Araujo (2021)
- **Algoritmos e Estrutura de Dados em Java (Apostila)**
  - Caelum ensino e inovação. (2021)